

C++ Game Development with SFML

As a Foundation for Reinforcement Learning

A Beginner-Friendly Journey from the Game Loop
to Custom RL Environments – Coin Collector Basic RL Environment –

by S K VIGNESHWAR

Topics covered:

- C++ and SFML setup on Ubuntu
- Game loops and event handling
- Rendering and movement
- Delta time
- Vectors and normalization
- Collision detection
- Obstacles and enemies
- Coin collection
- Random number generation
- Object-oriented game architecture
- State spaces, actions, rewards and DQN

Contents

1	Introduction	9
2	Is Deep Supervised Learning Knowledge Required?	9
3	Why Start with SFML?	10
4	Installing SFML on Ubuntu	11
5	Creating the First Window	11
6	What is the <code>sf</code> Namespace?	12
7	Understanding <code>sf::VideoMode</code>	13
8	Understanding <code>sf::RenderWindow</code>	13
9	The Coordinate System	14
10	Understanding the Game Loop	14
11	Events and <code>sf::Event</code>	14
12	What is <code>pollEvent()</code> ?	15
13	Handling the Close Event	16
14	Rendering	16
15	Creating a Player	17
16	Understanding <code>sf::Vector2f</code>	17
17	Position and Origin	18
18	Continuous Keyboard Input	18
19	<code>setPosition()</code> versus <code>move()</code>	19
20	The Frame-Rate Dependence Problem	19
21	Delta Time	20
22	Using <code>sf::Clock</code>	20
23	Direction Vectors	21
24	Why Diagonal Movement is Faster	21
25	Vector Normalization	21
26	Keeping the Player Inside the Window	22
27	Adding Obstacles	23

28 Collision Detection	23
29 Simple Collision Response	23
30 Axis-Separated Collision Response	24
31 Multiple Obstacles	24
32 Tunnelling	25
33 Simple Enemies	25
34 Building the Coin Collector	26
35 Player–Coin Collision	27
36 Random Number Generation	27
37 Score	28
38 Why Refactor into Classes?	28
39 Header and Implementation Files	29
40 The Player Class	29
40.1 Player.hpp	29
41 The Player Constructor	30
42 References and const	30
43 The Coin Class	31
43.1 Coin.hpp	31
44 Member Initializer Lists	31
45 The Game Class	32
45.1 Game.hpp	32
46 The Clean Game Loop	33
47 The Final main.cpp	33
48 Compiling Multiple Files	33
49 Connecting the Game to Reinforcement Learning	34
50 Discrete Actions with Continuous Motion	35
51 What is the State Space?	35
52 State Discretization	36
53 Using DQN	36
54 Relative State Representation	37

55	Representing Obstacles with Sensors	38
56	Representing Enemies	38
57	Reward Design	39
58	The Future RL Environment Interface	39
59	Why RL Training Should Not Depend on Real Time	40
60	Recommended Development Progression	41
61	Conclusion	42
62	A Proper Introduction to Reinforcement Learning	44
63	The Main Components of a Reinforcement Learning Problem	45
63.1	The Agent	45
63.2	The Environment	45
63.3	State	45
63.4	Action	45
63.5	Reward	46
64	Episodes and Time Steps	46
65	What is Q-Learning?	47
66	The Q-Table	47
67	Immediate Reward versus Future Reward	48
68	Return and Discounted Future Rewards	49
69	The Bellman Optimality Idea	50
70	A Numerical Bellman Example	50
71	The Q-Learning Update Equation	51
72	Numerical Example of a Complete Q-Learning Update	52
73	Understanding the Learning Rate α	53
74	How Reward Information Propagates Backwards	53
75	The Exploration versus Exploitation Problem	54
76	The Epsilon-Greedy Strategy	55
77	Random Tie-Breaking During Exploitation	55
78	The Complete Tabular Q-Learning Algorithm	56
79	Case Study: The FrozenLake Environment	58
80	Numbering the FrozenLake States	58

81 FrozenLake Action Space	59
82 Rewards in FrozenLake	59
83 A FrozenLake Learning Example	60
84 Training FrozenLake with Q-Learning	61
85 Understanding the FrozenLake Training Code	63
86 A Note About Terminal States	64
87 Visualizing the Learned FrozenLake Policy	64
88 Effect of the Learning Rate in FrozenLake	65
88.1 Low Learning Rate	65
88.2 High Learning Rate	66
89 Effect of the Discount Factor in FrozenLake	66
89.1 Lower Discount Factor	66
89.2 Higher Discount Factor	66
90 Deterministic FrozenLake	66
91 Slippery FrozenLake: A Stochastic Environment	67
92 What Does the Q-Value Mean in a Stochastic Environment?	67
93 Risk-Aware Behaviour on the Slippery Lake	68
94 Why Tabular Q-Learning Works Well for FrozenLake	68
95 The Fundamental Limitation of Tabular Q-Learning	68
96 From Q-Learning Toward Deep Q-Learning	69
97 From Q-Learning to Deep Q-Networks	70
98 A Fundamental Question: If We Do Not Know the Optimal Q-Value, How Can We Define a Loss?	71
99 The Bellman Target and Bootstrapping	72
100 How Reward Information Propagates Backward	73
101 The Problem with Using Only One Neural Network	74
102 Online Network and Target Network	75
103 Why Gradients Do Not Flow Through the Target Network	76
104 Experience Replay	76
104.1 What Is an Experience?	77
104.2 Random Mini-Batch Sampling	77

105A Key Question About Batch Dimensions	78
105.1Online-Network Output	78
105.2Target-Network Output	79
105.3Why Only the Chosen Action Is Directly Trained	80
106The Complete DQN Training Loop	80
106.1Epsilon-Greedy Action Selection	81
106.2Environment Steps, Training Steps, and Episodes	82
107Moving from DQN Theory to the C++ Environment	82
108The Fundamental RL Data Structures	83
108.1Action	83
108.2State	84
108.3StepResult	85
109The GameEnvironment Class	85
110Why a Separate Player::move() Function Was Needed	86
111Implementing getState()	88
112Implementing reset()	89
113Implementing step(Action)	91
113.1Why Use a Fixed Timestep?	91
113.2Converting Actions to Directions	92
114Reusing Boundary Logic	92
115Reward and Episode Termination	93
116Complete Initial step() Implementation	94
117Testing the Environment with a Random Agent	95
118Random-Agent Baseline Result	97
119How the Random Agent Relates to the Future DQN	97
120Current Architecture of the Project	98
121Summary of the Journey So Far	98
122Connecting the C++ Environment to Python	100
123What is pybind11?	100
124The C++ Structures Exposed to Python	101
125Creating the pybind11 Bindings	102
125.1The PYBIND11_MODULE Macro	103
125.2Binding the Action Enumeration	104
125.3Binding the State Structure	104
125.4Binding the StepResult Structure	105

126	Compiling the C++ Environment into a Python Module	105
127	Testing the C++ Environment from Python	106
128	The Python DQN Architecture	107
129	Experience Replay	108
130	A Question About Sampling a Batch	109
131	The DQN Training Step	110
132	Why We Used an Online Network and a Target Network	112
133	The First Random-Agent Baseline	113
134	Initial Local Training	113
135	Why We Did Not Simply Upload the Local .so File	113
136	Preparing the Project for Kaggle	114
137	Understanding Kaggle Input and Working Directories	114
138	Installing Dependencies on Kaggle	115
139	Compiling the C++ Environment on Kaggle	115
140	Testing the Environment on Kaggle	116
141	Important Training Hyperparameters	117
142	Epsilon-Greedy Action Selection	117
143	The Main Environment Interaction Loop	118
144	The First 100-Episode Kaggle Experiment	119
145	Why Two RL Runs Can Produce Different Results	120
146	The Better 100-Episode Run	120
147	Why the Loss Sometimes Became Large	121
148	Why Training Success Is Not the Same as Policy Quality	122
149	Adding Greedy Evaluation	122
150	Checkpointing	124
151	Checkpoint Versus Final Model	125
152	Final Training Results	126
153	Why Evaluation Performance Was Non-Monotonic	126
154	Loading the Trained Model	127

155	Watching the Trained Agent Play	128
156	Adding Optional Rendering to GameEnvironment	129
157	Rendering Methods	129
158	A Rendering Design Problem	130
159	The Undefined Symbol Error	131
160	Exposing Rendering Functions Through pybind11	132
161	Recompiling the Updated Environment	133
162	The watch_agent.py Script	134
163	Running the Trained Agent	136
164	Complete System Architecture	137
164.1	Layer 1: C++ Game Objects	137
164.2	Layer 2: C++ RL Environment	137
164.3	Layer 3: pybind11 Interface	138
164.4	Layer 4: Python DQN Agent	138
165	Key Lessons from the First Complete RL Game Project	138
165.1	The Environment and Agent Can Use Different Languages	138
165.2	Rendering Should Be Separate from Training	139
165.3	Always Establish a Random Baseline	139
165.4	Training Success and Evaluation Success Are Different	139
165.5	RL Training Is Stochastic	139
165.6	Loss Alone Does Not Measure Agent Performance	139
165.7	Save the Best Model, Not Only the Final Model	140
166	Conclusion	140
A	Detailed C++ Object-Oriented Programming and Language Concepts	141
A.1	What Are Classes, Objects, and Methods?	141
A.2	How Does a Method Know Which Object Called It?	142
A.3	Defining Methods Outside a Class	143
A.4	Constructors	144
A.5	Public and Private Members	145
A.6	Why Do Header Files Exist?	146
A.7	Declaration Versus Definition	147
A.8	Creating a Header File	148
A.9	What Does <code>#include</code> Actually Do?	149
A.10	What Happens When a Multi-File C++ Project Is Compiled?	150
A.10.1	Preprocessing	150
A.10.2	Compilation and Assembly	151
A.10.3	Linking	151
A.11	Why Is the Complete Class Declaration Usually in the Header?	152
A.12	Header Guards	152
A.13	Why Not Include a <code>.cpp</code> File?	153
A.14	Understanding <code>sf::FloatRect getBounds() const</code>	154
A.15	What Is <code>sf::FloatRect</code> ?	154

A.16 Why Is <code>const</code> Written at the End of a Method?	155
A.17 Why Is <code>handleInput()</code> Not Const?	155
A.18 Two Different Uses of <code>const</code>	156
A.19 The Hidden <code>this</code> Pointer and Const Methods	156
A.20 Why Const Member Functions Matter	157
A.21 <code>const</code> Does Not Only Mean a Constant Literal	157
A.22 Understanding <code>std::mt19937</code>	158
A.23 Why Is It Called Pseudo-Random?	159
A.24 Default Seed Versus Random Seed	160
A.25 Why Not Use the Raw Generator Output Directly?	160
A.26 Integer and Floating-Point Distributions	161
A.27 Using Random Generation in the Coin Class	162
A.28 Understanding Constructor Initializer Lists	162
A.29 What Happens Before the Constructor Body?	163
A.30 Breaking Down <code>std::random_device{}()</code>	164
A.31 Why Not Assign Inside the Constructor Body?	164
A.32 Simple Initializer-List Example	165
A.33 Initializer Lists with Constructor Parameters	166
A.34 When Are Initializer Lists Mandatory?	167
A.34.1 Const Data Members	167
A.34.2 Reference Members	168
A.35 Member Initialization Order	168
A.36 Complete Mental Model	169
B Smart Pointers, <code>std::unique_ptr</code>, and Window Ownership	171
B.1 What Is <code>std::unique_ptr<Type>?</code>	171
B.2 Starting with an Ordinary Object	172
B.3 What Is a Raw Pointer?	172
B.4 The Problem with Raw Pointers and <code>new</code>	173
B.5 Enter <code>std::unique_ptr</code>	174
B.6 Why Is It Called <code>unique_ptr</code> ?	175
B.7 Transferring Ownership with <code>std::move</code>	175
B.8 What Does <code>std::make_unique</code> Do?	176
B.9 How Is a <code>unique_ptr</code> Used?	177
B.10 A <code>unique_ptr</code> Can Be Empty	177
B.11 Why Use a <code>unique_ptr</code> for the Game Window?	178
B.12 Creating the Window Only When Needed	179
B.13 Rendering with an Optional Window	180
B.14 Understanding <code>window->clear()</code>	181
B.15 Why Is <code>*window</code> Passed to <code>draw()</code> ?	181
B.16 Why Not Simply Store the Window Directly?	182
B.17 Why Not Use a Raw Pointer for the Optional Window?	183
B.18 RAII: Resource Acquisition Is Initialization	184
B.19 Direct Object, Raw Pointer, and <code>unique_ptr</code> Compared	184
B.20 Complete Lifecycle of the Window in the RL Environment	185
B.21 Important Clarification: Is <code>unique_ptr</code> Mandatory?	186
B.22 Key Mental Model	186

1 Introduction

The goal of this article is to build a strong foundation in two connected areas:

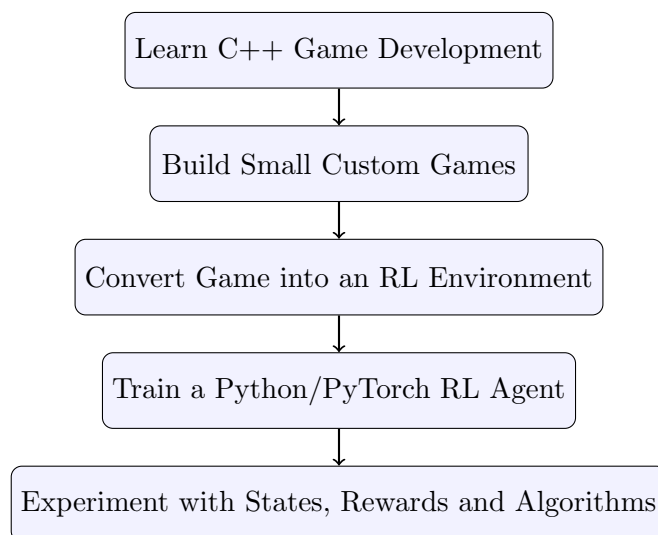
1. 2D game development using C++ and SFML.
2. Reinforcement learning using small custom-built games as environments.

Instead of immediately using a large game engine or an existing RL environment, we will build our own games from scratch.

This gives us control over:

- the game rules,
- the player,
- movement,
- collisions,
- observations,
- actions,
- rewards,
- episode termination,
- and eventually the RL training process itself.

The overall journey is:



2 Is Deep Supervised Learning Knowledge Required?

A common question is:

Question

Is it necessary to have deep knowledge of supervised learning before starting reinforcement learning?

The answer is no.

Knowing the fundamentals of neural networks, CNNs, and basic Q-learning is sufficient to begin learning reinforcement learning.

In supervised learning, the basic process is:

$$\text{Input} \rightarrow \text{Model} \rightarrow \text{Prediction}$$

The prediction is compared with a known correct label.

In reinforcement learning:

$$\text{State} \rightarrow \text{Agent} \rightarrow \text{Action}$$

The environment then responds with:

$$(\text{next state}, \text{reward}, \text{done})$$

There is generally no teacher explicitly saying:

“The correct action in this state is action number 3.”

Instead, the agent must discover good behaviour through interaction and rewards.

Neural-network knowledge becomes particularly useful when moving from a Q-table to algorithms such as Deep Q-Networks, or DQN.

CNNs become useful when the RL agent receives raw game images as observations.

Initially, however, it is much easier to provide structured information such as:

Player position
Coin relative position
Enemy relative position
Distance to nearby walls
Current velocity

3 Why Start with SFML?

SFML stands for:

Simple and Fast Multimedia Library

It provides tools for:

- windows,
- graphics,
- keyboard and mouse input,
- audio,
- clocks and timing,
- sprites and textures,
- shapes and text.

For learning the fundamentals of game development, SFML is useful because it does not hide the core game loop.

A typical game loop looks like:

```
1 while (window.isOpen())
2 {
3     handleEvents();
4     update();
5     render();
6 }
```

This makes fundamental concepts visible.

4 Installing SFML on Ubuntu

Open a terminal:

Ctrl + Alt + T

Update package information:

```
sudo apt update
```

Install SFML:

```
sudo apt install libsFML-dev
```

Verify installation:

```
dpkg -l | grep sfml
```

Ensure the C++ compiler is available:

```
g++ --version
```

If necessary:

```
sudo apt install build-essential
```

Create a project:

```
mkdir -p ~/GameDevRL/first_game
cd ~/GameDevRL/first_game
code.
```

5 Creating the First Window

Create a file called main.cpp:

```
1 #include <SFML/Graphics.hpp>
2
3 int main()
4 {
5     sf::RenderWindow window(
6         sf::VideoMode(800, 600),
7         "Game Dev + RL"
8     );
9
10    while (window.isOpen())
```

```
11 {  
12 }  
13  
14 return 0;  
15 }
```

Before adding more code, we need to understand every object involved.

6 What is the `sf` Namespace?

Consider:

```
1 sf::RenderWindow
```

The `sf` part is a namespace.

C++ already uses namespaces extensively. For example:

```
1 std::cout << "Hello";
```

Here:

```
1 std::cout
```

means:

Use the object called `cout` from the namespace called `std`.

Similarly:

```
1 sf::RenderWindow
```

means:

Use the class called `RenderWindow` from the namespace called `sf`.

SFML places its classes in this namespace:

```
1 sf::RenderWindow  
2 sf::VideoMode  
3 sf::Event  
4 sf::RectangleShape  
5 sf::CircleShape  
6 sf::Vector2f  
7 sf::Clock  
8 sf::Color
```

Namespaces prevent naming conflicts.

For example:

```
1 LibraryA::Window  
2 LibraryB::Window
```

Both libraries can define a class called `Window` without ambiguity.

7 Understanding `sf::VideoMode`

Consider:

```
1 sf::VideoMode(800, 600)
```

This describes the dimensions of the window:

width = 800

height = 600

We could write:

```
1 sf::VideoMode mode(800, 600);  
2  
3 sf::RenderWindow window(  
4     mode,  
5     "Game Dev + RL"  
6 );
```

This is equivalent to:

```
1 sf::RenderWindow window(  
2     sf::VideoMode(800, 600),  
3     "Game Dev + RL"  
4 );
```

The second version simply constructs the `VideoMode` directly as an argument.

8 Understanding `sf::RenderWindow`

Consider:

```
1 sf::RenderWindow window(  
2     sf::VideoMode(800, 600),  
3     "Game Dev + RL"  
4 );
```

Here:

- `sf::RenderWindow` is a class.
- `window` is an object of that class.

The object provides functions such as:

```
1 window.isOpen();  
2 window.pollEvent(event);  
3 window.clear();  
4 window.draw(object);  
5 window.display();  
6 window.close();
```

A `RenderWindow` is both an operating-system window and a place where SFML can render graphics.

9 The Coordinate System

In an SFML window:

(0,0)

is at the top-left.

The x-coordinate increases toward the right.

The y-coordinate increases downward.

Therefore:

```
1 x += 10;
```

moves right, while:

```
1 y += 10;
```

moves downward.

This is different from the usual Cartesian coordinate system used in mathematics.

10 Understanding the Game Loop

The heart of a real-time game is:

```
1 while (window.isOpen())
2 {
3 }
```

Each loop iteration is approximately one frame.

The usual structure is:

```
1 while (window.isOpen())
2 {
3     handleEvents();
4     update();
5     render();
6 }
```

Conceptually:

Input → Update → Render → Repeat

At 60 frames per second, this happens approximately 60 times every second.

11 Events and `sf::Event`

An event is something that happened involving the window or user input.

Examples include:

- close button clicked,
- keyboard key pressed,
- keyboard key released,
- mouse moved,
- mouse clicked,

- window resized,
- window lost focus.

We create:

```
1 sf::Event event;
```

This object stores information about one event.

For example, conceptually:

```
event.type = KeyPressed  
event.key  = Space
```

or:

```
event.type = Closed
```

12 What is pollEvent()?

Consider:

```
1 window.pollEvent(event)
```

This asks the window:

Is there an unprocessed event waiting? If yes, retrieve it and store its information in `event`.

It returns:

- `true` if an event was found,
- `false` if no event remains.

Therefore:

```
1 while (window.pollEvent(event))  
2 {  
3 }
```

means:

Continue retrieving events until the event queue becomes empty.

Suppose the queue contains:

```
Mouse moved  
A pressed  
A released  
Window resized
```

Each call to `pollEvent()` retrieves one event.

When no events remain, it returns `false`, ending the loop.

13 Handling the Close Event

The code is:

```
1 sf::Event event;
2
3 while (window.pollEvent(event))
4 {
5     if (event.type == sf::Event::Closed)
6     {
7         window.close();
8     }
9 }
```

Clicking the X button does not necessarily force the program to close immediately. Instead, the operating system sends a close request. SFML represents that as:

```
1 sf::Event::Closed
```

The application decides what to do. This allows a game to:

- ask the user to save,
- show a confirmation dialog,
- clean up resources,
- cancel the close operation.

14 Rendering

The standard rendering pattern is:

```
1 window.clear();
2
3 window.draw(object1);
4 window.draw(object2);
5
6 window.display();
```

The process is:

1. Erase the previous frame.
2. Draw the new game objects.
3. Display the completed frame.

Objects are not permanently painted onto the screen. Instead, the scene is repeatedly erased and redrawn. This rapid sequence creates the illusion of motion.

15 Creating a Player

Create a rectangle:

```
1 sf::RectangleShape player;
```

Set its size:

```
1 player.setSize(sf::Vector2f(50.0f, 50.0f));
```

Set its colour:

```
1 player.setFillColor(sf::Color::Green);
```

Set its position:

```
1 player.setPosition(375.0f, 275.0f);
```

Draw it:

```
1 window.draw(player);
```

16 Understanding sf::Vector2f

A `Vector2f` contains two floating-point values:

```
1 sf::Vector2f vector(50.0f, 30.0f);
```

Conceptually:

`vector.x = 50`

`vector.y = 30`

The name means:

Vector + 2 + f

Vector -> stores components

2 -> x and y

f -> float

For a size:

```
1 sf::Vector2f(50.0f, 30.0f)
```

means:

width = 50, height = 30

For movement:

```
1 sf::Vector2f(5.0f, -2.0f)
```

means:

- move 5 pixels right,
- move 2 pixels upward.

17 Position and Origin

By default, a rectangle's position refers to its top-left corner.

For a 50 by 50 player:

```
1 player.setPosition(375.0f, 275.0f);
```

The top-left corner is at:

(375, 275)

The centre is at:

(400, 300)

because:

$$375 + 25 = 400$$

and:

$$275 + 25 = 300$$

18 Continuous Keyboard Input

To check whether W is currently held:

```
1 if (sf::Keyboard::isKeyPressed(sf::Keyboard::W))
2 {
3     player.move(0.0f, -5.0f);
4 }
```

Similarly:

```
1 if (sf::Keyboard::isKeyPressed(sf::Keyboard::S))
2     player.move(0.0f, 5.0f);
3
4 if (sf::Keyboard::isKeyPressed(sf::Keyboard::A))
5     player.move(-5.0f, 0.0f);
6
7 if (sf::Keyboard::isKeyPressed(sf::Keyboard::D))
8     player.move(5.0f, 0.0f);
```

The directions are:

```

      -y
      W
      ^

-x      A <- PLAYER -> D      +x

      v
      S
      +y
```

19 setPosition() versus move()

Consider:

```
1 player.setPosition(100.0f, 200.0f);
```

This puts the player exactly at:

(100, 200)

But:

```
1 player.move(10.0f, 5.0f);
```

adds to the current position.

If the current position is:

(100, 200)

then after movement:

(110, 205)

Mathematically:

new position = old position + movement

20 The Frame-Rate Dependence Problem

Suppose:

```
1 float speed = 5.0f;  
2 player.move(speed, 0.0f);
```

This really means:

Move 5 pixels per frame.

At 30 FPS:

$$30 \times 5 = 150$$

pixels per second.

At 60 FPS:

$$60 \times 5 = 300$$

pixels per second.

At 144 FPS:

$$144 \times 5 = 720$$

pixels per second.

The game would run at different speeds on different computers.

21 Delta Time

Delta time, usually written as `dt`, is the amount of time elapsed since the previous frame.

At approximately 60 FPS:

$$dt \approx \frac{1}{60} = 0.0167 \text{ seconds}$$

At approximately 30 FPS:

$$dt \approx \frac{1}{30} = 0.0333 \text{ seconds}$$

Movement becomes:

$$\text{distance} = \text{speed} \times dt$$

If speed is:

300 pixels per second

then at 60 FPS:

$$300 \times 0.0167 \approx 5$$

pixels per frame.

At 30 FPS:

$$300 \times 0.0333 \approx 10$$

pixels per frame.

But over one second, both travel approximately 300 pixels.

22 Using `sf::Clock`

Create:

```
1 sf::Clock clock;
```

At the beginning of every frame:

```
1 float dt = clock.restart().asSeconds();
```

The function `restart()`:

1. returns the elapsed time,
2. resets the clock to zero.

Therefore:

```
1 player.move(speed * dt, 0.0f);
```

gives frame-independent movement.

23 Direction Vectors

Instead of immediately moving for every key press, create:

```
1 sf::Vector2f direction(0.0f, 0.0f);
```

Then:

```
1 if (sf::Keyboard::isKeyPressed(sf::Keyboard::W))
2     direction.y -= 1.0f;
3
4 if (sf::Keyboard::isKeyPressed(sf::Keyboard::S))
5     direction.y += 1.0f;
6
7 if (sf::Keyboard::isKeyPressed(sf::Keyboard::A))
8     direction.x -= 1.0f;
9
10 if (sf::Keyboard::isKeyPressed(sf::Keyboard::D))
11     direction.x += 1.0f;
```

Examples:

Input	Direction
W	$(0, -1)$
S	$(0, 1)$
A	$(-1, 0)$
D	$(1, 0)$
W+D	$(1, -1)$
S+A	$(-1, 1)$

24 Why Diagonal Movement is Faster

For horizontal movement:

$$\vec{d} = (1, 0)$$

Magnitude:

$$|\vec{d}| = \sqrt{1^2 + 0^2} = 1$$

For diagonal movement:

$$\vec{d} = (1, -1)$$

Magnitude:

$$|\vec{d}| = \sqrt{1^2 + (-1)^2} = \sqrt{2} \approx 1.414$$

Therefore, diagonal movement is approximately 41.4% faster.

25 Vector Normalization

Normalization preserves direction while changing the magnitude to 1.

For:

$$\vec{d} = (x, y)$$

the magnitude is:

$$|\vec{d}| = \sqrt{x^2 + y^2}$$

The normalized vector is:

$$\hat{d} = \frac{\vec{d}}{|\vec{d}|}$$

In C++:

```
1 if (direction.x != 0.0f || direction.y != 0.0f)
2 {
3     float length = std::sqrt(
4         direction.x * direction.x +
5         direction.y * direction.y
6     );
7
8     direction.x /= length;
9     direction.y /= length;
10 }
```

We must not normalize (0,0) because that would involve division by zero.

Movement becomes:

```
1 player.move(direction * speed * dt);
```

26 Keeping the Player Inside the Window

Retrieve information:

```
1 sf::Vector2f position = player.getPosition();
2 sf::Vector2u windowSize = window.getSize();
3 sf::Vector2f playerSize = player.getSize();
```

Then constrain:

```
1 if (position.x < 0.0f)
2     position.x = 0.0f;
3
4 if (position.y < 0.0f)
5     position.y = 0.0f;
6
7 if (position.x + playerSize.x > windowSize.x)
8     position.x = windowSize.x - playerSize.x;
9
10 if (position.y + playerSize.y > windowSize.y)
11     position.y = windowSize.y - playerSize.y;
12
13 player.setPosition(position);
```

For an 800 by 600 window and 50 by 50 player:

$$0 \leq x \leq 750$$

$$0 \leq y \leq 550$$

27 Adding Obstacles

Create:

```
1 sf::RectangleShape obstacle;  
2  
3 obstacle.setSize(sf::Vector2f(200.0f, 100.0f));  
4 obstacle.setFillColor(sf::Color::Red);  
5 obstacle.setPosition(300.0f, 250.0f);
```

Draw:

```
1 window.draw(obstacle);
```

28 Collision Detection

Collision detection asks:

Are these two objects overlapping?

Collision response asks:

What should happen after they overlap?

These are different problems.

SFML provides:

```
1 player.getGlobalBounds()
```

which returns a `sf::FloatRect`.

Collision can be checked using:

```
1 if (player.getGlobalBounds().intersects(  
2     obstacle.getGlobalBounds()))  
3 {  
4     // collision  
5 }
```

29 Simple Collision Response

Remember the old position:

```
1 sf::Vector2f oldPosition = player.getPosition();
```

Move:

```
1 player.move(direction * speed * dt);
```

If collision occurs:

```
1 if (player.getGlobalBounds().intersects(  
2     obstacle.getGlobalBounds()))  
3 {  
4     player.setPosition(oldPosition);  
5 }
```

This works but causes a problem with diagonal movement.

If horizontal movement causes collision but vertical movement is valid, both movements are cancelled.

30 Axis-Separated Collision Response

Calculate:

```
1 float dx = direction.x * speed * dt;
2 float dy = direction.y * speed * dt;
```

Move horizontally first:

```
1 player.move(dx, 0.0f);
2
3 if (player.getGlobalBounds().intersects(
4     obstacle.getGlobalBounds()))
5 {
6     player.move(-dx, 0.0f);
7 }
```

Then vertically:

```
1 player.move(0.0f, dy);
2
3 if (player.getGlobalBounds().intersects(
4     obstacle.getGlobalBounds()))
5 {
6     player.move(0.0f, -dy);
7 }
```

This allows the player to slide smoothly along walls.

31 Multiple Obstacles

Use:

```
1 std::vector<sf::RectangleShape> obstacles;
```

A helper function:

```
1 sf::RectangleShape createObstacle(
2     float x,
3     float y,
4     float width,
5     float height
6 )
7 {
8     sf::RectangleShape obstacle;
9
10    obstacle.setSize(sf::Vector2f(width, height));
11    obstacle.setPosition(x, y);
12    obstacle.setFillColor(sf::Color::Red);
13
14    return obstacle;
15 }
```

Add:

```
1 obstacles.push_back(
2     createObstacle(200, 150, 200, 50)
3 );
4
5 obstacles.push_back(
```

```
6     createObstacle(500, 300, 100, 200)
7 );
```

Check collision:

```
1 for (const auto& obstacle: obstacles)
2 {
3     if (player.getGlobalBounds().intersects(
4         obstacle.getGlobalBounds()))
5     {
6         player.move(-dx, 0.0f);
7         break;
8     }
9 }
```

Render:

```
1 for (const auto& obstacle: obstacles)
2 {
3     window.draw(obstacle);
4 }
```

32 Tunnelling

At very high speeds, an object may move completely through an obstacle in a single frame.

For example:

Frame N:

PLAYER -----> WALL

Frame N+1:

WALL PLAYER

At neither sampled position are the objects overlapping.

Therefore, collision detection may fail.

This phenomenon is called **tunnelling**.

More advanced solutions include:

- continuous collision detection,
- swept AABB,
- smaller fixed timesteps,
- physics engines.

For simple games with moderate speeds, basic collision detection is sufficient.

33 Simple Enemies

Suppose an enemy moves toward the player.

Get positions:

```
1 sf::Vector2f enemyPosition = enemy.getPosition();
2 sf::Vector2f playerPosition = player.getPosition();
```

Direction:

```
1 sf::Vector2f direction =
2   playerPosition - enemyPosition;
```

Normalize:

```
1 float length = std::sqrt(
2     direction.x * direction.x +
3     direction.y * direction.y
4 );
5
6 if (length != 0.0f)
7 {
8     direction.x /= length;
9     direction.y /= length;
10 }
```

Move:

```
1 enemy.move(direction * enemySpeed * dt);
```

With multiple enemies:

```
1 for (auto& enemy: enemies)
2 {
3     // Calculate direction
4     // Normalize
5     // Move
6     // Check collisions
7 }
```

34 Building the Coin Collector

Our first actual small game contains:

- one player,
- one coin,
- WASD movement,
- random coin respawning,
- score.

Create the coin:

```
1 sf::CircleShape coin(15.0f);
2
3 coin.setFillColor(sf::Color::Yellow);
4 coin.setPosition(500.0f, 300.0f);
```

The radius is 15 pixels.

Therefore:

$$\text{diameter} = 30 \text{ pixels}$$

35 Player–Coin Collision

Check:

```
1 if (player.getGlobalBounds().intersects(
2     coin.getGlobalBounds()))
3 {
4     // Coin collected
5 }
```

The circle is checked using its rectangular bounding box.

This is not mathematically perfect circle–rectangle collision detection, but is sufficient for this simple game.

36 Random Number Generation

Include:

```
1 #include <random>
```

Create:

```
1 std::random_device rd;
2 std::mt19937 generator(rd());
```

The `std::mt19937` object is a pseudo-random number generator based on the Mersenne Twister algorithm.

The `random_device` provides a seed.

Conceptually:

```
random_device
  |
  v
Initial random seed
  |
  v
mt19937 generator
  |
  v
Sequence of pseudo-random numbers
```

Create distributions:

```
1 std::uniform_real_distribution<float> randomX(
2     0.0f,
3     770.0f
4 );
5
6 std::uniform_real_distribution<float> randomY(
7     0.0f,
8     570.0f
9 );
```

Generate:

```
1 float x = randomX(generator);
2 float y = randomY(generator);
```

Then:

```
1 coin.setPosition(x, y);
```

37 Score

Create:

```
1 int score = 0;
```

When a collision occurs:

```
1 if (player.getGlobalBounds().intersects(
2     coin.getGlobalBounds()))
3 {
4     score++;
5
6     float x = randomX(generator);
7     float y = randomY(generator);
8
9     coin.setPosition(x, y);
10
11     std::cout << "Score: "
12                << score
13                << '\n';
14 }
```

38 Why Refactor into Classes?

As the game grows, keeping everything inside `main()` becomes difficult.

A better structure is:

```
CoinCollector/
|
|-- main.cpp
|-- Game.hpp
|-- Game.cpp
|-- Player.hpp
|-- Player.cpp
|-- Coin.hpp
'-- Coin.cpp
```

The architecture becomes:

```
main.cpp
|
v
Game
/  \
Player Coin
```

39 Header and Implementation Files

A header file usually declares what a class contains:

```
1 // Player.hpp
2
3 class Player
4 {
5 public:
6     void move();
7 };
```

The implementation file defines how it works:

```
1 // Player.cpp
2
3 void Player::move()
4 {
5     // movement logic
6 }
```

The operator:

```
1 ::
```

is called the **scope resolution operator**.

Therefore:

```
1 Player::move()
```

means:

The function `move()` belonging to the `Player` class.

40 The Player Class

40.1 Player.hpp

```
1 #ifndef PLAYER_HPP
2 #define PLAYER_HPP
3
4 #include <SFML/Graphics.hpp>
5
6 class Player
7 {
8 private:
9     sf::RectangleShape shape;
10    float speed;
11
12 public:
13    Player();
14
15    void handleInput(float dt);
16    void keepInsideWindow(
17        const sf::RenderWindow& window
18    );
19
20    void draw(sf::RenderWindow& window);
21 }
```

```

22     sf::FloatRect getBounds() const;
23     sf::Vector2f getPosition() const;
24 };
25
26 #endif

```

The private data is:

```

1 sf::RectangleShape shape;
2 float speed;

```

This is encapsulation.

External code should not directly manipulate every internal detail of the player.

41 The Player Constructor

```

1 Player::Player()
2 {
3     shape.setSize(
4         sf::Vector2f(50.0f, 50.0f)
5     );
6
7     shape.setFillColor(sf::Color::Green);
8
9     shape.setPosition(100.0f, 100.0f);
10
11     speed = 300.0f;
12 }

```

When:

```

1 Player player;

```

is executed, the constructor is automatically called.

42 References and const

Consider:

```

1 void Player::keepInsideWindow(
2     const sf::RenderWindow& window
3 )

```

The `&` means the window is passed by reference.

The entire window object is not copied.

The `const` means:

This function promises not to modify the window.

Similarly:

```

1 sf::FloatRect Player::getBounds() const

```

The final `const` means:

Calling this function will not modify the Player object.

43 The Coin Class

43.1 Coin.hpp

```
1 #ifndef COIN_HPP
2 #define COIN_HPP
3
4 #include <SFML/Graphics.hpp>
5 #include <random>
6
7 class Coin
8 {
9 private:
10     sf::CircleShape shape;
11     std::mt19937 generator;
12
13 public:
14     Coin();
15
16     void respawn(
17         const sf::RenderWindow& window
18     );
19
20     void draw(sf::RenderWindow& window);
21
22     sf::FloatRect getBounds() const;
23     sf::Vector2f getPosition() const;
24 };
25
26 #endif
```

44 Member Initializer Lists

The constructor is:

```
1 Coin::Coin()
2     : generator(std::random_device{}())
3 {
4     shape.setRadius(15.0f);
5     shape.setFillColor(sf::Color::Yellow);
6     shape.setPosition(500.0f, 300.0f);
7 }
```

A common question is:

Question

What is the `: generator(...)` after the constructor parentheses but before the constructor body?

It is called a **member initializer list**.

The general syntax is:

```
1 ClassName::ClassName()
2     : member1(value1),
3       member2(value2)
4 {
```



```
5 }
```

Members are initialized before the constructor body runs.
Therefore:

```
1 Coin::Coin()  
2   : generator(std::random_device{}())
```

initializes the `generator` member using a seed produced by:

```
1 std::random_device{}()
```

This is often preferable to assigning members later inside the constructor body.

45 The Game Class

The `Game` class owns:

- the window,
- player,
- coin,
- clock,
- score.

45.1 `Game.hpp`

```
1 #ifndef GAME_HPP  
2 #define GAME_HPP  
3  
4 #include <SFML/Graphics.hpp>  
5  
6 #include "Player.hpp"  
7 #include "Coin.hpp"  
8  
9 class Game  
10 {  
11 private:  
12     sf::RenderWindow window;  
13  
14     Player player;  
15     Coin coin;  
16  
17     sf::Clock clock;  
18  
19     int score;  
20  
21 public:  
22     Game();  
23  
24     void run();  
25  
26 private:  
27     void processEvents();  
28     void update(float dt);
```

```
29     void render();
30 };
31
32 #endif
```

46 The Clean Game Loop

```
1 void Game::run()
2 {
3     while (window.isOpen())
4     {
5         float dt =
6             clock.restart().asSeconds();
7
8         processEvents();
9
10        update(dt);
11
12        render();
13    }
14 }
```

This is much easier to understand:

Events → Update → Render

47 The Final main.cpp

After refactoring:

```
1 #include "Game.hpp"
2
3 int main()
4 {
5     Game game;
6
7     game.run();
8
9     return 0;
10 }
```

The main function simply says:

Create a game and run it.

48 Compiling Multiple Files

Compile:

```
g++ main.cpp Game.cpp Player.cpp Coin.cpp \
-o game \
-lsfml-graphics \
-lsfml-window \
-lsfml-system
```

Or:

```
g++ *.cpp -o game \  
-lsfml-graphics \  
-lsfml-window \  
-lsfml-system
```

Run:

```
./game
```

49 Connecting the Game to Reinforcement Learning

Currently:

```
Keyboard  
  |  
  v  
Player::handleInput(dt)  
  |  
  v  
Player moves
```

Eventually:

```
RL Agent  
  |  
  v  
Action  
  |  
  v  
GameEnvironment::step(action)  
  |  
  v  
Player moves  
  |  
  v  
Next state + reward + done
```

The Player should ideally not care who decided the action.
The movement command could come from:

- a human,
- a scripted bot,
- a Q-learning agent,
- a DQN,
- another RL algorithm.

50 Discrete Actions with Continuous Motion

Even if the player's position is continuous:

$$x = 327.481$$

$$y = 192.735$$

the action space can still be discrete:

```
0 -> Nothing
1 -> Up
2 -> Down
3 -> Left
4 -> Right
```

Therefore:

Continuous state space $\nRightarrow$ continuous action space

A possible enum is:

```
1 enum class Action
2 {
3     Nothing,
4     Up,
5     Down,
6     Left,
7     Right
8 };
```

51 What is the State Space?

The simplest state could be:

$$s = (p_x, p_y, c_x, c_y)$$

where:

- p_x, p_y are player coordinates,
- c_x, c_y are coin coordinates.

For example:

```
Player = (200.4, 300.7)
Coin   = (600.2, 100.5)
```

State:

[200.4, 300.7, 600.2, 100.5]

Since these values are continuous, a traditional Q-table is not practical.

52 State Discretization

One solution is to divide the world into cells.

Suppose:

$$\text{cell size} = 50 \times 50$$

For an 800 by 600 window:

$$800/50 = 16$$

columns, and:

$$600/50 = 12$$

rows.

Then:

$$(327.481, 192.735)$$

could become:

$$(6, 3)$$

This allows tabular Q-learning but sacrifices the full continuous nature of the game.

53 Using DQN

Instead of storing:

$$Q(s, a)$$

in a table, a neural network approximates it:

$$Q_{\theta}(s, a)$$

Suppose the input is:

$$[p_x, p_y, c_x, c_y]$$

A network could be:

```

4 inputs
|
Dense(64)
|
ReLU
|
Dense(64)
|
ReLU
|
5 outputs

```

The five outputs represent:

$Q(\text{Up})$
 $Q(\text{Down})$
 $Q(\text{Left})$
 $Q(\text{Right})$
 $Q(\text{Nothing})$

For example:

Up -> 4.2
 Down -> -1.7
 Left -> 0.8
 Right -> 6.5
 Nothing -> 1.1

The greedy action is Right.

54 Relative State Representation

Instead of absolute coordinates, use:

$$dx = c_x - p_x$$

$$dy = c_y - p_y$$

Suppose:

Player = (200, 300)

and:

Coin = (500, 200)

Then:

$$dx = 300$$

$$dy = -100$$

The state becomes:

$$[300, -100]$$

This directly tells the agent:

The coin is 300 pixels to the right and 100 pixels above.

Normalize:

$$dx_{\text{norm}} = \frac{c_x - p_x}{\text{window width}}$$

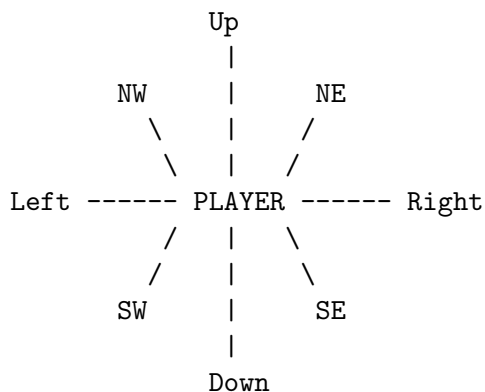
$$dy_{\text{norm}} = \frac{c_y - p_y}{\text{window height}}$$

This keeps values in a more manageable range.

55 Representing Obstacles with Sensors

Passing every obstacle coordinate directly becomes difficult when the number of obstacles changes.

Instead, the player can have virtual sensors:



Each sensor returns normalized distance to the nearest obstacle.

For example:

```
[
    0.8,
    0.3,
    0.1,
    0.5,
    1.0,
    1.0,
    0.7,
    0.4
]
```

A value near zero indicates a nearby obstacle.

This representation can work regardless of whether there are 3 or 300 obstacles.

56 Representing Enemies

For the nearest enemy, use:

enemy_dx

enemy_dy

enemy_distance

A possible complete state is:

```
[
    coin_dx,
    coin_dy,
    ray_up,
```

```

    ray_up_right,
    ray_right,
    ray_down_right,
    ray_down,
    ray_down_left,
    ray_left,
    ray_up_left,

    enemy_dx,
    enemy_dy,
    enemy_distance
]

```

This gives a 13-dimensional continuous state:

$$s \in \mathbb{R}^{13}$$

while actions remain discrete.

57 Reward Design

A simple reward system could be:

Event	Reward
Collect coin	+10
Hit enemy	-20
Every step	-0.01

The small step penalty encourages faster solutions.

If an agent collects a coin after 50 steps:

$$10 - 50(0.01) = 9.5$$

If it takes 500 steps:

$$10 - 500(0.01) = 5$$

Thus the faster path receives greater return.

Reward shaping must be designed carefully because agents may discover unintended ways of maximizing reward.

This is sometimes called **reward hacking**.

58 The Future RL Environment Interface

A possible state:

```

1 struct State
2 {
3     float coinDx;
4     float coinDy;
5
6     std::vector<float> obstacleSensors;
7
8     float enemyDx;

```



```

9     float enemyDy;
10    float enemyDistance;
11 };

```

A step result:

```

1 struct StepResult
2 {
3     State state;
4     float reward;
5     bool done;
6 };

```

Environment:

```

1 class GameEnvironment
2 {
3 public:
4     State reset();
5
6     StepResult step(int action);
7
8     void render();
9 };

```

The interaction becomes:

```

Agent
|
| action
v
Environment
|
| update physics
| check collisions
| calculate reward
v
next state, reward, done
|
v
Agent

```

59 Why RL Training Should Not Depend on Real Time

For human-controlled gameplay, we used:

```

1 player.move(direction * speed * dt);

```

But RL training may use a fixed timestep:

```

1 constexpr float dt = 1.0f / 60.0f;

```

or a fixed movement step.

This makes training:

- deterministic,
- reproducible,

- independent of computer frame rate,
- much faster than real time.

The environment should also be capable of running without opening a graphical window. Rendering millions of training frames would severely slow down training. Therefore, the final architecture should separate:

Game logic

```
|  
|-- state  
|-- movement  
|-- collisions  
|-- rewards  
|-- terminal conditions
```

Rendering

```
|  
|-- window  
|-- shapes  
|-- sprites  
|-- animations
```

60 Recommended Development Progression

A practical progression is:

1. Build a manually controlled player.
2. Add frame-independent movement.
3. Add screen boundaries.
4. Add obstacles and collisions.
5. Add a collectible coin.
6. Add random spawning and score.
7. Refactor into Player, Coin, and Game classes.
8. Separate input decisions from player movement.
9. Create an Action enum.
10. Add reset().
11. Add step(action).
12. Define state representation.
13. Define rewards.
14. Train an agent on player + coin.
15. Add obstacles.

16. Add sensor observations.
17. Add enemies.
18. Experiment with reward functions.
19. Compare different RL algorithms.

The environment can evolve as:

Stage 1:

Player + Coin

|
v

Stage 2:

Player + Coin + Walls

|
v

Stage 3:

Player + Coin + Obstacles

|
v

Stage 4:

Player + Coin + Moving Enemies

|
v

Stage 5:

Multiple Coins and Enemies

|
v

Stage 6:

Compare states, rewards, and RL algorithms

61 Conclusion

The central idea of this project is that game development and reinforcement learning fit naturally together.

A game already contains:

- a state,
- possible actions,
- transitions between states,

- goals,
- failures,
- rewards or scores.

These are precisely the ingredients required for reinforcement learning.

The game-development journey begins with:

```
1 while (window.isOpen())
2 {
3     processEvents();
4     update();
5     render();
6 }
```

The RL journey eventually transforms this into:

```
1 state = environment.reset();
2
3 while (!done)
4 {
5     action = agent.chooseAction(state);
6
7     result = environment.step(action);
8
9     agent.learn(
10         state,
11         action,
12         result.reward,
13         result.state,
14         result.done
15     );
16
17     state = result.state;
18 }
```

The bridge between the two is careful software architecture.

The Player should not care whether its movement was chosen by a keyboard, a scripted algorithm, or a neural network.

The environment should handle:

- applying actions,
- updating the world,
- detecting collisions,
- generating rewards,
- returning observations.

The agent should handle:

- observing the state,
- selecting actions,
- learning from rewards,

- improving its policy.

This separation allows the same custom C++ game to serve both as a playable game and as an experimental reinforcement-learning environment.

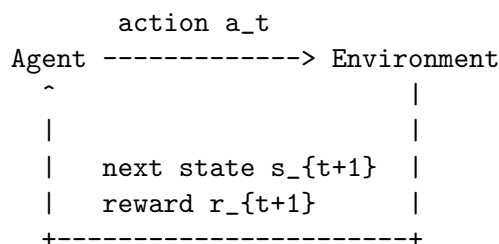
“latex

62 A Proper Introduction to Reinforcement Learning

Before moving from our C++ coin collector to Deep Q-Learning, it is important to understand the fundamental algorithm on which DQN is based: **Q-Learning**.

Reinforcement Learning, or RL, is a type of machine learning in which an **agent** learns by interacting with an **environment**.

The basic interaction is:



At each time step t :

1. The agent observes the current state s_t .
2. The agent chooses an action a_t .
3. The environment executes the action.
4. The environment transitions to a new state s_{t+1} .
5. The environment gives the agent a reward r_{t+1} .
6. The process repeats.

Mathematically:

$$s_t \xrightarrow{a_t} s_{t+1}, r_{t+1}$$

For our future coin-collector environment, this could mean:

Current state:

Coin is to the right and above the player.

Action:

Move RIGHT.

Environment response:

Player moves right.

New state is calculated.

Reward is returned.

The objective of the agent is not merely to maximize the immediate reward from the next action. Instead, the agent tries to maximize the total reward it can obtain over time.

63 The Main Components of a Reinforcement Learning Problem

A reinforcement-learning problem contains several important components.

63.1 The Agent

The **agent** is the decision-maker.

It observes the current situation and chooses an action.

Examples include:

- a player in a video game,
- a robot navigating a room,
- an autonomous car,
- a computer program playing chess,
- our future C++ coin-collector agent.

63.2 The Environment

The **environment** is the world in which the agent acts.

The environment is responsible for:

- maintaining the current state,
- receiving an action,
- updating the world,
- calculating a reward,
- returning the next state,
- determining whether the episode has ended.

63.3 State

The **state**, denoted by s , represents the current situation of the environment.

For example, in a grid world:

$$s = \text{current grid cell}$$

In our future coin collector:

$$s = [\text{relative horizontal position of coin}, \text{relative vertical position of coin}]$$

The design of the state representation is extremely important because the agent can only make decisions using the information available in the state.

63.4 Action

The **action**, denoted by a , is a decision made by the agent.

For a simple game:

$$A = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$$

63.5 Reward

The **reward**, denoted by r , is a numerical signal returned by the environment.

For example:

$$r = \begin{cases} +10 & \text{if a coin is collected} \\ -20 & \text{if the player hits an enemy} \\ -0.01 & \text{for every ordinary step} \end{cases}$$

The reward tells the agent what is desirable, but does not directly tell it which action to choose.

The agent must discover useful behaviour through experience.

64 Episodes and Time Steps

An **episode** is one complete attempt by the agent.

For example:

```
Episode begins
  |
  v
Agent starts at initial position.
  |
  v
Agent takes action
  |
  v
Environment changes
  |
  v
Agent takes another action
  |
  v
  ...
  |
  v
Agent reaches goal or fails
  |
  v
Episode ends
```

A single episode contains many **time steps**.

At time t :

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

represents one interaction.

After the episode ends, the environment can be reset and a new episode begins.

Training may involve thousands or millions of episodes.

65 What is Q-Learning?

Q-Learning is a reinforcement-learning algorithm that learns how valuable different actions are in different states.

Its central object is the **Q-function**:

$$Q(s, a)$$

This can be read as:

How good is it to take action a while currently in state s , considering both the immediate reward and the rewards that may be obtained in the future?

The letter Q is commonly interpreted as referring to the **quality** of a state-action pair. Suppose an agent is in some state s , and the available actions have the following Q-values:

Action	Q-value
UP	7.2
DOWN	3.1
LEFT	2.4
RIGHT	5.8

The action with the largest Q-value is:

UP

because:

$$Q(s, \text{UP}) = 7.2$$

If the agent wants to exploit its current knowledge, it chooses:

$$a^* = \arg \max_a Q(s, a)$$

where $\arg \max$ means:

Return the action that produces the largest Q-value.

66 The Q-Table

In tabular Q-Learning, Q-values are stored in a table.

Suppose there are:

N

possible states and:

M

possible actions.

The Q-table has dimensions:

$$N \times M$$

For example, if there are 16 states and 4 actions:

$$Q \in \mathbb{R}^{16 \times 4}$$

A simplified Q-table could look like:

State	LEFT	DOWN	RIGHT	UP
0	0.10	0.30	0.50	0.20
1	0.20	0.80	0.40	0.10
2	0.60	0.30	0.90	0.20
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
15	0	0	0	0

The entry:

$$Q[2][\text{RIGHT}] = 0.9$$

means that according to the agent's current knowledge, taking the RIGHT action while in state 2 has a Q-value of 0.9.

At the beginning of training, the agent knows nothing.

Therefore, the table is often initialized with zeros:

$$Q(s, a) = 0$$

for every state-action pair.

In Python:

```
1 Qtable = np.zeros((state_size, action_size))
```

The agent then improves these values through experience.

67 Immediate Reward versus Future Reward

A major idea in reinforcement learning is that an action may be valuable even if it produces no immediate reward.

Consider:

```
START -> State 1 -> State 2 -> State 3 -> GOAL
0         0         0         0         +10
```

The first few actions give zero immediate reward.

However, they are still useful because they eventually lead to the goal.

Therefore, Q-Learning must consider both:

- immediate reward,
- expected future reward.

This leads to the concept of the **return**.

68 Return and Discounted Future Rewards

Suppose the agent receives future rewards:

$$r_{t+1}, r_{t+2}, r_{t+3}, \dots$$

A simple total would be:

$$r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

However, reinforcement learning commonly discounts rewards that occur further into the future.

The return is:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

where:

$$0 \leq \gamma \leq 1$$

and γ is called the **discount factor**.

If:

$$\gamma = 0$$

then the agent only cares about immediate rewards.

If:

$$\gamma \approx 1$$

then the agent strongly considers long-term rewards.

For example, suppose:

$$\gamma = 0.9$$

and the future rewards are:

$$0, \quad 0, \quad 10$$

Then:

$$G = 0 + 0.9(0) + 0.9^2(10)$$

Therefore:

$$G = 8.1$$

The future reward is still valuable, but its value is discounted because it occurs several steps later.

69 The Bellman Optimality Idea

Suppose the agent is currently in state s and chooses action a .

The environment gives:

- immediate reward r ,
- next state s' .

Once the agent reaches s' , it can choose another action.

If the agent behaves optimally, it should choose the action with the greatest Q-value:

$$\max_{a'} Q(s', a')$$

Therefore, the estimated value of taking action a in state s is related to:

$$r + \gamma \max_{a'} Q(s', a')$$

This gives the Bellman optimality relationship:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

The meaning is:

The value of the current action equals the immediate reward plus the discounted value of the best possible action from the next state.

70 A Numerical Bellman Example

Suppose:

$$r = 2$$

and after taking the current action, the agent reaches state s' .

At s' , suppose:

Action	Q-value
UP	4
DOWN	2
LEFT	1
RIGHT	8

Therefore:

$$\max_{a'} Q(s', a') = 8$$

If:

$$\gamma = 0.9$$

then the Bellman target is:

$$2 + 0.9(8)$$

which gives:

$$9.2$$

Therefore, according to the current information, the Q-value of the original state-action pair should move toward 9.2.

71 The Q-Learning Update Equation

In practice, the old Q-value is not necessarily replaced immediately by the Bellman target.

Instead, Q-Learning uses:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where:

- s is the current state,
- a is the selected action,
- r is the immediate reward,
- s' is the next state,
- γ is the discount factor,
- α is the learning rate.

The expression:

$$r + \gamma \max_{a'} Q(s', a')$$

is the **TD target**.

The expression:

$$r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

is the **temporal-difference error**, or TD error.

Therefore:

$$\text{TD error} = \text{target} - \text{current estimate}$$

and the update can be written more simply as:

$$Q_{\text{new}} = Q_{\text{old}} + \alpha(\text{TD error})$$

72 Numerical Example of a Complete Q-Learning Update

Suppose:

$$Q(s, a) = 4$$

The immediate reward is:

$$r = 2$$

The discount factor is:

$$\gamma = 0.9$$

The largest Q-value in the next state is:

$$\max_{a'} Q(s', a') = 8$$

The learning rate is:

$$\alpha = 0.1$$

First calculate the TD target:

$$\text{TD target} = 2 + 0.9(8)$$

Therefore:

$$\text{TD target} = 9.2$$

Now calculate the TD error:

$$\text{TD error} = 9.2 - 4$$

Therefore:

$$\text{TD error} = 5.2$$

Finally:

$$Q_{\text{new}} = 4 + 0.1(5.2)$$

Therefore:

$$Q_{\text{new}} = 4.52$$

The value does not immediately jump from 4 to 9.2.

Instead, it moves partially toward the target according to the learning rate α .

73 Understanding the Learning Rate α

The learning rate controls how strongly new experience changes existing knowledge.

$$0 \leq \alpha \leq 1$$

If α is small, such as:

$$\alpha = 0.1$$

then Q-values change slowly.

The agent gives significant weight to its previous estimates.

If α is large, such as:

$$\alpha = 0.9$$

then Q-values change aggressively.

The agent gives significant weight to new information.

Conceptually:

Low alpha:

Old knowledge >>> New experience

High alpha:

New experience >>> Old knowledge

A high learning rate can produce fast learning, but in stochastic environments it may also produce unstable estimates because individual random experiences have a large influence.

A lower learning rate usually produces slower but smoother learning.

74 How Reward Information Propagates Backwards

One of the most important ideas in Q-Learning is that reward information gradually propagates backward through previously visited states.

Consider:

```
S1 ----> S2 ----> S3 ----> GOAL
0         0         0         +10
```

Initially:

$$Q(s, a) = 0$$

for every state-action pair.

Suppose the agent reaches the goal from S_3 .

The action from S_3 receives information about the reward:

$$+10$$

Therefore:

$$Q(S_3, \text{toward goal})$$

becomes positive.

Later, when updating the action from S_2 to S_3 , the update contains:

$$\gamma \max_a Q(S_3, a)$$

Since the best Q-value at S_3 is now positive, the action from S_2 also becomes valuable.

Similarly, the value propagates backward to S_1 .

For example, with:

$$\gamma = 0.9$$

the values might conceptually become:

S1		S2		S3		GOAL
7.29	->	8.1	->	9	->	10

This is how an action can become valuable even if it gives no immediate reward.

The agent learns:

This action is useful because it leads to another state from which a valuable future reward can eventually be obtained.

75 The Exploration versus Exploitation Problem

Suppose an agent has the following Q-values:

Action	Q-value
UP	5.1
DOWN	2.4
LEFT	1.7
RIGHT	5.8

According to its current knowledge, RIGHT is best.

Therefore, exploitation means choosing:

$$\arg \max_a Q(s, a)$$

However, what if the agent's knowledge is incomplete?

Perhaps another action could lead to an even better strategy that has never been explored.

This creates the fundamental exploration–exploitation dilemma:

Exploration: Try actions that may not currently appear optimal in order to discover new possibilities.

Exploitation: Use the best action according to current knowledge.

If the agent only explores, it never consistently uses what it has learned.

If the agent only exploits from the beginning, it may never discover better strategies.

76 The Epsilon-Greedy Strategy

A common solution is the **epsilon-greedy strategy**.

Let:

$$\epsilon \in [0, 1]$$

Then:

$$a = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg \max_a Q(s, a) & \text{with probability } 1 - \epsilon \end{cases}$$

For example:

$$\epsilon = 0.1$$

means:

10% probability -> Choose a random action

90% probability -> Choose the best-known action

At the beginning of training, the agent knows almost nothing.

Therefore, ϵ can initially be large:

$$\epsilon = 1.0$$

The agent explores heavily.

As training progresses, ϵ is gradually reduced:

Beginning:

epsilon = 1.00

Later:

epsilon = 0.70

Later:

epsilon = 0.30

Near the end:

epsilon = 0.05

This process is called **epsilon decay**.

The general idea is:

Explore heavily at the beginning, then increasingly exploit the knowledge that has been learned.

77 Random Tie-Breaking During Exploitation

There is an important implementation detail when multiple actions have exactly the same maximum Q-value.

Suppose:

$$Q(s, \text{LEFT}) = 5$$

and:

$$Q(s, \text{RIGHT}) = 5$$

If a function such as `argmax` always returns the first maximum, the agent may systematically prefer LEFT simply because it has a smaller action index.

This creates an unnecessary bias.

A better strategy is:

1. Find the maximum Q-value.
2. Find every action having that maximum value.
3. Randomly select one of those actions.

In Python:

```

1 def choose_action(
2     Qtable,
3     state,
4     epsilon,
5     action_size
6 ):
7     if np.random.random() < epsilon:
8         action = np.random.randint(
9             0,
10            action_size
11        )
12
13    else:
14        max_q = np.max(Qtable[state, :])
15
16        optimal_actions = np.where(
17            Qtable[state, :] == max_q
18        )[0]
19
20        action = np.random.choice(
21            optimal_actions
22        )
23
24    return action

```

This gives every equally optimal action an equal opportunity to be selected.

78 The Complete Tabular Q-Learning Algorithm

The complete process is:

1. Initialize the Q-table.
2. Begin an episode.
3. Reset the environment and observe state s .
4. Select action a using epsilon-greedy.
5. Execute a in the environment.

6. Observe reward r and next state s' .

7. Calculate:

$$\max_{a'} Q(s', a')$$

8. Calculate the TD target:

$$r + \gamma \max_{a'} Q(s', a')$$

9. Calculate the TD error:

$$\text{target} - Q(s, a)$$

10. Update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(\text{TD error})$$

11. Set:

$$s \leftarrow s'$$

12. Repeat until the episode terminates.

13. Reduce epsilon.

14. Begin another episode.

In pseudocode:

Initialize $Q(s, a) = 0$

for each episode:

 state = reset environment

 while episode is not finished:

 choose action using epsilon-greedy

 next_state, reward, done = environment.step(action)

 max_future_q = max $Q(\text{next_state}, \text{all actions})$

 td_target = reward + gamma * max_future_q

 td_error = td_target - $Q(\text{state}, \text{action})$

$Q(\text{state}, \text{action}) =$
 $Q(\text{state}, \text{action})$
 + alpha * td_error

 state = next_state

 decay epsilon

79 Case Study: The FrozenLake Environment

We now apply Q-Learning to the classic **FrozenLake** environment.
This is the same type of environment used in our earlier Q-Learning experiment.
Consider a frozen lake represented as a 4×4 grid:

	S		F		F		F	
	F		H		F		H	
	F		F		F		H	
	H		F		F		G	

The symbols mean:

Symbol	Meaning
S	Starting position
F	Safe frozen surface
H	Hole
G	Goal

The agent begins at:

S

and must reach:

G

without falling into any hole.

80 Numbering the FrozenLake States

The 16 grid cells can be numbered:

	+		+		+		+		+
	0		1		2		3		
+		+		+		+		+	
	4		5		6		7		
+		+		+		+		+	
	8		9		10		11		
+		+		+		+		+	
	12		13		14		15		
+		+		+		+		+	

- For the standard map:
- State 0 is the starting state.
 - States 5, 7, 11 and 12 are holes.

- State 15 is the goal.

Therefore:

$$\text{number of states} = 16$$

81 FrozenLake Action Space

There are four possible actions:

Action number	Direction
0	LEFT
1	DOWN
2	RIGHT
3	UP

Therefore:

$$\text{number of actions} = 4$$

The Q-table has dimensions:

$$16 \times 4$$

In Python:

```

1 state_size = env.observation_space.n
2 action_size = env.action_space.n
3
4 Qtable = np.zeros(
5     (state_size, action_size)
6 )

```

Initially:

$$Q = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

82 Rewards in FrozenLake

The environment provides a sparse reward.

Conceptually:

$$r = \begin{cases} 1 & \text{if the agent reaches the goal} \\ 0 & \text{otherwise} \end{cases}$$

Falling into a hole terminates the episode without receiving the goal reward.

This creates an interesting learning problem.

Initially, the agent knows nothing:

$$Q(s, a) = 0$$

If it moves through safe states but never reaches the goal, it receives only zeros. The first successful discovery of the goal is therefore extremely important. Once the agent reaches the goal and receives:

$$r = 1$$

this reward information begins propagating backward through earlier states.

83 A FrozenLake Learning Example

Consider state 14, which is directly to the left of the goal:

```
+-----+-----+
| 14   | 15   |
|      | G    |
+-----+-----+
```

Suppose the agent is in state:

$$s = 14$$

and chooses:

$$a = \text{RIGHT}$$

It reaches the goal:

$$s' = 15$$

and receives:

$$r = 1$$

Since the episode terminates at the goal, there is no future action to consider. Therefore, the target is simply:

$$1$$

If initially:

$$Q(14, \text{RIGHT}) = 0$$

and:

$$\alpha = 0.8$$

then:

$$Q_{\text{new}} = 0 + 0.8(1 - 0)$$

Therefore:

$$Q_{\text{new}}(14, \text{RIGHT}) = 0.8$$

Now suppose the agent later reaches state 13 and moves RIGHT into state 14. The immediate reward is:

$$r = 0$$

But state 14 now contains valuable information:

$$\max_a Q(14, a) = 0.8$$

If:

$$\gamma = 0.95$$

then the target from state 13 becomes:

$$0 + 0.95(0.8)$$

Therefore:

$$\text{target} = 0.76$$

The reward information has now propagated backward from the goal to state 14, and then to state 13.

With repeated experience, this information spreads throughout the useful parts of the map.

84 Training FrozenLake with Q-Learning

A training function follows the standard Q-Learning process:

```

1 def train_agent(env, Qtable_init, params):
2
3     Qtable = Qtable_init.copy()
4
5     rewards_per_episode = []
6     steps_per_episode = []
7
8     epsilon = params['epsilon']
9
10    epsilon_decay_rate = (
11        params['epsilon_decay_rate']
12    )
13
14    for episode in range(
15        params['total_episodes']
16    ):
17
18        state, info = env.reset()
19
20        terminated = False
21        truncated = False
22
23        total_reward = 0
24        steps = 0
25
26        while (
27            not terminated
28            and not truncated
29            and steps <
30                params['max_steps_per_episode']
31        ):

```

```
32
33     action = choose_action(
34         Qtable,
35         state,
36         epsilon,
37         env.action_space.n
38     )
39
40     (
41         new_state,
42         reward,
43         terminated,
44         truncated,
45         info
46     ) = env.step(action)
47
48     max_future_q = np.max(
49         Qtable[new_state, :]
50     )
51
52     td_target = (
53         reward
54         + params['gamma']
55         * max_future_q
56     )
57
58     td_error = (
59         td_target
60         - Qtable[state, action]
61     )
62
63     Qtable[state, action] += (
64         params['learning_rate']
65         * td_error
66     )
67
68     state = new_state
69
70     total_reward += reward
71     steps += 1
72
73     epsilon = max(
74         params['min_epsilon'],
75         epsilon - epsilon_decay_rate
76     )
77
78     rewards_per_episode.append(
79         total_reward
80     )
81
82     steps_per_episode.append(steps)
83
84     return (
85         Qtable,
86         rewards_per_episode,
87         steps_per_episode
88     )
```

The four fundamental operations inside the training loop are:

1. **Action selection:** Choose an action using epsilon-greedy.
2. **Interaction:** Execute the action and observe the result.
3. **Learning:** Update $Q(s, a)$ using the Q-Learning equation.
4. **Transition:** Replace the current state with the next state.

85 Understanding the FrozenLake Training Code

The environment is reset:

```
1 state, info = env.reset()
```

The agent selects an action:

```
1 action = choose_action(  
2     Qtable,  
3     state,  
4     epsilon,  
5     env.action_space.n  
6 )
```

The environment executes it:

```
1 new_state, reward, terminated, truncated, info = (  
2     env.step(action)  
3 )
```

The maximum future Q-value is:

```
1 max_future_q = np.max(  
2     Qtable[new_state, :]  
3 )
```

This corresponds to:

$$\max_{a'} Q(s', a')$$

The TD target is:

```
1 td_target = (  
2     reward  
3     + gamma * max_future_q  
4 )
```

The TD error is:

```
1 td_error = (  
2     td_target  
3     - Qtable[state, action]  
4 )
```

Finally:

```
1 Qtable[state, action] += (  
2     learning_rate * td_error  
3 )
```


This directly implements:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

86 A Note About Terminal States

A more careful implementation should avoid adding future value after the episode has terminated.

The target can be written as:

$$\text{target} = r + \gamma(1 - \text{done}) \max_{a'} Q(s', a')$$

where:

$$\text{done} = \begin{cases} 1 & \text{if the episode has terminated} \\ 0 & \text{otherwise} \end{cases}$$

Therefore, if:

$$\text{done} = 1$$

then:

$$\text{target} = r$$

because no future actions occur after the terminal state.

In code:

```

1 done = terminated or truncated
2
3 if done:
4     td_target = reward
5 else:
6     td_target = (
7         reward
8         + gamma * max_future_q
9     )

```

This distinction becomes particularly important when we later implement more advanced RL algorithms.

87 Visualizing the Learned FrozenLake Policy

After training, every state contains four Q-values.

The optimal action is:

$$\pi(s) = \arg \max_a Q(s, a)$$

The action numbers can be mapped to arrows:

```

1 action_map = {
2     0: 'LEFT',
3     1: 'DOWN',
4     2: 'RIGHT',
5     3: 'UP'
6 }

```

Or visually:

0 →←

1 →↓

2 →→

3 →↑

The resulting policy might look conceptually like:

```
+-----+-----+-----+-----+
| v | > | v | < |
+-----+-----+-----+-----+
| v | H | v | H |
+-----+-----+-----+-----+
| > | v | v | H |
+-----+-----+-----+-----+
| H | > | > | G |
+-----+-----+-----+-----+
```

The exact policy can vary, particularly when several actions are equally valuable.

A successful policy should guide the agent toward the goal while avoiding the holes at states:

5, 7, 11, 12

88 Effect of the Learning Rate in FrozenLake

The learning rate α determines how aggressively the Q-table changes.

88.1 Low Learning Rate

For example:

$$\alpha = 0.1$$

A low learning rate means:

- each experience produces only a small update,
- learning is gradual,
- estimates may be more stable,
- more training episodes may be required.

88.2 High Learning Rate

For example:

$$\alpha = 0.9$$

A high learning rate means:

- new experiences strongly affect Q-values,
- learning can be faster,
- estimates may fluctuate more in stochastic environments.

In a sparse-reward environment such as FrozenLake, discovering the goal reward is a significant event.

A reasonably high learning rate can allow this newly discovered information to propagate more quickly through preceding states.

89 Effect of the Discount Factor in FrozenLake

The discount factor γ controls how strongly the agent values future rewards.

89.1 Lower Discount Factor

For example:

$$\gamma = 0.7$$

A reward several steps away is heavily discounted.

The agent is relatively short-sighted.

89.2 Higher Discount Factor

For example:

$$\gamma = 0.99$$

Future rewards remain highly valuable.

This is especially important in FrozenLake because most ordinary actions produce:

$$r = 0$$

The only positive reward is located at the distant goal.

Therefore, a high discount factor is often useful because it allows the goal reward to retain significant value as it propagates backward through earlier states.

90 Deterministic FrozenLake

In the deterministic version:

```
is_slippery = False
```

If the agent chooses RIGHT, it moves right, assuming the environment permits that transition.

Therefore:

Chosen action:
RIGHT

Actual movement:
RIGHT

The environment is predictable.
The same state-action pair always leads to the same outcome.
This makes the learning problem easier.

91 Slippery FrozenLake: A Stochastic Environment

The standard FrozenLake environment can also be slippery:

`is_slippery = True`

Now, choosing an action does not guarantee movement in the intended direction.
For example:

Chosen action:
RIGHT

Possible actual outcomes:
UP
RIGHT
DOWN

In the standard slippery environment, the intended movement is only one of the possible outcomes.

The environment is therefore **stochastic**.
A state-action pair does not always produce the same next state.

92 What Does the Q-Value Mean in a Stochastic Environment?

In a deterministic environment:

$$Q(s, a)$$

can be understood in terms of a predictable consequence.
In a stochastic environment, the same action can have multiple possible outcomes.
Therefore, the Q-value represents an **expected return** over those possible outcomes.
Conceptually:

$$Q(s, a) = \sum_{s'} P(s'|s, a) \left[r + \gamma \max_{a'} Q(s', a') \right]$$

where:

$$P(s'|s, a)$$

is the probability of reaching next state s' after taking action a in state s .
The agent does not necessarily calculate this probability explicitly in model-free Q-Learning.
Instead, repeated experience allows the Q-values to approximate these expected outcomes.

93 Risk-Aware Behaviour on the Slippery Lake

Consider two possible paths:

Path A:

Short but passes directly beside several holes.

Path B:

Longer but remains farther from dangerous holes.

In a deterministic environment, Path A may be optimal because every movement is reliable.

In a slippery environment, Path A may become dangerous.

Even if the agent chooses the correct action, it could accidentally slip into a hole.

Through repeated experience, Q-Learning can discover that a longer route has a greater expected return.

This is an important observation:

The optimal policy is not necessarily the geometrically shortest path. It is the policy that maximizes expected future return under the dynamics of the environment.

Therefore, a trained stochastic FrozenLake agent may exhibit risk-averse behaviour.

94 Why Tabular Q-Learning Works Well for FrozenLake

FrozenLake is suitable for tabular Q-Learning because both its state and action spaces are small and discrete.

For the 4×4 version:

16 states

and:

4 actions

Therefore, only:

$$16 \times 4 = 64$$

Q-values are required.

This is tiny.

The agent can explicitly store one value for every state-action pair.

95 The Fundamental Limitation of Tabular Q-Learning

Now consider our C++ coin-collector game.

Suppose the state is:

$$s = (dx, dy)$$

where:

$$dx = \frac{x_{\text{coin}} - x_{\text{player}}}{\text{window width}}$$

and:

$$dy = \frac{y_{\text{coin}} - y_{\text{player}}}{\text{window height}}$$

One possible state is:

$$(0.531245, -0.271826)$$

After moving slightly:

$$(0.524912, -0.271826)$$

These are technically different states.

A Q-table would need separate entries:

```
Q[(0.531245, -0.271826)] [UP]
Q[(0.531245, -0.271826)] [DOWN]
Q[(0.531245, -0.271826)] [LEFT]
Q[(0.531245, -0.271826)] [RIGHT]
```

```
Q[(0.524912, -0.271826)] [UP]
Q[(0.524912, -0.271826)] [DOWN]
...
```

There are essentially infinitely many possible continuous coordinate values.

The Q-table becomes impractical.

Even worse, ordinary tabular Q-Learning does not naturally generalize between similar states.

If it learns that:

$$dx = 0.500$$

should favour RIGHT, it does not automatically know that:

$$dx = 0.501$$

is almost the same situation.

This is the fundamental motivation for moving beyond a Q-table.

96 From Q-Learning Toward Deep Q-Learning

The central Q-Learning idea is:

$$Q(s, a)$$

For small discrete environments such as FrozenLake, we can explicitly store these values in a table.

However, for environments with large or continuous state spaces, this becomes impractical.

The natural question is:

Instead of storing the Q-value of every possible state-action pair explicitly, can we learn a function that estimates Q-values?

This is the motivation behind function approximation and, eventually, Deep Q-Learning. Instead of:

Q-table

we will eventually use:

Neural network approximating $Q(s, a)$

However, this introduces several new challenges and ideas.

Those concepts should be studied separately and carefully rather than treating DQN as merely “Q-Learning with a neural network.”

The next stage of this journey will therefore begin with the question:

How can a neural network replace the Q-table, and what exactly should its inputs, outputs, targets and loss function be?

“`latex`

97 From Q-Learning to Deep Q-Networks

After understanding tabular Q-learning, the next natural question is how reinforcement learning can be applied when the state space is continuous.

In a small discrete environment such as FrozenLake, a Q-table can explicitly store one value for every state-action pair:

$$Q(s, a)$$

For example, if there are 16 states and 4 actions, the Q-table has shape

$$16 \times 4.$$

Each row corresponds to a state, and each column corresponds to an action.

However, in the coin-collection game developed earlier, the state can be represented by the relative displacement between the player and the coin:

$$s = \begin{bmatrix} dx \\ dy \end{bmatrix},$$

where

$$dx = \frac{x_{\text{coin}} - x_{\text{player}}}{\text{window width}}$$

and

$$dy = \frac{y_{\text{coin}} - y_{\text{player}}}{\text{window height}}.$$

Since dx and dy are continuous floating-point values, there are effectively infinitely many possible states. It is therefore impractical to create a Q-table containing every possible state.

The solution is to replace the explicit table with a neural network.

Instead of storing

$$Q[s][a],$$

we approximate the Q-function using

$$Q(s, a; \theta),$$

where θ represents the neural-network parameters.

For the coin-collection game, the neural network takes the state

$$[dx, dy]$$

as input and produces four outputs:

$$[Q(s, \text{Up}), Q(s, \text{Down}), Q(s, \text{Left}), Q(s, \text{Right})].$$

This is the fundamental transition from tabular Q-learning to a Deep Q-Network, or DQN.

98 A Fundamental Question: If We Do Not Know the Optimal Q-Value, How Can We Define a Loss?

Question

At this stage, an important doubt arose:

“As far as I understand, a Q-table is just a memory element, and we use backpropagation to recursively update the cumulative rewards. But in the case of a neural network, we cannot clearly define a loss function because we do not initially know the optimal Q-value.”

This question reaches the central idea behind DQN.

First, one correction is necessary: a Q-table is indeed a memory structure, but tabular Q-learning does not use backpropagation.

The Q-table is updated directly using

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

There are no gradients, neural-network weights, or backpropagation involved. Suppose

$$Q(s, a) = 4,$$

and the Bellman target is

$$y = 9.2.$$

If

$$\alpha = 0.1,$$

then

$$Q(s, a) \leftarrow 4 + 0.1(9.2 - 4) = 4.52.$$

One table entry is modified directly.

The deeper question remains:

If a neural network predicts a Q-value, but the true optimal Q-value is unknown, what target can be used to calculate the loss?

The answer is that DQN creates a temporary learning target using the Bellman equation.

99 The Bellman Target and Bootstrapping

Suppose an experience consists of

$$(s, a, r, s'),$$

where

- s is the current state,
- a is the action taken,
- r is the immediate reward,
- s' is the resulting next state.

The Bellman target is

$$y = r + \gamma \max_{a'} Q(s', a')$$

for a non-terminal transition.

The important idea is that y is not necessarily the true optimal Q-value. It is the current best estimate of what the Q-value should move toward.

Consider the coin-collection game.

Suppose

$$s = [0.6, -0.2],$$

and the online neural network predicts

$$Q(s, :) = [2.1, -0.5, -1.3, 4.7].$$

If the action chosen was Right, then

$$Q(s, \text{Right}) = 4.7.$$

After executing the action, suppose the environment returns

$$r = -0.01$$

and

$$s' = [0.55, -0.2].$$

Suppose the network estimates the Q-values of the next state as

$$Q(s', :) = [2.3, -0.4, -1.1, 5.0].$$

Then

$$\max_{a'} Q(s', a') = 5.0.$$

With

$$\gamma = 0.99,$$

the Bellman target becomes

$$y = -0.01 + 0.99(5.0) = 4.94.$$

The network predicted

$$4.7,$$

while the temporary target is

$$4.94.$$

Therefore, a loss can now be defined:

$$L = (y - Q(s, a; \theta))^2$$

or numerically,

$$L = (4.94 - 4.7)^2.$$

Backpropagation can now be used to update the neural-network parameters.

This method of using an existing estimate to improve another estimate is called **bootstrapping**.

100 How Reward Information Propagates Backward

Consider a simple path:

$$S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \text{Coin}.$$

Suppose the agent receives no reward until collecting the coin:

$$S_1 \xrightarrow{0} S_2 \xrightarrow{0} S_3 \xrightarrow{+10} \text{Terminal}.$$

Initially, assume all Q-value estimates are approximately zero.

When the agent collects the coin from S_3 ,

$$r = 10.$$

Since the episode terminates, there is no future value. Therefore,

$$y = 10.$$

The network gradually learns

$$Q(S_3, \text{toward coin}) \approx 10.$$

Later, when the agent experiences the transition from S_2 to S_3 ,

$$r = 0.$$

With

$$\gamma = 0.9,$$

the target becomes

$$y = 0 + 0.9(10) = 9.$$

Therefore,

$$Q(S_2, \text{toward } S_3) \approx 9.$$

Later still, for the transition from S_1 to S_2 ,

$$y = 0 + 0.9(9) = 8.1.$$

Thus the reward information propagates backward:

$$\boxed{S_1 : 8.1 \rightarrow S_2 : 9 \rightarrow S_3 : 10 \rightarrow \text{Coin}}$$

This is the same Bellman bootstrapping principle used in ordinary tabular Q-learning.

101 The Problem with Using Only One Neural Network

A naive neural-network Q-learning loss would be

$$L = \left[r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta) \right]^2.$$

Notice that the same neural network appears in two places:

1. It produces the current prediction $Q(s, a; \theta)$.
2. It also helps construct the target through $Q(s', a'; \theta)$.

This creates a moving-target problem.

Suppose initially

$$Q(s, \text{Right}) = 4,$$

and

$$\max_{a'} Q(s', a') = 8.$$

With

$$r = 0, \quad \gamma = 0.9,$$

the target is

$$y = 7.2.$$

The network should therefore move its prediction from 4 toward 7.2.

However, after updating the network weights, suppose

$$Q(s, \text{Right})$$

changes from

$$4 \rightarrow 4.5,$$

but the same update also changes

$$\max_{a'} Q(s', a')$$

from

$$8 \rightarrow 8.7.$$

The target then becomes

$$0.9(8.7) = 7.83.$$

Thus the prediction moved toward the target, but the target itself also moved:

$$7.2 \rightarrow 7.83.$$

This can make learning unstable.

102 Online Network and Target Network

DQN addresses this problem by using two neural networks with identical architectures:

$$Q_{\text{online}}(s, a; \theta)$$

and

$$Q_{\text{target}}(s, a; \theta^-).$$

The online network is responsible for:

- selecting actions,
- producing the current Q-value predictions,
- receiving gradient updates through backpropagation.

The target network is responsible for producing relatively stable future-value estimates. The Bellman target becomes

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-)$$

and the loss becomes

$$L = [y - Q_{\text{online}}(s, a; \theta)]^2.$$

Only the online-network parameters θ are updated by gradient descent. The target-network parameters θ^- remain frozen temporarily. Initially,

$$\theta^- \leftarrow \theta.$$

Training may proceed as follows:

Training Step	Online Network	Target Network
0	θ_0	θ_0
1	θ_1	θ_0
2	θ_2	θ_0
3	θ_3	θ_0
$\vdots$	$\vdots$	$\vdots$
1000	θ_{1000}	θ_0

Then the online parameters are copied to the target network:

$$\boxed{\theta^- \leftarrow \theta}$$

so that

$$\theta^- = \theta_{1000}.$$

Training then continues while the target network remains frozen again.

This gives the online network a more stable objective to learn toward.

103 Why Gradients Do Not Flow Through the Target Network

During target calculation, we conceptually compute

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a').$$

This value is treated as a temporary label.

Therefore, when implementing DQN in PyTorch, target calculation is usually performed without gradient tracking:

```
with torch.no_grad():
    target = reward + gamma * max_next_q
```

The desired gradient is

$$\frac{\partial L}{\partial \theta},$$

where θ represents the online-network parameters.

We do not want the target network to be updated through

$$\frac{\partial L}{\partial \theta^-}.$$

The conceptual gradient flow is therefore

Target Network $\rightarrow$ Target $y \rightarrow$ Loss $\leftarrow$ Online Network,

but gradients flow backward only through the online network.

104 Experience Replay

Another problem arises when training directly on consecutive game experiences.

Suppose the player repeatedly moves toward the coin:

$$[0.80, -0.20] \rightarrow [0.79, -0.20] \rightarrow [0.78, -0.20] \rightarrow [0.77, -0.20].$$

These consecutive states are extremely similar and highly correlated.

Training sequentially on such data can make neural-network learning unstable and overly influenced by recent experiences.

There is also another problem: rare valuable experiences may be encountered only occasionally.

For example,

$$\text{Collect coin} \rightarrow r = +10$$

may be much rarer than ordinary movement transitions with

$$r = -0.01.$$

If a successful experience were used for one gradient update and immediately discarded, useful information would be wasted.

DQN solves these problems using an **experience replay buffer**.

104.1 What Is an Experience?

Every environment interaction produces one transition:

$$(s, a, r, s', done)$$

where

- s : current state,
- a : action taken,
- r : reward received,
- s' : next state,
- $done$: whether the episode terminated.

For example,

$$([0.60, -0.20], \text{Right}, -0.01, [0.55, -0.20], \text{false})$$

is one complete experience.

The replay buffer stores many such transitions:

$$D = \{e_1, e_2, e_3, \dots, e_N\}.$$

When the buffer reaches maximum capacity, old transitions can be removed as new transitions arrive.

104.2 Random Mini-Batch Sampling

Instead of training on the newest transition alone, DQN randomly samples a mini-batch from replay memory.

For example,

$$B = 64.$$

The sampled transitions are

$$\{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{64}.$$

Because they are sampled randomly, they may come from different episodes and very different game situations.

This reduces temporal correlation and allows useful experiences to be reused multiple times.

105 A Key Question About Batch Dimensions

Question

A particularly important implementation question arose:

“From the experience buffer, suppose we sample 64 transitions. Do we run those states through the online network again? Then we obtain a 64×4 matrix. From the target network, for the next states s' , we get a 64×1 matrix. How do we calculate the loss? Do we select only $Q(s, a)$, which is 64×1 , and train that?”

The answer is yes.

More precisely, both networks initially produce 64×4 matrices.

105.1 Online-Network Output

Suppose

$$S = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_{64} \end{bmatrix}$$

has shape

$$64 \times 2.$$

Passing these states through the online network gives

$$Q_{\text{online}}(S) \in \mathbb{R}^{64 \times 4}.$$

Each row contains

$$[Q(s_i, \text{Up}), Q(s_i, \text{Down}), Q(s_i, \text{Left}), Q(s_i, \text{Right})].$$

However, every stored transition contains only one action that was actually taken. Suppose, for four transitions, the network outputs

$$Q_{\text{online}}(S) = \begin{bmatrix} 2.1 & -0.5 & -1.3 & 4.7 \\ 1.0 & 3.2 & 5.1 & 0.4 \\ 4.8 & 2.1 & -0.7 & 1.3 \\ -1.2 & 0.5 & 3.7 & 2.4 \end{bmatrix}.$$

Let the action mapping be

$$0 = \text{Up}, \quad 1 = \text{Down}, \quad 2 = \text{Left}, \quad 3 = \text{Right}.$$

Suppose the actual actions were

$$A = \begin{bmatrix} 3 \\ 2 \\ 0 \\ 2 \end{bmatrix}.$$

We select only the corresponding Q-values:

$$\begin{aligned}
Q(s_1, a_1) &= 4.7, \\
Q(s_2, a_2) &= 5.1, \\
Q(s_3, a_3) &= 4.8, \\
Q(s_4, a_4) &= 3.7.
\end{aligned}$$

Thus

$$Q_{\text{predicted}} = \begin{bmatrix} 4.7 \\ 5.1 \\ 4.8 \\ 3.7 \end{bmatrix}.$$

For batch size 64, the resulting selected prediction vector has shape

$$\boxed{64 \times 1}.$$

105.2 Target-Network Output

The next states

$$S' = \begin{bmatrix} s'_1 \\ s'_2 \\ \vdots \\ s'_{64} \end{bmatrix}$$

are passed through the target network.

The target network initially also produces

$$Q_{\text{target}}(S') \in \mathbb{R}^{64 \times 4}.$$

For each row, we take the maximum over the four actions:

$$\max_{a'} Q_{\text{target}}(s'_i, a').$$

This gives a vector of shape

$$\boxed{64 \times 1}.$$

The Bellman targets are then

$$\boxed{y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\text{target}}(s'_i, a')}$$

where $d_i = 1$ for a terminal transition and $d_i = 0$ otherwise.

Therefore,

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{64} \end{bmatrix}$$

also has shape

$$64 \times 1.$$

The loss is then calculated between

$$\underbrace{\begin{bmatrix} Q_{\text{online}}(s_1, a_1) \\ Q_{\text{online}}(s_2, a_2) \\ \vdots \\ Q_{\text{online}}(s_{64}, a_{64}) \end{bmatrix}}_{64 \times 1}$$

and

$$\underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{64} \end{bmatrix}}_{64 \times 1}.$$

For mean squared error,

$$L = \frac{1}{64} \sum_{i=1}^{64} [Q_{\text{online}}(s_i, a_i) - y_i]^2.$$

105.3 Why Only the Chosen Action Is Directly Trained

Suppose a transition is

$$(s, \text{Right}, r, s').$$

The environment tells us what happened when the agent chose Right.

It does not tell us what would have happened if the agent had chosen Up, Down, or Left from that same state.

Therefore, this transition directly provides a learning target only for

$$Q(s, \text{Right}).$$

The other three action values do not directly contribute to the loss for this particular experience.

However, because neural-network parameters are shared, backpropagation may indirectly alter predictions for other actions and other states.

Thus,

Only the selected action contributes directly to the loss,

but

updating shared parameters can indirectly affect many predictions.

106 The Complete DQN Training Loop

The major components of DQN can now be combined into one process:

Online Network + Target Network + Replay Buffer + ϵ -Greedy Exploration

At the beginning of training:

1. Initialize the online network with parameters θ .
2. Initialize the target network with parameters θ^- .
3. Copy

$$\theta^- \leftarrow \theta.$$

4. Create an empty replay buffer.
5. Initialize the exploration parameter ϵ .

At each environment step:

1. Observe the current state s .
2. Choose an action using ϵ -greedy exploration.
3. Execute the action in the environment.
4. Receive r , s' , and *done*.
5. Store

$$(s, a, r, s', done)$$

in replay memory.

6. If enough experiences exist, sample a random mini-batch.
7. Compute online-network predictions for the chosen actions.
8. Compute Bellman targets using the target network.
9. Calculate the batch loss.
10. Backpropagate through the online network.
11. Periodically copy the online parameters to the target network.
12. Set

$$s \leftarrow s'.$$

106.1 Epsilon-Greedy Action Selection

The action is selected according to

$$a = \begin{cases} \text{random action,} & u < \epsilon, \\ \arg \max_a Q_{\text{online}}(s, a), & u \geq \epsilon, \end{cases}$$

where

$$u \sim U(0, 1).$$

Early in training, ϵ is large, encouraging exploration.

Later, ϵ decreases, causing the agent to exploit what it has learned more frequently.

106.2 Environment Steps, Training Steps, and Episodes

These terms should be distinguished carefully.

An **environment step** is one action executed in the game:

$$s_t \xrightarrow{a_t} s_{t+1}.$$

A **training step** is one gradient update:

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

An **episode** begins with

```
s0 = environment.reset()
```

and ends when a terminal condition occurs.

For the initial coin-collection environment, an episode ends when either

- the player collects the coin, or
- the maximum number of allowed steps is reached.

107 Moving from DQN Theory to the C++ Environment

After understanding the core DQN theory, the next task was to convert the existing SFML coin-collection game into a proper reinforcement-learning environment.

A normal human-controlled game follows the flow

Keyboard Input → Update Game → Render

whereas an RL environment should expose an interface of the form

reset() → s

and

step(a) → (s', r, done).

The project was organized with separate entry points for the normal game and the RL environment:

```
project/
|
|-- include/
|   |-- Player.hpp
|   |-- Coin.hpp
|   |-- Game.hpp
|   '-- GameEnvironment.hpp
|
|-- src/
|   |-- Player.cpp
|   |-- Coin.cpp
|   |-- Game.cpp
```

```
| |-- GameEnvironment.cpp
| |-- main_game.cpp
| '-- main_rl.cpp
```

The two programs have different purposes:

`main_game.cpp`

```
|
v
Human-controlled Game
|
|-- Keyboard input
|-- Real elapsed time
|-- SFML rendering
'-- Human gameplay
```

`main_rl.cpp`

```
|
v
GameEnvironment
|
|-- reset()
|-- getState()
|-- step(Action)
|-- Fixed simulation timestep
'-- No keyboard dependency
```

Both reuse the same `Player` and `Coin` classes.

108 The Fundamental RL Data Structures

The following structures were introduced in `GameEnvironment.hpp`.

108.1 Action

```
enum class Action
{
    Up = 0,
    Down = 1,
    Left = 2,
    Right = 3
};
```

This mapping corresponds directly to the four outputs of the DQN:

$$[Q_{\text{Up}}, Q_{\text{Down}}, Q_{\text{Left}}, Q_{\text{Right}}].$$

Using an `enum class` is clearer than passing arbitrary integers such as

```
env.step(3);
```

Instead, one can write

```
env.step(Action::Right);
```

108.2 State

The state was defined as

```
struct State
{
    float dx;
    float dy;
};
```

where

$$dx = \frac{x_{\text{coin}} - x_{\text{player}}}{\text{window width}}$$

and

$$dy = \frac{y_{\text{coin}} - y_{\text{player}}}{\text{window height}}.$$

This normalized representation usually keeps values approximately within

$$[-1, 1].$$

For example, if

$$\text{Player} = (100, 300),$$

$$\text{Coin} = (600, 150),$$

and the window is

$$800 \times 600,$$

then

$$dx = \frac{600 - 100}{800} = 0.625,$$

and

$$dy = \frac{150 - 300}{600} = -0.25.$$

Therefore,

$$s = [0.625, -0.25].$$

In SFML screen coordinates,

$$+x$$

points right and

$$+y$$

points downward.

Thus,

- $dx > 0$: coin is to the right,

- $dx < 0$: coin is to the left,
- $dy > 0$: coin is below,
- $dy < 0$: coin is above.

108.3 StepResult

After receiving an action, the environment returns

```
struct StepResult
{
    State nextState;
    float reward;
    bool done;
};
```

The agent already knows the current state s and the action a it chose.
The environment returns

$$(s', r, done).$$

Together, they form the complete replay-buffer transition:

$$(s, a, r, s', done).$$

109 The GameEnvironment Class

The environment interface was designed as follows:

```
#pragma once

#include "Player.hpp"
#include "Coin.hpp"

enum class Action
{
    Up = 0,
    Down = 1,
    Left = 2,
    Right = 3
};

struct State
{
    float dx;
    float dy;
};

struct StepResult
{
    State nextState;
    float reward;
```

```

        bool done;
    };

class GameEnvironment
{
private:
    Player player;
    Coin coin;

    int windowWidth;
    int windowHeight;

    int currentStep;
    int maxSteps;

    std::mt19937 generator;

public:
    GameEnvironment(int width, int height);

    State reset();

    State getState() const;

    StepResult step(Action action);
};

```

The environment owns the player and coin because they are components of the world.

The RL agent should not directly manipulate internal game objects. Instead, interaction should occur through

```

State state = env.reset();

StepResult result = env.step(Action::Right);

```

110 Why a Separate `Player::move()` Function Was Needed

An important doubt arose during the conversion:

“Why do I need a `void move()` function? What problem does it solve? The `handleInput()` function still contains keyboard operations.”

The key is to separate two responsibilities:

from Deciding how to move

Actually moving the player.

Originally, `Player::handleInput()` performed both tasks:

1. Read keyboard input.

2. Convert keyboard input into a direction.
3. Move the player.

For human gameplay, the flow is

Keyboard $\rightarrow$ `handleInput()` $\rightarrow$ Player movement.

However, the DQN does not press physical keyboard keys.

It produces an action such as

`Action::Right.`

Therefore, the RL environment cannot use

```
player.handleInput(dt);
```

because that function asks whether physical keys are pressed.

The movement logic was therefore separated:

```
void Player::move(sf::Vector2f direction, float dt)
{
    if (direction.x != 0.0f || direction.y != 0.0f)
    {
        float length = std::sqrt(
            direction.x * direction.x +
            direction.y * direction.y
        );

        direction.x /= length;
        direction.y /= length;
    }

    shape.move(direction * speed * dt);
}
```

The keyboard function can still contain keyboard operations:

```
void Player::handleInput(float dt)
{
    sf::Vector2f direction(0.0f, 0.0f);

    if (sf::Keyboard::isKeyPressed(sf::Keyboard::W))
        direction.y -= 1.0f;

    if (sf::Keyboard::isKeyPressed(sf::Keyboard::S))
        direction.y += 1.0f;

    if (sf::Keyboard::isKeyPressed(sf::Keyboard::A))
        direction.x -= 1.0f;

    if (sf::Keyboard::isKeyPressed(sf::Keyboard::D))
        direction.x += 1.0f;

    move(direction, dt);
}
```


The final architecture becomes

```
Human:
Keyboard
  |
  v
handleInput()
  |
  v
direction
  |
  v
move()
```

```
RL Agent:
Action::Right
  |
  v
GameEnvironment::step()
  |
  v
direction = (1, 0)
  |
  v
move()
```

Thus,

- `handleInput()` answers: “What does the human want to do?”
- `step(Action)` answers: “What does the RL agent want to do?”
- `move()` answers: “Actually move the player.”

The human-controlled game still uses keyboard input, while the RL environment never needs to call `handleInput()`.

111 Implementing `getState()`

The environment converts the internal SFML world into the numerical state required by the neural network:

```
State GameEnvironment::getState() const
{
    sf::Vector2f playerPosition = player.getPosition();
    sf::Vector2f coinPosition = coin.getPosition();

    State state;

    state.dx =
        (coinPosition.x - playerPosition.x)
        / static_cast<float>(windowWidth);
```

```

        state.dy =
            (coinPosition.y - playerPosition.y)
            / static_cast<float>(windowHeight);

        return state;
    }

```

This creates the bridge

Internal SFML World

```

    |
    | Player position
    | Coin position
    v

```

```

getState()

```

```

    |
    v
State {dx, dy}

```

Future DQN Input

112 Implementing reset()

The responsibility of `reset()` is to begin a new episode.

It should

1. reset the step counter,
2. place the player at a valid starting position,
3. place the coin at a valid random position,
4. ensure the player and coin do not initially overlap,
5. return the initial state.

The player was given a public position-setting function:

```

void Player::setPosition(float x, float y)
{
    shape.setPosition(x, y);
}

```

The coin was also given a window-independent respawn overload:

```

void Coin::respawn(unsigned int width, unsigned int height)
{
    float diameter = shape.getRadius() * 2.0f;

    float maxX = static_cast<float>(width) - diameter;
    float maxY = static_cast<float>(height) - diameter;
}

```

```

std::uniform_real_distribution<float> randomX(0.0f, maxX);
std::uniform_real_distribution<float> randomY(0.0f, maxY);

shape.setPosition(
    randomX(generator),
    randomY(generator)
);
}

```

This allows the human game to use

```
coin.respawn(window);
```

while the RL environment can use

```
coin.respawn(windowWidth, windowHeight);
```

without requiring an SFML render window.

A reset function using the SFML API version with `width`, `height`, and `intersects()` is

```

State GameEnvironment::reset()
{
    currentStep = 0;

    float playerWidth = player.getBounds().width;
    float playerHeight = player.getBounds().height;

    std::uniform_real_distribution<float> randomPlayerX(
        0.0f,
        static_cast<float>(windowWidth) - playerWidth
    );

    std::uniform_real_distribution<float> randomPlayerY(
        0.0f,
        static_cast<float>(windowHeight) - playerHeight
    );

    player.setPosition(
        randomPlayerX(generator),
        randomPlayerY(generator)
    );

    do
    {
        coin.respawn(windowWidth, windowHeight);
    }
    while (player.getBounds().intersects(coin.getBounds()));

    return getState();
}

```

Randomizing both the player and coin encourages the agent to learn a general policy based on relative displacement rather than memorizing one fixed starting position.

113 Implementing step(Action)

The `step()` function is the heart of the RL environment.

Given an action, it must

1. convert the action into a movement direction,
2. move the player,
3. keep the player inside the environment,
4. increment the step counter,
5. check whether the coin was collected,
6. calculate the reward,
7. determine whether the episode terminated,
8. return the next state, reward, and terminal flag.

113.1 Why Use a Fixed Timestep?

A human-controlled real-time game may calculate

```
float dt = clock.restart().asSeconds();
```

and move according to actual elapsed wall-clock time.

This is not appropriate for fast RL training.

If the environment is running without rendering, one iteration may take only a tiny fraction of a second. Movement based on real elapsed time would then become hardware-dependent and inconsistent.

Instead, a fixed simulation timestep is used:

```
constexpr float FIXED_DT = 1.0f / 60.0f;
```

If the player speed is

300 pixels/second,

then every RL action moves

$$300 \times \frac{1}{60} = 5 \text{ pixels.}$$

Therefore,

`Action::Right`

always means exactly one deterministic 5-pixel movement to the right, regardless of how quickly the computer executes the simulation.

113.2 Converting Actions to Directions

The discrete actions are converted into SFML direction vectors:

```
sf::Vector2f direction(0.0f, 0.0f);

switch (action)
{
    case Action::Up:
        direction.y = -1.0f;
        break;

    case Action::Down:
        direction.y = 1.0f;
        break;

    case Action::Left:
        direction.x = -1.0f;
        break;

    case Action::Right:
        direction.x = 1.0f;
        break;
}
```

The resulting directions are

$$\begin{aligned}\text{Up} &= (0, -1), \\ \text{Down} &= (0, +1), \\ \text{Left} &= (-1, 0), \\ \text{Right} &= (+1, 0).\end{aligned}$$

The player is then moved with

```
player.move(direction, FIXED_DT);
```

114 Reusing Boundary Logic

Instead of duplicating boundary-handling code inside `GameEnvironment`, the existing `Player` class was extended with an overload:

```
void keepInsideBounds(const sf::RenderWindow& window);
void keepInsideBounds(float width, float height);
```

The RL environment can call

```
player.keepInsideBounds(
    static_cast<float>(windowWidth),
    static_cast<float>(windowHeight)
);
```

A clean implementation avoids duplicating the actual boundary logic:

```

void Player::keepInsideBounds(const sf::RenderWindow& window)
{
    keepInsideBounds(
        static_cast<float>(window.getSize().x),
        static_cast<float>(window.getSize().y)
    );
}

```

The numerical overload contains the actual implementation:

```

void Player::keepInsideBounds(float width, float height)
{
    sf::Vector2f position = shape.getPosition();
    sf::FloatRect bounds = shape.getGlobalBounds();

    if (position.x < 0.0f)
        position.x = 0.0f;

    if (position.y < 0.0f)
        position.y = 0.0f;

    if (position.x + bounds.width > width)
        position.x = width - bounds.width;

    if (position.y + bounds.height > height)
        position.y = height - bounds.height;

    shape.setPosition(position);
}

```

This allows both the human game and RL environment to reuse the same boundary logic.

115 Reward and Episode Termination

The initial reward function was chosen as

$$r = \begin{cases} +10, & \text{if the coin is collected,} \\ -0.01, & \text{otherwise.} \end{cases}$$

The small step penalty encourages shorter paths.

For example, a 20-step successful path gives approximately

$$10 - 20(0.01) = 9.8,$$

whereas a 100-step path gives approximately

$$10 - 100(0.01) = 9.0.$$

Therefore, reaching the coin quickly is preferable.

For the first version,

One episode = attempt to collect one coin.

The episode ends when either

- the coin is collected, or
- the maximum number of steps is reached.

The initial collision and reward logic is

```
float reward = -0.01f;
bool done = false;

if (player.getBounds().intersects(coin.getBounds()))
{
    reward = 10.0f;
    done = true;
}

if (currentStep >= maxSteps)
{
    done = true;
}
```

116 Complete Initial step() Implementation

The complete first version is

```
StepResult GameEnvironment::step(Action action)
{
    constexpr float FIXED_DT = 1.0f / 60.0f;

    sf::Vector2f direction(0.0f, 0.0f);

    switch (action)
    {
        case Action::Up:
            direction.y = -1.0f;
            break;

        case Action::Down:
            direction.y = 1.0f;
            break;

        case Action::Left:
            direction.x = -1.0f;
            break;

        case Action::Right:
            direction.x = 1.0f;
            break;
    }

    player.move(direction, FIXED_DT);

    player.keepInsideBounds(
```

```

        static_cast<float>(windowWidth),
        static_cast<float>(windowHeight)
    );

    currentStep++;

    float reward = -0.01f;
    bool done = false;

    if (player.getBounds().intersects(coin.getBounds()))
    {
        reward = 10.0f;
        done = true;
    }

    if (currentStep >= maxSteps)
    {
        done = true;
    }

    return {
        getState(),
        reward,
        done
    };
}

```

The environment now provides the fundamental RL interface

$\text{reset}() \rightarrow s$

and

$\text{step}(a) \rightarrow (s', r, done).$

117 Testing the Environment with a Random Agent

Before connecting the environment to a neural network, it is important to test it with a random agent.

The random agent chooses uniformly among

{Up, Down, Left, Right}.

The test program in `main_rl.cpp` follows the same basic structure that will later be used by the DQN:

```

#include <iostream>
#include <random>

#include "GameEnvironment.hpp"

int main()

```



```
{
    GameEnvironment env(800, 600);

    std::random_device rd;
    std::mt19937 generator(rd());

    std::uniform_int_distribution<int> actionDistribution(0, 3);

    const int numberOfEpisodes = 1000;

    int totalSuccesses = 0;

    for (int episode = 1;
        episode <= numberOfEpisodes;
        episode++)
    {
        State state = env.reset();

        bool done = false;
        float totalReward = 0.0f;
        int steps = 0;

        while (!done)
        {
            int actionIndex = actionDistribution(generator);

            Action action =
                static_cast<Action>(actionIndex);

            StepResult result = env.step(action);

            totalReward += result.reward;
            steps++;

            done = result.done;
            state = result.nextState;
        }

        if (totalReward > 0.0f)
        {
            totalSuccesses++;
        }

        std::cout
            << "Episode " << episode
            << " | Steps: " << steps
            << " | Reward: " << totalReward
            << " | Successes: " << totalSuccesses
            << '\n';
    }
}
```

```

std::cout
    << "\nSuccess rate: "
    << (100.0f * totalSuccesses / numberOfEpisodes)
    << "%\n";

return 0;
}

```

118 Random-Agent Baseline Result

The random-agent test was successfully executed for 1000 episodes.

The observed success rate was

3.6%

.

This is an important baseline.

The random agent has no understanding of the state

 $[dx, dy]$.

Its actions are completely independent of the location of the coin.

Therefore, the measured 3.6% success rate represents the performance of an agent with no learned policy.

When the DQN is implemented, its performance can be compared against this baseline.

A successful learning process should eventually achieve a substantially higher success rate and should generally require fewer steps to reach the coin.

119 How the Random Agent Relates to the Future DQN

The random-agent loop already resembles the future DQN training loop.

Currently,

State $\rightarrow$ Random Action $\rightarrow$ `env.step(action)` $\rightarrow (r, s', done)$

Later, the flow will become

```

State  $\rightarrow$   $\epsilon$ -greedy action from online network
     $\rightarrow$  env.step(action)
     $\rightarrow (r, s', done)$ 
     $\rightarrow$  store  $(s, a, r, s', done)$  in replay buffer
     $\rightarrow$  sample mini-batch
     $\rightarrow$  calculate Bellman targets
     $\rightarrow$  calculate loss
     $\rightarrow$  backpropagate through online network.

```

Thus, the C++ environment is now functionally prepared for reinforcement learning.

120 Current Architecture of the Project

At this stage, the system can be viewed as two separate layers.

The first is the C++ environment:

$$\text{Action} \rightarrow \text{C++ GameEnvironment} \rightarrow (\text{Next State}, \text{Reward}, \text{Done})$$

Its responsibilities are

- maintaining player and coin positions,
- applying actions,
- enforcing boundaries,
- detecting collisions,
- calculating rewards,
- deciding when an episode terminates,
- producing numerical state representations.

The future Python/PyTorch agent will be responsible for

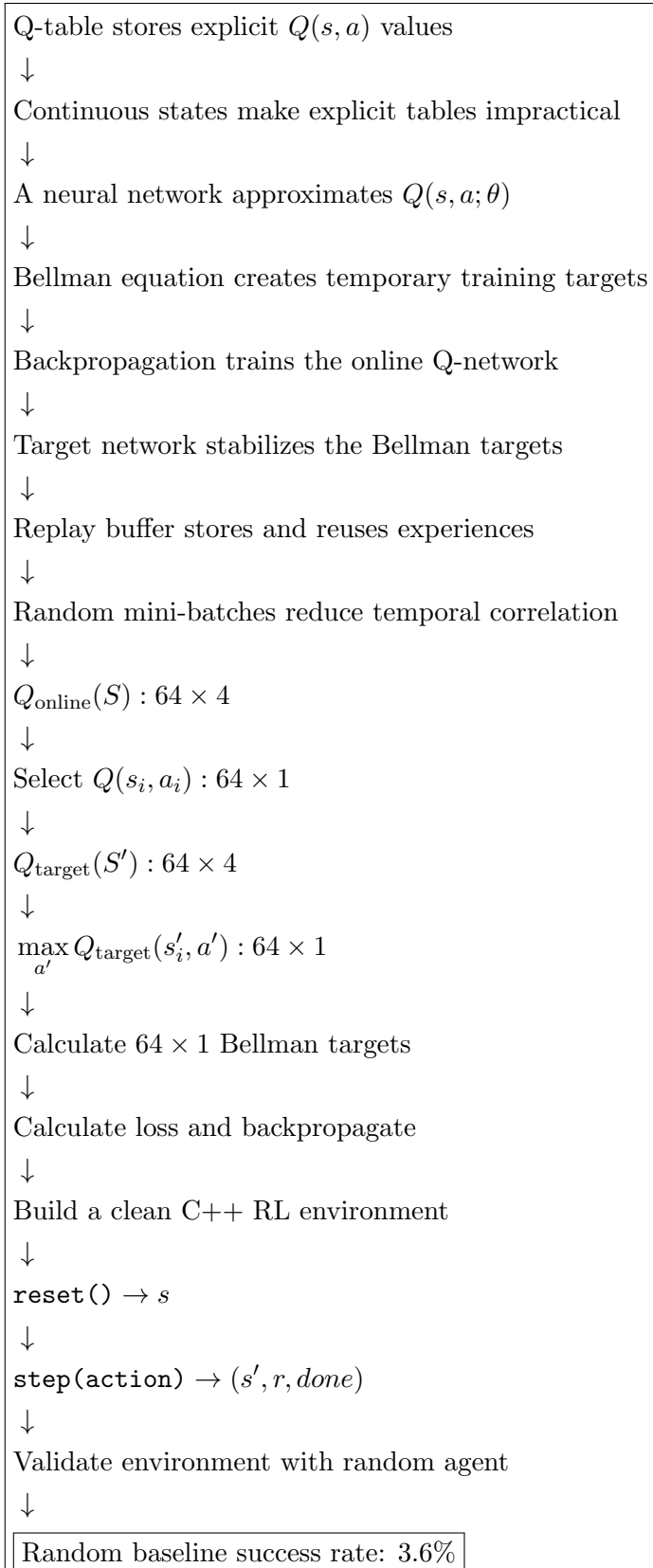
- running the online neural network,
- selecting actions using ϵ -greedy exploration,
- storing experiences,
- sampling replay batches,
- calculating Bellman targets,
- calculating loss,
- performing backpropagation,
- periodically updating the target network.

The interface between the two sides can be summarized as

$$\text{Python Agent} \xrightarrow{\text{Action}} \text{C++ Environment} \xrightarrow{s', r, done} \text{Python Agent.}$$

121 Summary of the Journey So Far

The progression from a simple Q-table to the current C++ RL environment can be summarized as follows:



The next major engineering problem is to connect the C++ environment with the Python/PyTorch DQN agent in a way that is simple, efficient, and easy to debug.

Once that interface is established, the complete DQN training loop can be implemented on top of the environment developed here.

“`latex`

122 Connecting the C++ Environment to Python

At this stage, we had successfully created a reinforcement learning environment in C++. The environment contained:

- A player.
- A coin.
- A discrete action space:

$$\mathcal{A} = \{\text{Up, Down, Left, Right}\}.$$

- A state representation based on the relative position between the player and the coin:

$$s = \begin{bmatrix} dx \\ dy \end{bmatrix}.$$

- A reward function.
- A terminal condition.
- A `reset()` function.
- A `step(action)` function.

The next major question was:

How can a reinforcement learning agent written in Python interact with an environment written in C++?

This is an important architectural question.

PyTorch is primarily used through Python, whereas our game environment was written in C++. We therefore needed a bridge between the two languages.

Our desired architecture was:

$$\boxed{\text{Python DQN Agent} \longleftrightarrow \text{C++ Game Environment}}$$

More specifically:

$$\text{Python} \xrightarrow{\text{env.step(action)}} \text{C++} \xrightarrow{\text{State, Reward, Done}} \text{Python}.$$

We used **pybind11** to create this bridge.

123 What is pybind11?

pybind11 is a lightweight C++ library that allows C++ classes, functions, structures, and enumerations to be exposed directly to Python.

Suppose we have a C++ class:

```

class GameEnvironment
{
public:
    GameEnvironment(int width, int height);

    State reset();

    StepResult step(Action action);
};

```

Normally, Python cannot directly do this:

```

env = GameEnvironment(800, 600)
state = env.reset()

```

However, after creating pybind11 bindings and compiling the C++ code into a Python extension module, Python can import the C++ environment almost like an ordinary Python library:

```

import game_env

env = game_env.GameEnvironment(800, 600)

state = env.reset()

result = env.step(game_env.Action.Right)

```

The important fact is that `GameEnvironment` is still a C++ object.

The environment logic is still executed by compiled C++ code. Python merely receives an interface through which it can create objects and call functions.

124 The C++ Structures Exposed to Python

Our environment used a state structure similar to:

```

struct State
{
    float dx;
    float dy;
};

```

The state represented the relative displacement between the player and the coin. For example:

$$dx = x_{\text{coin}} - x_{\text{player}},$$

$$dy = y_{\text{coin}} - y_{\text{player}}.$$

Depending on the exact implementation, these values could also be normalized by the dimensions of the environment.

The environment also returned a result from every action:

```
struct StepResult
{
    State nextState;
    float reward;
    bool done;
};
```

This corresponds exactly to the standard reinforcement learning transition:

$$(s, a, r, s', d),$$

where:

- s is the current state,
- a is the selected action,
- r is the received reward,
- s' is the next state,
- d indicates whether the episode terminated.

Our action enumeration was:

```
enum class Action
{
    Up,
    Down,
    Left,
    Right
};
```

125 Creating the pybind11 Bindings

We created a C++ source file:

src/bindings.cpp

A representative binding implementation was:

```
#include <pybind11/pybind11.h>

#include "GameEnvironment.hpp"

namespace py = pybind11;

PYBIND11_MODULE(game_env, m)
{
    py::enum_<Action>(m, "Action")
        .value("Up", Action::Up)
        .value("Down", Action::Down)
        .value("Left", Action::Left)
        .value("Right", Action::Right);
```

```

py::class_<State>(m, "State")
    .def_readonly("dx", &State::dx)
    .def_readonly("dy", &State::dy);

py::class_<StepResult>(m, "StepResult")
    .def_readonly(
        "nextState",
        &StepResult::nextState
    )
    .def_readonly(
        "reward",
        &StepResult::reward
    )
    .def_readonly(
        "done",
        &StepResult::done
    );

py::class_<GameEnvironment>(
    m,
    "GameEnvironment"
)
    .def(py::init<int, int>())
    .def(
        "reset",
        &GameEnvironment::reset
    )
    .def(
        "getState",
        &GameEnvironment::getState
    )
    .def(
        "step",
        &GameEnvironment::step
    );
}

```

Let us understand what each part does.

125.1 The PYBIND11_MODULE Macro

The line

```
PYBIND11_MODULE(game_env, m)
```

defines a Python module called:

```
game_env
```

After compilation, Python can therefore write:

```
import game_env
```

The name inside PYBIND11_MODULE must match the name of the compiled Python extension module.

125.2 Binding the Action Enumeration

The code

```
py::enum_<Action>(m, "Action")
    .value("Up", Action::Up)
    .value("Down", Action::Down)
    .value("Left", Action::Left)
    .value("Right", Action::Right);
```

allows Python to access the C++ enumeration as:

```
game_env.Action.Up
game_env.Action.Down
game_env.Action.Left
game_env.Action.Right
```

This became important when the Python DQN returned an integer action index. For example, we used:

```
ACTIONS = [
    game_env.Action.Up,
    game_env.Action.Down,
    game_env.Action.Left,
    game_env.Action.Right
]
```

Therefore:

$0 \rightarrow \text{Up},$

$1 \rightarrow \text{Down},$

$2 \rightarrow \text{Left},$

$3 \rightarrow \text{Right}.$

125.3 Binding the State Structure

The code

```
py::class_<State>(m, "State")
    .def_readonly("dx", &State::dx)
    .def_readonly("dy", &State::dy);
```

allows Python to access:

```
state.dx
state.dy
```

Therefore:

```
state_obj = env.reset()
```

```
state = [
    state_obj.dx,
    state_obj.dy
]
```

converts the C++ state into a simple Python list.

125.4 Binding the StepResult Structure

Similarly:

```
py::class_<StepResult>(m, "StepResult")
    .def_readonly(
        "nextState",
        &StepResult::nextState
    )
    .def_readonly(
        "reward",
        &StepResult::reward
    )
    .def_readonly(
        "done",
        &StepResult::done
    );
```

allows Python to do:

```
result = env.step(action)

next_state = [
    result.nextState.dx,
    result.nextState.dy
]

reward = result.reward
done = result.done
```

126 Compiling the C++ Environment into a Python Module

Our project structure was:

```
first_game/
|
|-- include/
|   |-- Player.hpp
|   |-- Coin.hpp
|   '-- GameEnvironment.hpp
|
|-- src/
|   |-- Player.cpp
```

```
|  |-- Coin.cpp
|  |-- GameEnvironment.cpp
|  '-- bindings.cpp
|
|-- rl_training/
|   |-- dqn.py
|   |-- replay_buffer.py
|   '-- train.py
```

We compiled the C++ environment using:

```
c++ -O3 -Wall -shared -std=c++17 -fPIC \
    $(python3 -m pybind11 --includes) \
    -Iinclude \
    src/bindings.cpp \
    src/GameEnvironment.cpp \
    src/Player.cpp \
    src/Coin.cpp \
    -o rl_training/game_env\
$(python3-config --extension-suffix) \
    -lsfml-graphics \
    -lsfml-window \
    -lsfml-system
```

This produced a file similar to:

```
game_env.cpython-312-x86_64-linux-gnu.so
```

This shared object file is the compiled Python extension module.

Python can then simply use:

```
import game_env
```

127 Testing the C++ Environment from Python

Before attempting any reinforcement learning, we first verified that the language bridge worked correctly.

For example:

```
import game_env

env = game_env.GameEnvironment(800, 600)

state = env.reset()

print(state.dx)
print(state.dy)

result = env.step(game_env.Action.Right)

print(result.nextState.dx)
print(result.nextState.dy)
print(result.reward)
print(result.done)
```

This test was extremely important.

Before debugging a DQN, we should separately verify that:

1. The C++ module imports correctly.
2. The environment can be created.
3. `reset()` returns a valid state.
4. `step()` accepts actions.
5. The next state is correctly returned.
6. Rewards are correct.
7. Terminal states are detected.

This isolates environment problems from neural-network problems.

128 The Python DQN Architecture

Our state contained two numbers:

$$s = \begin{bmatrix} dx & dy \end{bmatrix}.$$

Therefore, the input dimension of the neural network was:

2.

Our action space contained four actions, so the output dimension was:

4.

The DQN architecture was:

$$2 \rightarrow 64 \rightarrow 64 \rightarrow 4.$$

A representative implementation in `dqn.py` was:

```
import torch
import torch.nn as nn

class DQN(nn.Module):

    def __init__(self):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(2, 64),
            nn.ReLU(),

            nn.Linear(64, 64),
            nn.ReLU(),

            nn.Linear(64, 4)
```

```

    )

    def forward(self, x):
        return self.network(x)

```

For one state:

$$s = [dx, dy],$$

the network outputs:

$$Q(s, \cdot) = [Q(s, \text{Up}) \quad Q(s, \text{Down}) \quad Q(s, \text{Left}) \quad Q(s, \text{Right})].$$

For example:

$$Q(s, \cdot) = [1.2 \quad 0.7 \quad -0.3 \quad 2.1].$$

The greedy action is:

$$a^* = \arg \max_a Q(s, a).$$

Therefore, in this example, the agent chooses Right.

129 Experience Replay

We created a replay buffer that stored transitions:

$$(s, a, r, s', d).$$

A representative implementation was:

```

import random
from collections import deque

class ReplayBuffer:

    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(
        self,
        state,
        action,
        reward,
        next_state,
        done
    ):
        self.buffer.append(
            (
                state,
                action,
                reward,
                next_state,

```

```

        done
    )
)

def sample(self, batch_size):
    return random.sample(
        self.buffer,
        batch_size
    )

def __len__(self):
    return len(self.buffer)

```

130 A Question About Sampling a Batch

An important question raised during the implementation was:

If we sample 64 transitions from the experience buffer, do we run those 64 states through the online network again? Then we get a 64×4 matrix. But from the target network, for s' , we ultimately get a 64×1 matrix. How can we calculate the loss? Do we select only $Q(s, a)$, which is 64×1 , and train that?

The answer is yes.

Suppose:

$$B = 64.$$

The state batch has shape:

$$[64, 2].$$

Passing it through the online network gives:

$$Q_{\text{online}}(s, \cdot) \in \mathbb{R}^{64 \times 4}.$$

However, each replay-buffer transition contains the action that was actually taken.

Suppose the action tensor is:

$$a \in \mathbb{R}^{64}.$$

We therefore select only:

$$Q_{\text{online}}(s_i, a_i)$$

for each transition i .

In PyTorch:

```

all_q_values = online_network(states)

predicted_q_values = all_q_values.gather(
    1,
    actions.unsqueeze(1)
).squeeze(1)

```

The result has shape:

$$[64].$$

For the next states, the target network produces:

$$Q_{\text{target}}(s', \cdot) \in \mathbb{R}^{64 \times 4}.$$

We select the maximum action value for each next state:

$$\max_{a'} Q_{\text{target}}(s'_i, a').$$

In PyTorch:

```
next_q_values = target_network(next_states)

max_next_q_values = next_q_values.max(
    dim=1
).values
```

This produces:

$$[64].$$

The Bellman targets are:

$$y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\text{target}}(s'_i, a').$$

Thus:

$$\text{Predictions} \in \mathbb{R}^{64},$$

and

$$\text{Targets} \in \mathbb{R}^{64}.$$

The loss is therefore perfectly well-defined:

$$L = \frac{1}{64} \sum_{i=1}^{64} \ell(Q_{\text{online}}(s_i, a_i), y_i).$$

131 The DQN Training Step

Our training function followed this structure:

```
def train_step(
    online_network,
    target_network,
    replay_buffer,
    optimizer,
    batch_size,
    gamma,
    min_replay_size,
    device
):
```

```
if len(replay_buffer) < min_replay_size:
    return None

batch = replay_buffer.sample(batch_size)

states, actions, rewards, \
next_states, dones = zip(*batch)

states = torch.tensor(
    states,
    dtype=torch.float32,
    device=device
)

actions = torch.tensor(
    actions,
    dtype=torch.long,
    device=device
)

rewards = torch.tensor(
    rewards,
    dtype=torch.float32,
    device=device
)

next_states = torch.tensor(
    next_states,
    dtype=torch.float32,
    device=device
)

dones = torch.tensor(
    dones,
    dtype=torch.float32,
    device=device
)

all_q_values = online_network(states)

predicted_q_values = all_q_values.gather(
    1,
    actions.unsqueeze(1)
).squeeze(1)

with torch.no_grad():
    next_q_values = target_network(
        next_states
    )
```



```

    max_next_q_values = next_q_values.max(
        dim=1
    ).values

    target_q_values = (
        rewards
        + gamma
        * (1.0 - dones)
        * max_next_q_values
    )

    loss = torch.nn.functional.smooth_l1_loss(
        predicted_q_values,
        target_q_values
    )

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    return loss.item()

```

132 Why We Used an Online Network and a Target Network

The online network is updated continuously using gradient descent.

If the same rapidly changing network were also used to calculate its own targets, then the target itself would move after every update.

This creates instability.

We therefore maintain:

$$Q_{\theta}$$

as the online network, and

$$Q_{\theta^-}$$

as the target network.

The Bellman target becomes:

$$y = r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a').$$

Periodically, we copy:

$$\theta^- \leftarrow \theta.$$

In code:

```

if (
    total_environment_steps
    % TARGET_UPDATE_FREQUENCY

```

```

    == 0
):
    target_network.load_state_dict(
        online_network.state_dict()
    )

```

133 The First Random-Agent Baseline

Before training the DQN, we tested random actions.

The random agent achieved approximately:

3.6%

success.

This baseline was important.

Without a baseline, seeing a trained agent achieve, for example, 10% success gives little context.

But comparing:

3.6% → 28% → 88%

clearly demonstrates learning.

134 Initial Local Training

During one local laptop run, the agent reached approximately:

36%

cumulative success by episode 80.

However, the laptop began heating significantly.

This led to the practical question:

Can the entire training pipeline be run on Kaggle instead?

The answer was yes.

However, our project was more complex than a normal PyTorch notebook because the environment was written in C++ and used:

- SFML,
- pybind11,
- a compiled shared object file,
- Python training code.

135 Why We Did Not Simply Upload the Local .so File

A compiled file such as:

`game_env.cpython-312-x86_64-linux-gnu.so`

depends on several system-specific factors:

- Python version,
- CPU architecture,
- compiler ABI,
- system libraries,
- installed SFML version.

Therefore, a .so compiled on an Ubuntu laptop may not work on Kaggle.
The robust solution was:

Upload C++ source code → Compile inside Kaggle

136 Preparing the Project for Kaggle

The project uploaded to Kaggle contained:

```
first_game/  
|  
|-- include/  
|   |-- Coin.hpp  
|   |-- GameEnvironment.hpp  
|   '-- Player.hpp  
|  
|-- src/  
|   |-- GameEnvironment.cpp  
|   |-- Coin.cpp  
|   |-- bindings.cpp  
|   '-- Player.cpp  
|  
'-- rl_training/  
    |-- train.py  
    |-- replay_buffer.py  
    '-- dqn.py
```

The actual attached Kaggle path was:

```
/kaggle/input/datasets/vigneshwarsk/  
c-rl-coin-collection-environment/  
first_game (Copy)/
```

137 Understanding Kaggle Input and Working Directories

Kaggle provides two important locations.

The input directory is:

```
/kaggle/input/
```

This contains attached datasets and should effectively be treated as read-only.

The working directory is:

```
/kaggle/working/
```

This is writable.

We therefore copied the project:

```
import shutil
import os

SOURCE = (
    "/kaggle/input/datasets/vigneshwarsk/"
    "c-rl-coin-collection-environment/"
    "first_game (Copy)"
)

PROJECT = "/kaggle/working/first_game"

shutil.copytree(
    SOURCE,
    PROJECT,
    dirs_exist_ok=True
)

print("Project copied successfully!")
print(os.listdir(PROJECT))
```

138 Installing Dependencies on Kaggle

We installed pybind11:

```
!python3 -m pip install -q pybind11
```

Then SFML:

```
!apt-get update -qq
!apt-get install -y libsFML-dev
```

We verified them using:

```
!python3 -m pybind11 --includes
!pkg-config --modversion sfml-graphics
```

139 Compiling the C++ Environment on Kaggle

Initially, the compilation command attempted to use:

```
python3-config --extension-suffix
```

However, Kaggle returned:

```
/bin/bash: line 1:
python3-config: command not found
```

The solution was to obtain the Python extension suffix directly through Python's `sysconfig` module.

For example:

```
import sysconfig

print(
    sysconfig.get_config_var("EXT_SUFFIX")
)
```

The final compilation command became:

```
!c++ -O3 -Wall -shared -std=c++17 -fPIC \
    $(python3 -m pybind11 --includes) \
    -I"/kaggle/working/first_game/include" \
    "/kaggle/working/first_game/src/bindings.cpp" \
    "/kaggle/working/first_game/src/GameEnvironment.cpp" \
    "/kaggle/working/first_game/src/Player.cpp" \
    "/kaggle/working/first_game/src/Coin.cpp" \
    -o "/kaggle/working/first_game/rl_training/\
game_env$(python3 -c \
'import sysconfig; \
print(sysconfig.get_config_var("EXT_SUFFIX"))')" \
    -lsfml-graphics \
    -lsfml-window \
    -lsfml-system
```

140 Testing the Environment on Kaggle

After compiling, we added the training directory to Python's module search path:

```
import sys

TRAINING_DIR = (
    "/kaggle/working/"
    "first_game/rl_training"
)

if TRAINING_DIR not in sys.path:
    sys.path.insert(0, TRAINING_DIR)

import game_env

print("Module imported successfully!")

Then:

env = game_env.GameEnvironment(800, 600)

state = env.reset()

print("Initial state:")
print("dx =", state.dx)
print("dy =", state.dy)
```

Finally:

```
result = env.step(game_env.Action.Right)

print("Next state:")
print("dx =", result.nextState.dx)
print("dy =", result.nextState.dy)

print("Reward:", result.reward)
print("Done:", result.done)
```

Only after all of these tests passed did we proceed to DQN training.

141 Important Training Hyperparameters

The main training configuration was:

```
NUM_EPISODES = 500

BATCH_SIZE = 64
REPLAY_CAPACITY = 100_000
MIN_REPLAY_SIZE = 1000

GAMMA = 0.99
LEARNING_RATE = 1e-3

EPSILON_START = 1.0
EPSILON_END = 0.05
EPSILON_DECAY = 0.995

TARGET_UPDATE_FREQUENCY = 1000

CHECKPOINT_FREQUENCY = 50
PRINT_FREQUENCY = 10

ROLLING_WINDOW_SIZE = 100

EVALUATION_FREQUENCY = 50
NUM_EVALUATION_EPISODES = 100
```

142 Epsilon-Greedy Action Selection

During training, the agent used epsilon-greedy exploration.

With probability:

$$\epsilon,$$

a random action was selected.

Otherwise:

$$a = \arg \max_a Q(s, a).$$

The implementation was:

```
def select_action(
    state,
    online_network,
    epsilon,
    device
):

    if random.random() < epsilon:
        return random.randint(0, 3)

    state_tensor = torch.tensor(
        state,
        dtype=torch.float32,
        device=device
    )

    with torch.no_grad():
        q_values = online_network(
            state_tensor
        )

    return q_values.argmax().item()
```

143 The Main Environment Interaction Loop

For every episode:

1. Reset the C++ environment.
2. Observe the initial state.
3. Select an action.
4. Execute the action in C++.
5. Receive the next state, reward, and terminal flag.
6. Store the transition in replay memory.
7. Sample a batch and train the online network.
8. Periodically update the target network.
9. Repeat until the episode terminates.

The central loop was:

```
while not done:

    action_index = select_action(
        state,
        online_network,
        epsilon,
        device
```

```
)

cpp_action = ACTIONS[action_index]

result = env.step(cpp_action)

next_state = [
    result.nextState.dx,
    result.nextState.dy
]

replay_buffer.push(
    state,
    action_index,
    result.reward,
    next_state,
    result.done
)

loss = train_step(
    online_network,
    target_network,
    replay_buffer,
    optimizer,
    BATCH_SIZE,
    GAMMA,
    MIN_REPLAY_SIZE,
    device
)

state = next_state
done = result.done
```

144 The First 100-Episode Kaggle Experiment

The first Kaggle experiment reached only:

8%

cumulative success at episode 100.

Some output was:

Episode: 10

Overall Success: 0.00%

Episode: 40

Overall Success: 5.00%

Episode: 80

Overall Success: 8.75%

Episode: 100

Overall Success: 8.00%

The recent average reward improved from:

−5.0

to approximately:

−3.914.

This suggested some improvement, but the result was considerably worse than the earlier laptop run.

145 Why Two RL Runs Can Produce Different Results

The first Kaggle run reached only 8% success at episode 100.

A later run reached:

28%

at episode 100.

This large difference illustrates the stochastic nature of reinforcement learning.

Possible sources of randomness include:

- Random neural-network initialization.
- Random player positions.
- Random coin positions.
- Random exploratory actions.
- Random replay-buffer batches.
- The number and timing of early successful experiences.

In sparse-reward environments, an agent that accidentally discovers several successful trajectories early can learn much faster.

146 The Better 100-Episode Run

The second run produced:

Episode: 10

Overall Success: 10.00%

Episode: 40

Overall Success: 12.50%

Episode: 60

Overall Success: 25.00%

Episode: 80

Overall Success: 26.25%

Episode: 100

Overall Success: 28.00%

At episode 100:

```
Reward: 8.330
Steps: 168
Epsilon: 0.6058
Overall Success: 28.00%
Recent Avg Reward: -1.328
```

The reward matched the reward system:

$$10 - 167(0.01) = 8.33.$$

Thus, the agent successfully collected the coin after 168 actions.

147 Why the Loss Sometimes Became Large

Most observed losses were extremely small:

```
0.000001
0.000003
0.000269
```

but occasional larger values appeared:

```
0.124837
0.104472
```

This was not necessarily a problem.

Most transitions had a small step penalty such as:

$$r = -0.01.$$

A successful terminal transition had a much larger positive reward.

Therefore, when a sampled batch contained important transitions associated with successful trajectories, the temporal-difference error could become significantly larger.

Another important observation was that the original code printed only the final batch loss from an episode.

This is not representative of the entire episode.

We therefore improved the code to collect:

```
episode_losses = []
```

and calculate:

```
average_episode_loss = (
    sum(episode_losses)
    / len(episode_losses)
    if episode_losses
    else None
)
```

148 Why Training Success Is Not the Same as Policy Quality

At episode 100:

$$\epsilon \approx 0.6058.$$

This means that approximately 60.58% of actions were still selected randomly.

Therefore, the observed training success rate did not directly measure the quality of the neural network.

We distinguished:

Training Performance $\neq$ Greedy Policy Performance

Training uses epsilon-greedy exploration.

Evaluation uses:

$$\epsilon = 0.$$

Therefore:

$$a = \arg \max_a Q(s, a)$$

for every action during evaluation.

149 Adding Greedy Evaluation

We added an evaluation function:

```
def evaluate_agent(
    env,
    online_network,
    num_episodes,
    device
):
    successes = 0
    total_success_steps = 0
    total_rewards = 0.0

    was_training = online_network.training

    online_network.eval()

    for _ in range(num_episodes):
        state_obj = env.reset()

        state = [
            state_obj.dx,
            state_obj.dy
        ]

        done = False
```

```
episode_steps = 0
episode_reward = 0.0

while not done:

    action_index = select_action(
        state,
        online_network,
        epsilon=0.0,
        device=device
    )

    result = env.step(
        ACTIONS[action_index]
    )

    state = [
        result.nextState.dx,
        result.nextState.dy
    ]

    episode_steps += 1
    episode_reward += result.reward

    done = result.done

    if (
        result.done
        and result.reward > 0
    ):
        successes += 1
        total_success_steps += (
            episode_steps
        )

    total_rewards += episode_reward

if was_training:
    online_network.train()

success_rate = (
    100.0
    * successes
    / num_episodes
)

average_success_steps = (
    total_success_steps / successes
    if successes > 0
    else None
)
```

```
average_reward = (  
    total_rewards / num_episodes  
)  
  
return (  
    success_rate,  
    average_success_steps,  
    average_reward  
)
```

150 Checkpointing

Kaggle's working directory should not be treated as permanent storage.

During experimentation, the working directory was found empty after returning to the notebook later.

Therefore, checkpointing became essential.

The checkpoint stored:

- Episode number.
- Online-network parameters.
- Target-network parameters.
- Optimizer state.
- Current epsilon.
- Total environment steps.
- Total successes.
- Replay buffer.
- Recent success history.
- Recent reward history.

The checkpoint dictionary was:

```
checkpoint = {  
    "episode":  
        episode,  
  
    "online_network":  
        online_network.state_dict(),  
  
    "target_network":  
        target_network.state_dict(),  
  
    "optimizer":  
        optimizer.state_dict(),  
  
    "epsilon":
```

```
        epsilon,

        "total_environment_steps":
            total_environment_steps,

        "total_successes":
            total_successes,

        "replay_buffer":
            list(replay_buffer.buffer),

        "recent_successes":
            list(recent_successes),

        "recent_rewards":
            list(recent_rewards)
    }
```

It was saved with:

```
torch.save(checkpoint, path)
```

151 Checkpoint Versus Final Model

Two different files were produced:

```
dqn_checkpoint.pt
dqn_final_model.pt
```

They serve different purposes.

The checkpoint contains the complete training state:

- Online network.
- Target network.
- Optimizer.
- Epsilon.
- Episode count.
- Replay memory.
- Statistics.

It is intended for:

Resuming Training

The final model contains only:

```
online_network.state_dict()
```

It is intended for:

Inference and Evaluation

Therefore, to watch the trained agent play, only the final model is required.

152 Final Training Results

The agent was trained for 500 episodes.

The progression of greedy evaluation success was approximately:

$$38\% \rightarrow 62\% \rightarrow 33\% \rightarrow 28\% \rightarrow 60\% \rightarrow 65\% \rightarrow 33\% \rightarrow 63\% \rightarrow 39\% \rightarrow 88\%.$$

At episode 500:

Greedy Evaluation Success = 88%

The recent training success over the latest 100 episodes reached approximately:

$$93\%.$$

The overall training success across all 500 episodes reached approximately:

$$71\%.$$

This included the earliest episodes, when the agent had little or no useful knowledge.
The random baseline was only:

$$3.6\%.$$

Therefore, the complete progression was approximately:

3.6% random baseline $\rightarrow$ 88% trained greedy policy
--

153 Why Evaluation Performance Was Non-Monotonic

The evaluation success rate did not continuously increase.

For example:

$$65\% \rightarrow 33\% \rightarrow 63\% \rightarrow 39\% \rightarrow 88\%.$$

This does not necessarily indicate a bug.

DQN training can be unstable because:

- The online network changes continuously.
- Q-value estimates bootstrap from other Q-value estimates.
- Replay batches are random.
- Small changes in Q-values can change the greedy action.

For example:

$$Q(\text{Right}) = 3.152,$$

$$Q(\text{Down}) = 3.151.$$

The agent chooses Right.

After a small update:

$$Q(\text{Right}) = 3.149,$$

$$Q(\text{Down}) = 3.153.$$

Now the agent chooses Down.

Although the numerical Q-values changed only slightly, the selected action changed completely.

This is one reason why a future improvement should be to save:

`best_model.pt`

whenever greedy evaluation reaches a new maximum, instead of saving only the final model.

154 Loading the Trained Model

The downloaded model was:

`dqn_final_model.pt`

It could be loaded using:

```
import torch

from dqn import DQN

device = torch.device("cpu")

model = DQN().to(device)

model.load_state_dict(
    torch.load(
        "dqn_final_model.pt",
        map_location=device,
        weights_only=True
    )
)

model.eval()
```

The neural network can then evaluate any state:

```
state = [0.4, -0.2]

state_tensor = torch.tensor(
    state,
    dtype=torch.float32,
    device=device
)

with torch.no_grad():
    q_values = model(state_tensor)

action_index = q_values.argmax().item()
```


155 Watching the Trained Agent Play

The next objective was to visually observe the trained agent.

The architecture became:

SFML Rendering $\leftarrow$ C++ GameEnvironment $\leftarrow$ pybind11 $\leftarrow$ Python $\leftarrow$ Trained PyTorch DQN

At every frame:

1. The C++ environment provides the current state.
2. Python converts it to a tensor.
3. The DQN predicts four Q-values.
4. Python selects the action with maximum Q-value.
5. The action is sent to the C++ environment.
6. The C++ environment moves the player.
7. SFML renders the updated game.
8. The process repeats.

Symbolically:

Player and Coin Positions

↓

$s = [dx, dy]$

↓

DQN

↓

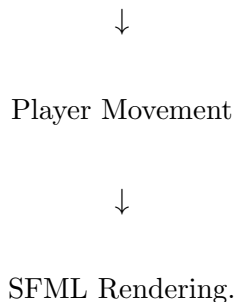
$[Q_{\text{Up}}, Q_{\text{Down}}, Q_{\text{Left}}, Q_{\text{Right}}]$

↓

$\arg \max$

↓

C++ Action



156 Adding Optional Rendering to GameEnvironment

The environment needed to remain headless during training.

Creating an SFML window during thousands of training episodes would be unnecessary and inefficient.

Therefore, rendering was made optional.

The environment stored:

```
std::unique_ptr<sf::RenderWindow> window;
```

Initially:

```
window == nullptr
```

Therefore:

Training → No Window.

Only when visualization was explicitly enabled:

```
env.enable_rendering()
```

was an SFML window created.

157 Rendering Methods

The relevant methods were:

```
void enableRendering();
```

```
void renderWindow();
```

```
bool isWindowOpen() const;
```

```
void handleWindowEvents();
```

A representative implementation of window creation was:

```

void GameEnvironment::enableRendering()
{
    if (!window)
    {
        window =
            std::make_unique<sf::RenderWindow>(
                sf::VideoMode(
                    windowWidth,
                    windowHeight
                ),
                "DQN Agent Playing"
            );

        window->setFramerateLimit(60);
    }
}

```

The window status function was:

```

bool GameEnvironment::isWindowOpen() const
{
    return window && window->isOpen();
}

```

Event handling was:

```

void GameEnvironment::handleWindowEvents()
{
    if (!window)
    {
        return;
    }

    sf::Event event;

    while (window->pollEvent(event))
    {
        if (event.type == sf::Event::Closed)
        {
            window->close();
        }
    }
}

```

158 A Rendering Design Problem

Initially, two functions with the same name existed conceptually:

```

void render(sf::RenderWindow& window);

void render();

```

The binding attempted:

```
.def(
    "render",
    &GameEnvironment::render
)
```

However, C++ could not determine which overloaded function was intended. The compiler produced an error indicating:

```
unresolved overloaded function type
```

The immediate technical solution could have been:

```
static_cast<void (GameEnvironment::*)()>(
    &GameEnvironment::render
)
```

However, a cleaner architectural decision was made. The new rendering function was renamed:

```
void renderWindow();
```

The pybind11 binding could still expose it to Python under the simple name:

```
.def(
    "render",
    &GameEnvironment::renderWindow
)
```

Therefore:

```
env.render()
```

in Python calls:

```
GameEnvironment::renderWindow()
```

in C++.

159 The Undefined Symbol Error

After resolving the overloaded binding, another runtime error appeared:

```
ImportError:
game_env.cpython-312-x86_64-linux-gnu.so:
undefined symbol:
_ZN15GameEnvironment6renderERN2sf12RenderWindowE
```

This corresponded to the missing C++ function:

```
GameEnvironment::render(
    sf::RenderWindow&
)
```

The reason was that `renderWindow()` was calling:

```
render(*window);
```

but the matching function had been declared without a linked implementation.
The final design decision was to remove the unnecessary overloaded:

```
render(sf::RenderWindow&)
```

entirely.

Instead, the environment kept only:

```
void enableRendering();
void renderWindow();
bool isWindowOpen() const;
void handleWindowEvents();
```

The player and coin could each be given their own drawing method:

```
void draw(sf::RenderWindow& window);
```

Then:

```
void GameEnvironment::renderWindow()
{
    if (!window)
    {
        return;
    }

    window->clear();

    player.draw(*window);
    coin.draw(*window);

    window->display();
}
```

This produces a clean responsibility structure:

GameEnvironment → controls when rendering occurs
--

while:

Player and Coin → know how to draw themselves

160 Exposing Rendering Functions Through pybind11

The final relevant bindings were:

```
py::class_<GameEnvironment>(
    m,
    "GameEnvironment"
)
    .def(py::init<int, int>())
```

```

    .def(
        "reset",
        &GameEnvironment::reset
    )
    .def(
        "getState",
        &GameEnvironment::getState
    )
    .def(
        "step",
        &GameEnvironment::step
    )
    .def(
        "enable_rendering",
        &GameEnvironment::enableRendering
    )
    .def(
        "render",
        &GameEnvironment::renderWindow
    )
    .def(
        "is_window_open",
        &GameEnvironment::isWindowOpen
    )
    .def(
        "handle_window_events",
        &GameEnvironment::handleWindowEvents
    );

```

161 Recompiling the Updated Environment

Before recompiling, the old shared object could be removed:

```
rm -f rl_training/game_env*.so
```

Then, from the project root:

```

c++ -O3 -Wall -shared -std=c++17 -fPIC \
    $(python3 -m pybind11 --includes) \
    -Iinclude \
    src/bindings.cpp \
    src/GameEnvironment.cpp \
    src/Player.cpp \
    src/Coin.cpp \
    -o rl_training/game_env\
$(python3-config --extension-suffix) \
    -lsfml-graphics \
    -lsfml-window \
    -lsfml-system

```

162 The watch_agent.py Script

The final model was placed inside:

```
rl_training/
```

The directory contained:

```
rl_training/  
|  
|-- dqn.py  
|-- replay_buffer.py  
|-- train.py  
|-- watch_agent.py  
|-- dqn_final_model.pt  
'-- game_env.cpython-312-x86_64-linux-gnu.so
```

The visualization script was:

```
import time  
  
import torch  
  
import game_env  
  
from dqn import DQN  
  
MODEL_PATH = "dqn_final_model.pt"  
  
WINDOW_WIDTH = 800  
WINDOW_HEIGHT = 600  
  
STEP_DELAY = 0.03  
  
ACTIONS = [  
    game_env.Action.Up,  
    game_env.Action.Down,  
    game_env.Action.Left,  
    game_env.Action.Right  
]  
  
device = torch.device("cpu")  
  
model = DQN().to(device)  
  
model.load_state_dict(  
    torch.load(  
        MODEL_PATH,  
        map_location=device,
```

```
        weights_only=True
    )
)

model.eval()

print(
    "Trained DQN loaded successfully!"
)

env = game_env.GameEnvironment(
    WINDOW_WIDTH,
    WINDOW_HEIGHT
)

env.enable_rendering()

episode = 1

while env.is_window_open():

    state_obj = env.reset()

    state = [
        state_obj.dx,
        state_obj.dy
    ]

    done = False

    total_reward = 0.0
    steps = 0

    print(
        f"\nStarting episode {episode}"
    )

    while (
        not done
        and env.is_window_open()
    ):

        env.handle_window_events()

        if not env.is_window_open():
            break

        state_tensor = torch.tensor(
            state,
```



```
        dtype=torch.float32,
        device=device
    )

    with torch.no_grad():
        q_values = model(state_tensor)

    action_index = (
        q_values.argmax().item()
    )

    result = env.step(
        ACTIONS[action_index]
    )

    state = [
        result.nextState.dx,
        result.nextState.dy
    ]

    total_reward += result.reward
    steps += 1

    env.render()

    time.sleep(STEP_DELAY)

    done = result.done

    if not env.is_window_open():
        break

    success = total_reward > 0

    print(
        f"Episode {episode} finished | "
        f"Success: {success} | "
        f"Steps: {steps} | "
        f"Reward: {total_reward:.3f}"
    )

    time.sleep(0.5)

    episode += 1
```

163 Running the Trained Agent

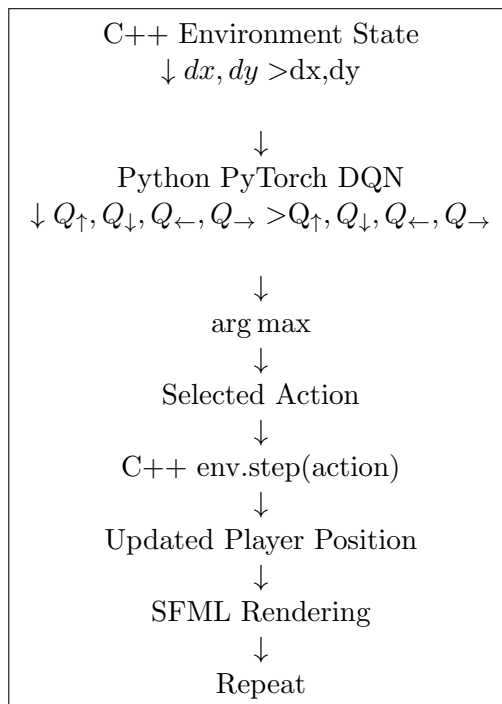
From the project root:

```
cd rl_training
```

Then:

`python3 watch_agent.py`

The expected execution flow is:



164 Complete System Architecture

The complete project now consists of four conceptual layers.

164.1 Layer 1: C++ Game Objects

- Player
- Coin
- Position and movement
- Collision detection
- SFML shapes

164.2 Layer 2: C++ RL Environment

- `reset()`
- `step(action)`
- State generation
- Reward calculation
- Episode termination
- Optional rendering

164.3 Layer 3: pybind11 Interface

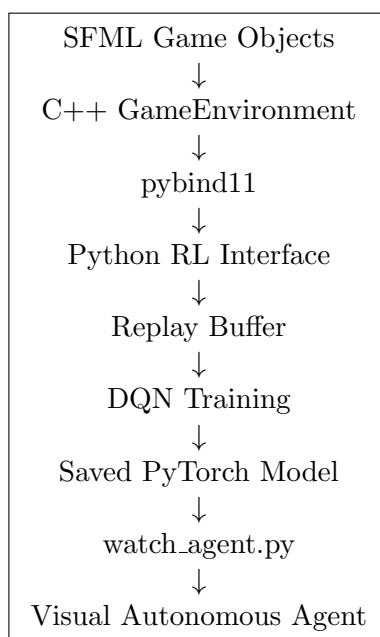
This exposes the C++ environment to Python:

C++ $\longleftrightarrow$ Python.

164.4 Layer 4: Python DQN Agent

- Online network
- Target network
- Replay buffer
- Epsilon-greedy exploration
- Bellman targets
- Huber loss
- Backpropagation
- Evaluation
- Checkpointing

The complete architecture is:



165 Key Lessons from the First Complete RL Game Project

This first project demonstrated several important principles.

165.1 The Environment and Agent Can Use Different Languages

The environment was written in C++.

The DQN was written in Python using PyTorch.

pybind11 connected them.

This architecture can scale to significantly more complex games.

165.2 Rendering Should Be Separate from Training

During training:

Rendering Off.

During visualization:

Rendering On.

This avoids wasting computational resources during thousands of training steps.

165.3 Always Establish a Random Baseline

The random agent achieved approximately:

3.6%.

The trained greedy policy eventually achieved approximately:

88%.

Therefore, the improvement was measurable and meaningful.

165.4 Training Success and Evaluation Success Are Different

Training uses exploration.

Evaluation uses the greedy learned policy.

Therefore, both should be measured separately.

165.5 RL Training Is Stochastic

One 100-episode run achieved only:

8%.

Another reached:

28%.

Early random experiences can significantly affect learning speed.

165.6 Loss Alone Does Not Measure Agent Performance

A low loss does not necessarily mean the policy is good.

A temporary high loss does not necessarily mean training has failed.

The most meaningful measurements include:

- Success rate.
- Average reward.
- Greedy evaluation success.
- Average successful episode length.

165.7 Save the Best Model, Not Only the Final Model

The greedy evaluation score fluctuated significantly.

Therefore, a future implementation should save:

```
best_model.pt
```

whenever:

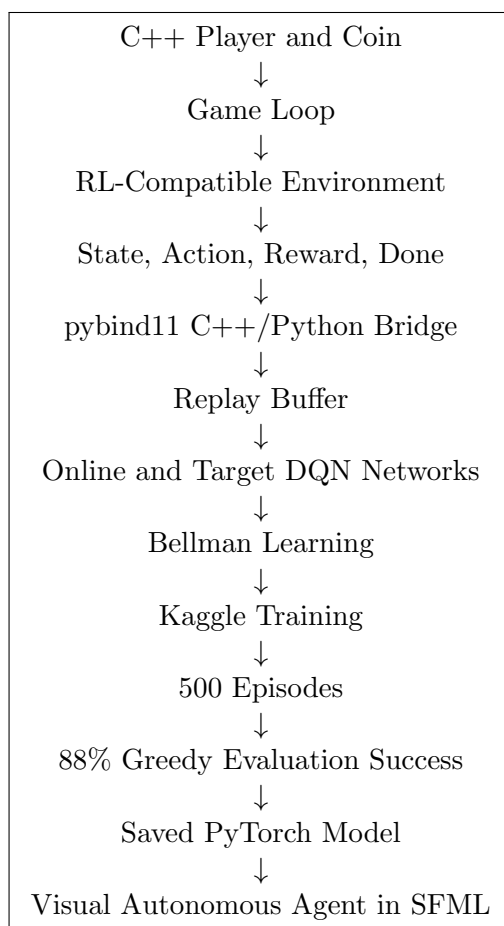
```
current evaluation score > best previous evaluation score.
```

This guarantees that temporary deterioration later in training does not destroy access to the strongest policy found.

166 Conclusion

The project began as a simple C++ game containing only a player and a coin.

It evolved through several stages:



The most important achievement is not simply that the agent learned to collect a coin.

The complete pipeline now exists:

Custom C++ Game + Custom RL Environment + Python DQN + Cloud Training + Model Deployment Back

This architecture forms the foundation for progressively more complex reinforcement learning environments involving:

- Multiple coins.
- Obstacles.
- Enemies.
- Health and damage.
- Larger state spaces.
- Continuous observations.
- Image-based observations.
- Convolutional neural networks.
- More advanced reinforcement learning algorithms.

The simple coin-collection game therefore serves as the first complete end-to-end experiment in combining C++ game development with deep reinforcement learning.

=====

A Detailed C++ Object-Oriented Programming and Language Concepts

This appendix records a series of questions that naturally arose while refactoring the SFML coin-collector game into classes. Rather than merely presenting the final code, the goal is to understand exactly what C++ is doing underneath: how classes create objects, how methods operate on particular objects, how header files work, what the compiler and linker do, what **const** means in different positions, how random-number generators work, and why constructors use member initializer lists.

A.1 What Are Classes, Objects, and Methods?

Question

How are classes, methods, and object methods created in C++?

A **class** is a user-defined type that acts as a blueprint for creating objects. It describes:

- what data each object stores, and
- what operations each object can perform.

For example, suppose every player in a game should have a position, speed, and the ability to move:

```
1 class Player
2 {
3 public:
4     float x;
5     float y;
6     float speed;
7
8     void moveRight()
9     {
```

```
10     x += speed;
11 }
12 };
```

Here, `Player` is a class. At this point, it is only a type definition; no actual player object has necessarily been created yet.

An object is created from the class as follows:

```
1 Player player1;
```

Here:

- `Player` is the class or type,
- `player1` is an object, also called an instance of the class.

The members of an object are accessed using the dot operator:

```
1 player1.x = 100.0f;
2 player1.y = 200.0f;
3 player1.speed = 5.0f;
4
5 player1.moveRight();
```

After calling `moveRight()`, the value of `player1.x` becomes:

$$100 + 5 = 105$$

The general syntax is:

```
1 object.memberVariable
2 object.memberFunction()
```

A function belonging to a class is called a **member function**, and is commonly referred to as a **method**.

A.2 How Does a Method Know Which Object Called It?

Consider:

```
1 class Player
2 {
3 public:
4     float x = 0.0f;
5     float speed = 5.0f;
6
7     void moveRight()
8     {
9         x += speed;
10    }
11 };
```

Now create two objects:

```
1 Player player1;
2 Player player2;
3
4 player1.x = 10.0f;
5 player2.x = 100.0f;
6
7 player1.moveRight();
```

Inside:

```
1 void moveRight()
2 {
3     x += speed;
4 }
```

how does C++ know that `x` refers to `player1.x` rather than `player2.x`?

Every non-static member function has access to a special hidden pointer called:

```
1 this
```

The `this` pointer points to the object on which the method was called.

Therefore:

```
1 player1.moveRight();
```

means that, inside `moveRight()`, `this` points to `player1`.

Conceptually:

```
1 void Player::moveRight()
2 {
3     this->x += this->speed;
4 }
```

Writing:

```
1 x += speed;
```

is therefore equivalent, in this context, to:

```
1 this->x += this->speed;
```

If:

```
1 player1.moveRight();
```

is called, then conceptually:

```
1 this == &player1
```

If:

```
1 player2.moveRight();
```

is called, then conceptually:

```
1 this == &player2
```

This is how the same member-function definition can operate on many different objects.

A.3 Defining Methods Outside a Class

A method can be declared inside the class and defined separately:

```
1 class Player
2 {
3 public:
4     float x;
5     float speed;
6
7     void moveRight();
8 };
```


The implementation can then be written as:

```
1 void Player::moveRight()
2 {
3     x += speed;
4 }
```

The operator:

```
1 ::
```

is called the **scope resolution operator**.

Thus:

```
1 Player::moveRight
```

means:

The function `moveRight` belonging to the scope of the `Player` class.

This distinction is necessary because different classes can have methods with the same name:

```
1 Player::move()
2 Enemy::move()
3 Coin::move()
```

A.4 Constructors

A constructor is a special member function that is automatically called when an object is created.

For example:

```
1 class Player
2 {
3 public:
4     float x;
5     float y;
6     float speed;
7
8     Player()
9     {
10         x = 100.0f;
11         y = 100.0f;
12         speed = 5.0f;
13     }
14 };
```

Now:

```
1 Player player;
```

automatically calls:

```
1 Player()
```

The object begins with:

```
x      = 100
y      = 100
speed = 5
```

A constructor:

- has the same name as its class,
- has no return type, not even `void`,
- is automatically invoked during object construction.

Constructors can also accept arguments:

```
1 class Player
2 {
3 public:
4     float x;
5     float y;
6
7     Player(float startX, float startY)
8     {
9         x = startX;
10        y = startY;
11    }
12};
```

Objects can then be constructed as:

```
1 Player player1(100.0f, 200.0f);
2 Player player2(500.0f, 300.0f);
```

A.5 Public and Private Members

Consider:

```
1 class Player
2 {
3 private:
4     float x;
5     float y;
6
7 public:
8     void moveRight()
9     {
10        x += 5.0f;
11    }
12};
```

The method can be called externally:

```
1 Player player;
2 player.moveRight();
```

But this is not allowed:

```
1 player.x = 100.0f; // Error: x is private
```

The basic distinction is:

- **private**: accessible by the class's own member functions and friends,
- **public**: accessible from outside the class.

Keeping internal implementation details private is part of **encapsulation**.

For example:

```

1 class Player
2 {
3 private:
4     sf::RectangleShape shape;
5     float speed;
6
7 public:
8     void move(float dx, float dy);
9     void update();
10    void draw(sf::RenderWindow& window);
11 };

```

External code uses the public interface without directly manipulating every internal detail of the player.

A.6 Why Do Header Files Exist?

Question

What are header files? How are they created, what is written in them, what is their purpose, and what does the compiler actually do with them?

A small program may place everything in one source file:

```

1 #include <SFML/Graphics.hpp>
2
3 class Player
4 {
5 private:
6     sf::CircleShape shape;
7     float speed;
8
9 public:
10    Player(float x, float y);
11    void move(float dx, float dy);
12    void draw(sf::RenderWindow& window);
13 };
14
15 Player::Player(float x, float y)
16 {
17     shape.setRadius(20.0f);
18     shape.setPosition({x, y});
19     speed = 5.0f;
20 }
21
22 void Player::move(float dx, float dy)
23 {
24     shape.move({dx * speed, dy * speed});
25 }
26
27 void Player::draw(sf::RenderWindow& window)
28 {
29     window.draw(shape);
30 }
31

```

```

32 int main()
33 {
34     // Game code
35 }

```

This works. However, as the game grows, it may contain:

```

Player
Coin
Enemy
Obstacle
RLAgent
Environment
Game
NeuralNetwork

```

Keeping all of this in one file quickly becomes difficult to maintain.

A cleaner project structure is:

```

project/
|
|-- main.cpp
|-- Player.hpp
|-- Player.cpp
|-- Coin.hpp
|-- Coin.cpp
|-- Game.hpp
'-- Game.cpp

```

The fundamental idea is:

`.hpp` file: declares **what exists**.
`.cpp` file: usually defines **how it works**.

A.7 Declaration Versus Definition

A declaration tells the compiler that something exists and describes its interface.

For example:

```

1 void move(float dx, float dy);

```

This tells the compiler:

- a function called `move` exists,
- it takes two `float` arguments,
- it returns `void`.

However, it does not provide the function body.

The definition does:

```

1 void Player::move(float dx, float dy)
2 {
3     shape.move({dx * speed, dy * speed});
4 }

```

Conceptually:

Declaration:

"I promise that this function exists."

Definition:

"Here is its actual implementation."

A.8 Creating a Header File

A possible `Player.hpp` is:

```
1 #ifndef PLAYER_HPP
2 #define PLAYER_HPP
3
4 #include <SFML/Graphics.hpp>
5
6 class Player
7 {
8 private:
9     sf::CircleShape shape;
10    float speed;
11
12 public:
13    Player(float x, float y);
14
15    void move(float dx, float dy);
16    void draw(sf::RenderWindow& window);
17 };
18
19 #endif
```

The header contains the complete class declaration, including:

- member variables,
- access specifiers such as `private` and `public`,
- constructor declarations,
- method declarations.

The implementation can be placed in `Player.cpp`:

```
1 #include "Player.hpp"
2
3 Player::Player(float x, float y)
4 {
5     shape.setRadius(20.0f);
6     shape.setPosition({x, y});
7     speed = 5.0f;
8 }
9
10 void Player::move(float dx, float dy)
11 {
12     shape.move({dx * speed, dy * speed});
13 }
14
15 void Player::draw(sf::RenderWindow& window)
```

```
16 {
17     window.draw(shape);
18 }
```

Then `main.cpp` can use the class:

```
1 #include "Player.hpp"
2
3 int main()
4 {
5     Player player(100.0f, 100.0f);
6
7     player.move(1.0f, 0.0f);
8
9     return 0;
10 }
```

The source file using `Player` does not need to see the body of `move()` merely to compile a valid call. It needs the declaration so that the compiler knows the method's name, parameter types, and return type.

A.9 What Does `#include` Actually Do?

A common misconception is that:

```
1 #include "Player.hpp"
```

works exactly like importing a Python module.

At the preprocessing stage, it is better to think of `#include` as inserting the contents of the included file at that location.

Suppose `main.cpp` contains:

```
1 #include "Player.hpp"
2
3 int main()
4 {
5     Player player(100.0f, 100.0f);
6 }
```

and `Player.hpp` contains:

```
1 class Player
2 {
3 public:
4     Player(float x, float y);
5 };
```

Conceptually, after preprocessing, the compiler sees something equivalent to:

```
1 class Player
2 {
3 public:
4     Player(float x, float y);
5 };
6
7 int main()
8 {
9     Player player(100.0f, 100.0f);
10 }
```

Real preprocessing also handles macros, conditional compilation, and nested includes, but textual inclusion is the essential mental model.

A.10 What Happens When a Multi-File C++ Project Is Compiled?

Suppose the project contains:

```
main.cpp
Player.cpp
Player.hpp
```

and is compiled using:

```
g++ main.cpp Player.cpp -o game
```

The broad pipeline is:

```
Source code
  |
  v
Preprocessor
  |
  v
Compiler
  |
  v
Assembler
  |
  v
Object files
  |
  v
Linker
  |
  v
Executable
```

A.10.1 Preprocessing

The preprocessor handles directives such as:

```
1 #include "Player.hpp"
```

Each source file is processed independently.

Conceptually:

```
main.cpp + included headers
  |
  v
Preprocessed main translation unit

Player.cpp + included headers
  |
  v
Preprocessed Player translation unit
```

A.10.2 Compilation and Assembly

Each source file is translated independently into an object file:

```
main.cpp
  |
  v
main.o

Player.cpp
  |
  v
Player.o
```

At this stage, `main.o` may effectively contain a requirement such as:

```
I use:
    Player::move(float, float)
```

but its implementation is elsewhere.

Meanwhile, `Player.o` provides the compiled definition.

A.10.3 Linking

The linker combines the object files and required libraries:

```
main.o
  +
Player.o
  +
SFML libraries
  |
  v
Final executable
```

The linker matches references to definitions.

If only:

```
g++ main.cpp -o game
```

is compiled while the implementation exists in `Player.cpp`, the compiler may successfully understand a call from the declarations in the header, but the linker can fail with an error similar to:

```
undefined reference to 'Player::move(float, float)'
```

The reason is that the required implementation was never supplied to the linker.

A.11 Why Is the Complete Class Declaration Usually in the Header?

Consider:

```

1 class Player
2 {
3 private:
4     float x;
5     float y;
6     float speed;
7
8 public:
9     void move();
10 };

```

Even though external code cannot directly access the private members, the compiler must know the complete object layout when creating an object by value:

```

1 Player player;

```

Ignoring alignment and padding for simplicity:

```

float x      -> 4 bytes
float y      -> 4 bytes
float speed  -> 4 bytes
-----
Player       -> approximately 12 bytes

```

Therefore, the compiler needs enough information to determine the object's size and layout.

A.12 Header Guards

Consider:

```

1 #ifndef PLAYER_HPP
2 #define PLAYER_HPP
3
4 class Player
5 {
6     // ...
7 };
8
9 #endif

```

These directives form a **header guard**. Their purpose is to prevent the same header contents from being processed more than once within one translation unit.

Imagine:

```

1 #include "Player.hpp"
2 #include "Game.hpp"

```

and suppose `Game.hpp` also contains:

```

1 #include "Player.hpp"

```

Without protection, the same class definition could appear twice after preprocessing.

The guard works conceptually as follows.

First inclusion:

Is `PLAYER_HPP` already defined?
No.

Define `PLAYER_HPP`.
Process the contents of the header.

Later inclusion:

Is `PLAYER_HPP` already defined?
Yes.

Skip everything until `#endif`.

A commonly supported alternative is:

```
1 #pragma once
```

Then the header can simply be:

```
1 #pragma once
2
3 class Player
4 {
5     // ...
6 };
```

A.13 Why Not Include a `.cpp` File?

Normally, one should include header files:

```
1 #include "Player.hpp"
```

rather than implementation files:

```
1 // Usually incorrect project organization:
2 #include "Player.cpp"
```

Source files are intended to be compiled independently:

```
main.cpp -----> main.o
                        \
                        ----> Linker ----> Game
                        /
Player.cpp -----> Player.o
```

If `main.cpp` includes `Player.cpp`, and the build command also compiles `Player.cpp` separately, the same function definitions may appear in multiple object files, causing linker errors for multiple definitions.

The usual rule is:

Include <code>.hpp</code> files. Compile <code>.cpp</code> files. Link the resulting object files.

A.14 Understanding `sf::FloatRect getBounds() const`

Consider the following declaration from the `Player` class:

```
1 sf::FloatRect getBounds() const;
```

Question

What is `sf::FloatRect getBounds() const`, and why is `const` written at the end? It is not a constant literal.

There are two separate concepts:

1. `sf::FloatRect` is the return type.
2. The final `const` qualifies the member function.

A.15 What Is `sf::FloatRect`?

Just as C++ has types such as:

```
1 int
2 float
3 bool
4 std::string
```

SFML provides its own types.

An `sf::FloatRect` represents a rectangular region using floating-point values. Conceptually, a rectangle contains information describing its position and size.

A simplified conceptual representation could be imagined as:

```
1 struct FloatRect
2 {
3     float x;
4     float y;
5     float width;
6     float height;
7 };
```

The exact member API depends on the SFML version, but the essential idea is that the object represents a floating-point rectangle.

Therefore:

```
1 sf::FloatRect getBounds() const;
```

means:

There is a method called `getBounds()` that takes no explicit arguments and returns an object of type `sf::FloatRect`.

For the player:

```
1 sf::FloatRect Player::getBounds() const
2 {
3     return shape.getGlobalBounds();
4 }
```

If the player's bounding rectangle is approximately:

```
x      = 100
y      = 200
width  = 50
height = 60
```

then `getBounds()` returns an `sf::FloatRect` describing that region.
This is useful for collision detection.

A.16 Why Is `const` Written at the End of a Method?

Consider:

```
1 sf::Vector2f getPosition() const;
```

The final `const` means:

This member function promises not to modify the ordinary state of the object on which it is called.

For example:

```
1 sf::Vector2f Player::getPosition() const
2 {
3     return shape.getPosition();
4 }
```

This is valid because the method only retrieves information.

However:

```
1 sf::Vector2f Player::getPosition() const
2 {
3     speed = 100.0f; // Error
4     return shape.getPosition();
5 }
```

is generally invalid because the function was declared as a `const` member function but attempts to modify the object's member variable.

A.17 Why Is `handleInput()` Not `Const`?

Consider:

```
1 void handleInput(float dt);
```

Its implementation may do something like:

```
1 void Player::handleInput(float dt)
2 {
3     if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::W))
4     {
5         shape.move({0.0f, -speed * dt});
6     }
7 }
```

This changes the player's `shape`, and therefore changes the state of the `Player` object.
Thus it should not be declared:

```
1 void handleInput(float dt) const;
```

Similarly, `keepInsideWindow()` may change the player's position to keep it within the window.

By contrast:

```
1 sf::FloatRect getBounds() const;
2 sf::Vector2f getPosition() const;
```

only retrieve information, so they are natural candidates for `const` member functions.

A.18 Two Different Uses of `const`

Compare:

```
1 void keepInsideWindow(const sf::RenderWindow& window);
```

with:

```
1 sf::FloatRect getBounds() const;
```

In the first case:

```
1 const sf::RenderWindow& window
```

means that the function is not allowed to modify the window through that reference.

In the second case:

```
1 getBounds() const
```

means that the method is not allowed to modify the ordinary state of the `Player` object on which it was called.

A member function can contain both forms:

```
1 void someFunction(const sf::RenderWindow& window) const;
```

Here:

- the first `const` protects the window through the reference,
- the final `const` applies to the current object.

A.19 The Hidden `this` Pointer and Const Methods

Recall that a non-static member function has access to a hidden pointer:

```
1 this
```

For a normal member function, the conceptual idea is similar to:

```
1 Player* const this;
```

The pointer itself cannot be redirected, but the pointed-to `Player` object can normally be modified.

For a `const` member function:

```
1 sf::Vector2f getPosition() const;
```

the conceptual model is closer to:

```
1 const Player* const this;
```

Thus the object is treated as `const` through `this`.

This explains why:

```

1 sf::Vector2f Player::getPosition() const
2 {
3     speed = 100.0f; // Error
4     return shape.getPosition();
5 }

```

is rejected.

A.20 Why Const Member Functions Matter

A const object can call only member functions that are valid for const access.

For example:

```

1 const Player player;

```

Suppose the class has:

```

1 sf::Vector2f getPosition() const;
2 void handleInput(float dt);

```

Then:

```

1 player.getPosition();

```

is allowed, but:

```

1 player.handleInput(0.1f);

```

is not, because `handleInput()` may modify the player.

This also matters when passing an object by const reference:

```

1 void printPosition(const Player& player)
2 {
3     auto position = player.getPosition();
4 }

```

Because `player` is a const reference, only const-compatible member functions can be called through it.

A.21 const Does Not Only Mean a Constant Literal

The keyword `const` is much broader than:

```

1 const float PI = 3.14159f;

```

Its meaning depends on its position.

For example:

```

1 const int x = 10;

```

means `x` cannot be modified.

```

1 const Player& player

```

means the `Player` cannot be modified through this reference.

```

1 const Player* ptr;

```

means the pointed-to `Player` cannot be modified through this pointer.

```

1 Player* const ptr = &player;

```

means the pointer itself cannot later point somewhere else.

Finally:

```
1 sf::FloatRect getBounds() const;
```

means that the member function does not modify the ordinary state of the object on which it is called.

A useful mental model is:

`const` expresses a restriction on what may be modified through a particular variable, pointer, reference, or member function.

A.22 Understanding `std::mt19937`

Question

What is `std::mt19937 generator`? What is this random generator, and how is it used?

The type comes from:

```
1 #include <random>
```

When we write:

```
1 std::mt19937 generator;
```

we create an object called `generator` whose type is `std::mt19937`.

The name refers to the Mersenne Twister pseudo-random number generator with a period related to $2^{19937} - 1$.

The important idea is:

`std::mt19937` is a pseudo-random number generator engine that produces a deterministic sequence of integer values from an internal state.

The engine can be called like a function:

```
1 #include <iostream>
2 #include <random>
3
4 int main()
5 {
6     std::mt19937 generator;
7
8     std::cout << generator() << '\n';
9     std::cout << generator() << '\n';
10    std::cout << generator() << '\n';
11 }
```

It may produce raw values such as:

```
3499211612
581869302
3890346734
```

The syntax:

```
1 generator()
```

works because `std::mt19937` is a class whose function-call operator is overloaded.
Conceptually:

```
1 class RandomGenerator
2 {
3 public:
4     unsigned int operator() ()
5     {
6         // Calculate and return next pseudo-random value
7     }
8 };
```

Thus:

```
1 generator();
```

conceptually invokes:

```
1 generator.operator() ();
```

A.23 Why Is It Called Pseudo-Random?

A pseudo-random generator starts from an initial state determined by a **seed**.

Conceptually:

Seed

|

v

Number 1

|

v

Number 2

|

v

Number 3

|

v

...

The crucial property is:

The same seed produces the same pseudo-random sequence.

For example:

```
1 std::mt19937 gen1(42);
2 std::mt19937 gen2(42);
3
4 std::cout << gen1() << '\n';
5 std::cout << gen2() << '\n';
```

Both engines produce the same first output because they began with the same seed.
This reproducibility is extremely useful for:

- debugging,
- testing,
- scientific experiments,
- reinforcement-learning comparisons.

A.24 Default Seed Versus Random Seed

If we write:

```
1 std::mt19937 generator;
```

without explicitly providing a seed, the engine uses its defined default seed. Therefore, restarting the program produces the same pseudo-random sequence.

To obtain a varying seed, one common approach is:

```
1 std::random_device rd;
2 std::mt19937 generator(rd());
```

This can also be written compactly as:

```
1 std::mt19937 generator{std::random_device{}()};
```

Conceptually:

```
random_device
  |
  v
seed
  |
  v
mt19937
  |
  v
pseudo-random sequence
```

For reinforcement-learning experiments, a fixed seed may actually be preferable:

```
1 std::mt19937 generator(42);
```

because repeated runs can then use identical environment randomness.

A.25 Why Not Use the Raw Generator Output Directly?

Suppose the game window has dimensions:

$$800 \times 600$$

Calling:

```
1 generator()
```

may produce:

```
3499211612
```

This is not directly useful as an x -coordinate.

Instead, we use a **distribution**:

```
1 std::uniform_real_distribution<float> xDist(0.0f, 800.0f);
2 std::uniform_real_distribution<float> yDist(0.0f, 600.0f);
```

Then:

```
1 float x = xDist(generator);
2 float y = yDist(generator);
```

might produce:

```
x = 427.35
y = 182.71
```

The engine and distribution have different responsibilities:

Engine	Generates pseudo-random bits or raw integer outputs.
Distribution	Maps randomness into a desired statistical distribution.

Conceptually:

```
mt19937 engine
  |
  v
raw pseudo-random values
  |
  v
uniform_real_distribution<float>(0, 800)
  |
  v
floating-point values in desired range
```

A.26 Integer and Floating-Point Distributions

For random integers:

```
1 std::uniform_int_distribution<int> dist(1, 6);
2
3 int diceRoll = dist(generator);
```

The possible results are:

1, 2, 3, 4, 5, or 6

For floating-point values:

```
1 std::uniform_real_distribution<float> dist(0.0f, 1.0f);
2
3 float value = dist(generator);
```

Possible values include:

```
0.184
0.729
0.035
0.891
```

For continuous SFML coordinates, a real-valued distribution is natural.

A.27 Using Random Generation in the Coin Class

A possible class is:

```

1 class Coin
2 {
3 private:
4     sf::CircleShape shape;
5     std::mt19937 generator;
6
7 public:
8     Coin();
9
10    void respawn(const sf::RenderWindow& window);
11    void draw(sf::RenderWindow& window);
12 };

```

The constructor may initialize the generator:

```

1 Coin::Coin()
2     : generator(std::random_device{}())
3 {
4     shape.setRadius(10.0f);
5     shape.setFillColor(sf::Color::Yellow);
6 }

```

The respawn method may then use distributions:

```

1 void Coin::respawn(const sf::RenderWindow& window)
2 {
3     float maxX =
4         static_cast<float>(window.getSize().x)
5         - shape.getRadius() * 2.0f;
6
7     float maxY =
8         static_cast<float>(window.getSize().y)
9         - shape.getRadius() * 2.0f;
10
11    std::uniform_real_distribution<float>
12        xDist(0.0f, maxX);
13
14    std::uniform_real_distribution<float>
15        yDist(0.0f, maxY);
16
17    float x = xDist(generator);
18    float y = yDist(generator);
19
20    shape.setPosition({x, y});
21 }

```

Every call to:

```

1 coin.respawn(window);

```

uses the generator's current internal state and advances it while producing new random coordinates.

A.28 Understanding Constructor Initializer Lists

Consider:

```

1 Coin::Coin()
2   : generator(std::random_device{}())
3 {
4   // Initialize coin
5 }

```

Question

What is the `: generator(...)` after the constructor parentheses but before the constructor body?

This syntax is called a **constructor member initializer list**.

The part:

```

1 : generator(std::random_device{}())

```

means:

Directly construct the member variable `generator` using the result of `std::random_device{}()` as its seed argument.

A.29 What Happens Before the Constructor Body?

Suppose:

```

1 class Coin
2 {
3 private:
4     sf::CircleShape shape;
5     std::mt19937 generator;
6
7 public:
8     Coin();
9 };

```

When:

```

1 Coin coin;

```

is executed, C++ calls the constructor.

However, before the constructor body begins, the object's data members must already be initialized.

Conceptually:

Create Coin object

|

v

Construct shape

|

v

Construct generator

|

v

Enter constructor body { }

Therefore:

```
1 Coin::Coin()
2   : generator(std::random_device{}())
3   {
4   }
```

constructs `generator` before execution enters the braces.

A.30 Breaking Down `std::random_device{}()`

The expression:

```
1 std::random_device{}
```

creates a temporary `std::random_device` object.

Then:

```
1 std::random_device{}()
```

calls the temporary object's function-call operator.

It is conceptually similar to:

```
1 std::random_device rd;
2
3 unsigned int seed = rd();
4
5 std::mt19937 generator(seed);
```

Therefore:

```
1 Coin::Coin()
2   : generator(std::random_device{}())
3   {
4   }
```

can be understood as the following sequence:

1. Create a temporary `std::random_device`.
2. Call it to obtain a seed.
3. Pass that seed to the `std::mt19937` constructor.
4. Construct the `generator` member.
5. Enter the body of the `Coin` constructor.

A.31 Why Not Assign Inside the Constructor Body?

One might instead write:

```
1 Coin::Coin()
2 {
3     generator =
4         std::mt19937(std::random_device{}());
5 }
```

For this particular assignable type, such code may work, but it is a different process. With the initializer list:

```

1 Coin::Coin()
2   : generator(std::random_device{}())
3 {
4 }

```

the generator is directly constructed with the desired seed.
Conceptually:

Direct initialization:

```
construct generator(seed)
```

With assignment in the body:

Assignment in constructor body:

```

default-construct generator
      |
      v
enter constructor body
      |
      v
create another seeded value
      |
      v
assign it to generator

```

Direct initialization is often clearer and can avoid unnecessary work.

A.32 Simple Initializer-List Example

Consider:

```

1 class Player
2 {
3 private:
4     int health;
5
6 public:
7     Player();
8 };

```

The constructor can be defined as:

```

1 Player::Player()
2   : health(100)
3 {
4 }

```

This means:

When constructing a `Player`, directly initialize the member variable `health` with the value 100.

Multiple members can be initialized:

```
1 class Player
2 {
3 private:
4     float speed;
5     int health;
6     std::string name;
7
8 public:
9     Player();
10};
```

Then:

```
1 Player::Player()
2     : speed(5.0f),
3       health(100),
4       name("Vignesh")
5 {
6 }
```

The general form is:

```
1 ClassName::ClassName()
2     : member1(value1),
3       member2(value2),
4       member3(value3)
5 {
6     // Constructor body
7 }
```

A.33 Initializer Lists with Constructor Parameters

Consider:

```
1 class Player
2 {
3 private:
4     float speed;
5     int health;
6
7 public:
8     Player(float initialSpeed, int initialHealth);
9};
```

The constructor can be:

```
1 Player::Player(
2     float initialSpeed,
3     int initialHealth
4 )
5     : speed(initialSpeed),
6       health(initialHealth)
7 {
8 }
```

Then:

```
1 Player player(5.0f, 100);
```

initializes:

```
speed  = 5.0
health = 100
```

It is also common for parameters and members to have the same names:

```
1 class Player
2 {
3 private:
4     float speed;
5     int health;
6
7 public:
8     Player(float speed, int health);
9 };
```

Then:

```
1 Player::Player(float speed, int health)
2     : speed(speed),
3       health(health)
4 {
5 }
```

Although:

```
1 speed(speed)
```

looks unusual, it means:

Initialize the member variable `speed` using the constructor parameter named `speed`.

A.34 When Are Initializer Lists Mandatory?

Initializer lists are not merely stylistic. In some cases they are required.

A.34.1 Const Data Members

Consider:

```
1 class Example
2 {
3 private:
4     const int value;
5
6 public:
7     Example();
8 };
```

This is invalid:

```
1 Example::Example()
2 {
3     value = 10; // Error
4 }
```

By the time execution reaches the constructor body, the member must already have been initialized. Since it is `const`, it cannot then be assigned another value.

The correct form is:


```
1 Example::Example()  
2     : value(10)  
3 {  
4 }
```

A.34.2 Reference Members

References must also be initialized:

```
1 class Example  
2 {  
3 private:  
4     int& reference;  
5  
6 public:  
7     Example(int& x);  
8 };
```

Then:

```
1 Example::Example(int& x)  
2     : reference(x)  
3 {  
4 }
```

Initializer lists are also necessary when a member object has no usable default constructor and must be constructed with specific arguments.

A.35 Member Initialization Order

Consider:

```
1 class Example  
2 {  
3 private:  
4     int x;  
5     int y;  
6  
7 public:  
8     Example();  
9 };
```

Now suppose the constructor is written as:

```
1 Example::Example()  
2     : y(20),  
3       x(10)  
4 {  
5 }
```

It may appear that `y` is initialized first because it is listed first.

However, C++ initializes data members in the order in which they are declared inside the class.

Since the class declares:

```
1 int x;  
2 int y;
```

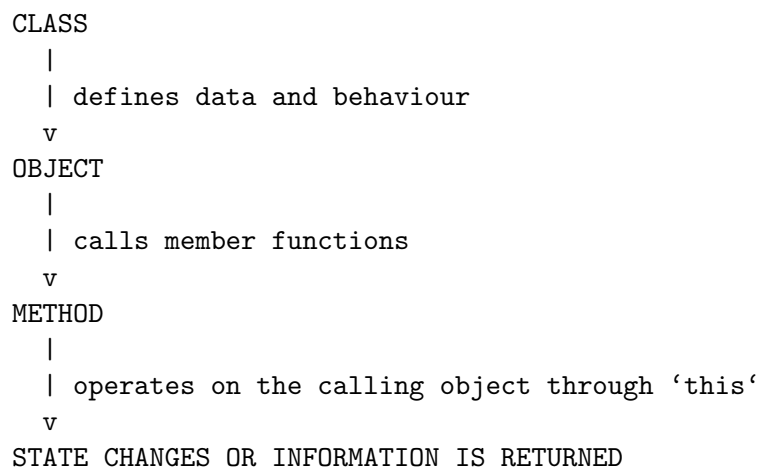
the actual initialization order is:

1. x
2. y

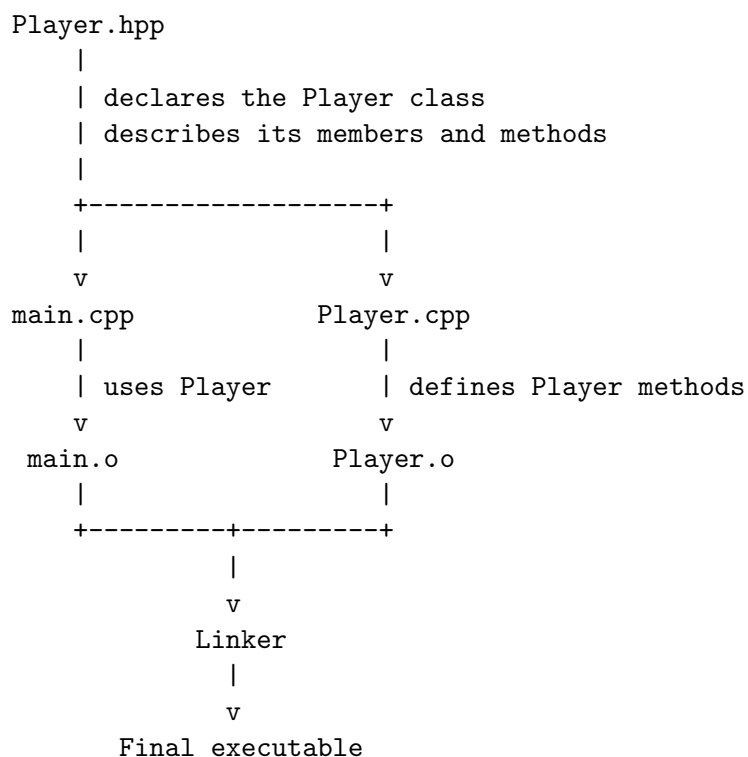
Therefore, good practice is to write the initializer list in the same order as the member declarations.

A.36 Complete Mental Model

The major concepts discussed in this appendix fit together as follows:



For multi-file programs:



For random number generation:

```

Seed
|

```

```

    v
std::mt19937 engine
    |
    v
Raw pseudo-random values
    |
    v
Distribution
    |
    v
Useful values in desired range
    |
    v
Random coin position

```

And for construction:

```

Create object
    |
    v
Initialize members
    |
    v
Run constructor body
    |
    v
Object is ready for use

```

The most important principles to retain are:

1. A class defines a new type; an object is an instance of that type.
2. Each object has its own non-static member data, while member-function definitions describe operations that can act on objects.
3. A non-static member function knows which object invoked it through the hidden `this` pointer.
4. Header files usually describe declarations and interfaces, while source files usually contain implementation definitions.
5. Each source file is compiled independently, and the linker later connects references to their definitions.
6. The final `const` on a member function promises not to modify the ordinary state of the object through that member function.
7. A pseudo-random engine such as `std::mt19937` produces a deterministic sequence controlled by its seed.
8. A distribution converts the engine's raw randomness into useful values, such as floating-point coordinates within the game window.
9. A constructor initializer list directly initializes members before the constructor body begins.

10. Some members, particularly references and certain const or non-default-constructible members, must be initialized through an initializer list or an equivalent in-class initialization mechanism.

This understanding provides the C++ foundation required to move from a simple single-file SFML program to a structured architecture containing `Player`, `Coin`, `Game`, and eventually a custom reinforcement-learning environment.

B Smart Pointers, `std::unique_ptr`, and Window Ownership

While designing the reinforcement-learning game environment, the following member appeared:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

This raises several important C++ questions. What exactly is a `unique_ptr`? How is it different from a normal object or a raw pointer? What does ownership mean? Why is `std::make_unique` used? Most importantly, why is the SFML window in the reinforcement-learning environment stored using a `unique_ptr`?

This section develops these concepts from first principles.

B.1 What Is `std::unique_ptr<Type>`?

Question

What is `std::unique_ptr<Type> object;`? How is it used, and why is it used in the game environment to create the SFML window?

The general syntax is:

```
1 std::unique_ptr<Type> object;
```

For example:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

This means:

`window` is a smart pointer that can exclusively own an object of type `sf::RenderWindow`.

The declaration can be broken down as:

```
std::unique_ptr < sf::RenderWindow > window;
|               |               |
|               |               +-- variable name
|               |
|               +-- type of owned object
|
+-- smart-pointer type
```

The `unique_ptr` type is provided by the standard library header:

```
1 #include <memory>
```

A `unique_ptr` is called a **smart pointer** because it provides pointer-like access to another object while automatically managing the lifetime of the object that it owns.

B.2 Starting with an Ordinary Object

Before understanding smart pointers, consider a normal SFML window object:

```
1 sf::RenderWindow window(
2     sf::VideoMode({800, 600}),
3     "My Game"
4 );
```

Here, `window` is the actual `sf::RenderWindow` object itself.

Conceptually:

```
window
+-----+
| sf::RenderWindow object |
|                           |
| width  = 800              |
| height = 600              |
| title  = "My Game"        |
| ...                      |
+-----+
```

Methods are accessed using the dot operator:

```
1 window.isOpen();
2 window.clear();
3 window.display();
```

The important point is that `window` directly contains the object. It is not a pointer to some other object.

B.3 What Is a Raw Pointer?

Instead of directly storing an object, C++ can store the address of an object:

```
1 sf::RenderWindow* window;
```

Now `window` is not itself a `RenderWindow`. It is a pointer capable of storing the memory address of a `RenderWindow`.

Conceptually:

```
window
+-----+
| 0x7AF320 |
+-----+
|           |
| points to  |
|           |
v
+-----+
| sf::RenderWindow object |
|                           |
| width  = 800              |
| height = 600              |
| ...                      |
+-----+
```

An object can be dynamically allocated using `new`:

```

1 sf::RenderWindow* window =
2   new sf::RenderWindow(
3     sf::VideoMode({800, 600}),
4     "My Game"
5   );

```

Because `window` is a pointer, methods are accessed using the arrow operator:

```

1 window->isOpen();
2 window->clear();
3 window->display();

```

The expression:

```
1 window->isOpen();
```

is essentially equivalent to:

```
1 (*window).isOpen();
```

The expression:

```
1 *window
```

dereferences the pointer and accesses the actual object to which it points.

B.4 The Problem with Raw Pointers and `new`

Suppose an object is created dynamically:

```

1 sf::RenderWindow* window =
2   new sf::RenderWindow(
3     sf::VideoMode({800, 600}),
4     "My Game"
5   );

```

The programmer is responsible for eventually destroying that object:

```
1 delete window;
```

If the object is never deleted:

```

1 sf::RenderWindow* window =
2   new sf::RenderWindow(...);
3
4 // Use the window...
5
6 // Forgot:
7 // delete window;

```

then the program may suffer from a **memory leak**.

A memory leak occurs when dynamically allocated memory is no longer properly released. Manual memory management also introduces several risks:

- forgetting to call `delete`,
- deleting the same object twice,
- using a pointer after its object has been destroyed,
- losing the address of dynamically allocated memory,

- exceptions or early returns accidentally skipping cleanup,
- unclear ownership between different parts of a program.

Modern C++ therefore generally prefers automatic resource management and smart pointers over explicit use of `new` and `delete`.

B.5 Enter `std::unique_ptr`

Instead of writing:

```
1 sf::RenderWindow* window;
```

we can write:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

Initially, this smart pointer may own nothing:

```
window
+-----+
| nullptr          |
| owns no object   |
+-----+
```

Later, an actual window can be dynamically created:

```
1 window = std::make_unique<sf::RenderWindow>(
2     sf::VideoMode({800, 600}),
3     "My Game"
4 );
```

Now the conceptual relationship is:

```
window
+-----+
| std::unique_ptr   |
| exclusive owner   |
+-----+-----+
|                   |
|                   | owns
|                   | v
+-----+-----+
| sf::RenderWindow object |
|                           |
| width  = 800             |
| height = 600             |
| title  = "My Game"       |
+-----+-----+
```

The crucial advantage is that when the `unique_ptr` is destroyed, the object it owns is automatically destroyed as well.

Therefore, there is normally no need to manually write:

```
1 delete window;
```

B.6 Why Is It Called `unique_ptr`?

The word `unique` refers to **unique ownership**.

At any given moment, one `unique_ptr` exclusively owns its dynamically allocated object.

For example:

```
1 std::unique_ptr<Player> player1 =
2   std::make_unique<Player>();
```

The following is not allowed:

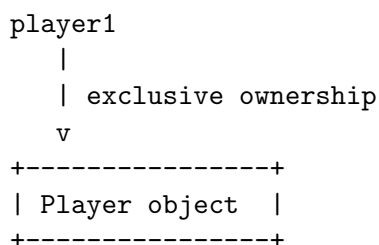
```
1 std::unique_ptr<Player> player2 = player1;
```

If both pointers owned the same object, an important question would arise:

Which pointer is responsible for destroying the object?

If both attempted to destroy it, the program would attempt to delete the same object twice. Therefore, copying a `unique_ptr` is prohibited.

The ownership model is:



There is exactly one owner.

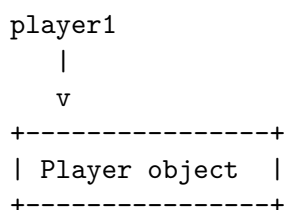
B.7 Transferring Ownership with `std::move`

Although a `unique_ptr` cannot be copied, its ownership can be transferred.

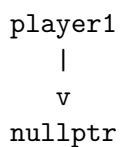
For example:

```
1 std::unique_ptr<Player> player1 =
2   std::make_unique<Player>();
3
4 std::unique_ptr<Player> player2 =
5   std::move(player1);
```

Before the move:



After the move:




```

player2
  |
  v
+-----+
| Player object |
+-----+

```

Ownership has been transferred from `player1` to `player2`.

The object itself does not need to be duplicated. Instead, responsibility for its lifetime moves to the new owner.

B.8 What Does `std::make_unique` Do?

Consider:

```

1 std::unique_ptr<Player> player =
2   std::make_unique<Player>();

```

The expression:

```

1 std::make_unique<Player>()

```

dynamically constructs a `Player` object and returns a `unique_ptr` that owns it.

If the constructor takes arguments:

```

1 class Player
2 {
3 public:
4   Player(float x, float y);
5 };

```

then:

```

1 auto player =
2   std::make_unique<Player>(100.0f, 200.0f);

```

constructs the object using:

```

1 Player(100.0f, 200.0f)

```

and returns exclusive ownership through a `unique_ptr`.

The general pattern is:

```

1 std::make_unique<Type>(
2   constructorArgument1,
3   constructorArgument2,
4   ...
5 );

```

A useful interpretation is:

Dynamically construct an object of `Type` using these constructor arguments, and return a `unique_ptr` that exclusively owns the new object.

B.9 How Is a `unique_ptr` Used?

Suppose:

```
1 std::unique_ptr<Player> player =  
2     std::make_unique<Player>();
```

Because `player` behaves like a pointer, its object's methods are accessed using the arrow operator:

```
1 player->move();  
2 player->update();  
3 player->draw();
```

Suppose:

```
1 class Player  
2 {  
3 public:  
4     void move()  
5     {  
6         std::cout << "Moving\n";  
7     }  
8 };
```

Then:

```
1 player->move();
```

calls `move()` on the `Player` object owned by the smart pointer.

Conceptually:

```
1 player->move();
```

is equivalent to:

```
1 (*player).move();
```

B.10 A `unique_ptr` Can Be Empty

A very important property of `unique_ptr` is that it does not always have to own an object.

Consider:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

At this point, no actual `sf::RenderWindow` has been created.

Conceptually:

```
window == nullptr
```

This can be checked as:

```
1 if (window)  
2 {  
3     // A RenderWindow currently exists.  
4 }
```

Or:

```
1 if (!window)  
2 {  
3     // No RenderWindow currently exists.  
4 }
```

Later, the actual object can be created:

```
1 window = std::make_unique<sf::RenderWindow>(
2     sf::VideoMode({800, 600}),
3     "My Game"
4 );
```

The owned object can also be explicitly destroyed:

```
1 window.reset();
```

After this:

```
1 window == nullptr
```

is true again.

Thus a `unique_ptr` naturally represents two possible states:

State 1:

No object exists.
`window == nullptr`

State 2:

An object exists.
`window` exclusively owns it.

B.11 Why Use a `unique_ptr` for the Game Window?

This question is especially important in the reinforcement-learning project.

The game environment has two different use cases:

Use case 1:

Human gameplay or agent visualization
 |
 v
 Window needed

Use case 2:

Reinforcement-learning training
 |
 v
 Window not needed

During RL training, an agent may execute:

- thousands of episodes,
- millions of environment steps,
- training on a remote server such as Kaggle,
- training without a graphical desktop or display.

Therefore, the logical game environment should not require an SFML window merely to calculate:

- player movement,
- coin movement,
- collisions,
- rewards,
- observations,
- episode termination.

The window should be optional.

A possible environment class is:

```

1 class GameEnvironment
2 {
3 private:
4     int windowHeight;
5     int windowHeight;
6
7     std::unique_ptr<sf::RenderWindow> window;
8
9 public:
10    GameEnvironment(int width, int height);
11
12    void createWindow();
13    void closeWindow();
14 };

```

When the environment is initially constructed:

```

1 GameEnvironment env(800, 600);

```

its state may be:

```

GameEnvironment
|
|-- windowHeight = 800
|-- windowHeight = 600
'-- window      = nullptr

```

No graphical window exists.

The environment can still execute logical operations:

```

1 env.reset();
2 env.step(action);
3 env.getObservation();

```

This is called **headless execution**: the game logic runs without opening or rendering a graphical window.

This is particularly useful for reinforcement learning.

B.12 Creating the Window Only When Needed

When visualization is required, a method can create the actual SFML window:

```

1 void GameEnvironment::createWindow()
2 {
3     window = std::make_unique<sf::RenderWindow>(
4         sf::VideoMode({
5             static_cast<unsigned int>(windowWidth),
6             static_cast<unsigned int>(windowHeight)
7         }),
8         "RL Game Environment"
9     );
10 }

```

Before calling `createWindow()`:

```

window
|
v
nullptr

```

After calling:

```

1 env.createWindow();

```

the relationship becomes:

```

GameEnvironment
|
|-- windowWidth  = 800
|-- windowHeight = 600
|
'-- window
    |
    | exclusive ownership
    v
    +-----+
    | sf::RenderWindow object |
    |                          |
    | width  = 800             |
    | height = 600             |
    +-----+

```

This is an example of **optional ownership**. The `GameEnvironment` may or may not currently own a window.

B.13 Rendering with an Optional Window

A render method can first check whether a window exists:

```

1 void GameEnvironment::render()
2 {
3     if (!window)
4     {
5         return;
6     }
7
8     window->clear();
9

```

```
10 player.draw(*window);
11 coin.draw(*window);
12
13 window->display();
14 }
```

The check:

```
1 if (!window)
```

means:

If the smart pointer does not currently own a window, stop rendering and return.

This prevents the program from trying to dereference a null smart pointer.

B.14 Understanding `window->clear()`

Suppose:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

and the pointer currently owns an actual window.

Then:

```
1 window->clear();
```

means:

Call the `clear()` method on the `sf::RenderWindow` object owned by `window`.

It is essentially equivalent to:

```
1 (*window).clear();
```

The arrow operator combines dereferencing and member access.

Thus:

```
1 window->display();
```

is conceptually equivalent to:

```
1 (*window).display();
```

B.15 Why Is `*window` Passed to `draw()`?

Suppose the player's draw function is declared as:

```
1 void Player::draw(sf::RenderWindow& window);
```

This function expects:

```
1 sf::RenderWindow&
```

which is a reference to an actual `sf::RenderWindow` object.

However, the environment stores:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

The variable `window` is a smart pointer, not the actual window object.

Therefore:

```
1 *window
```

dereferences the smart pointer and gives access to the actual `sf::RenderWindow` object.
Thus:

```
1 player.draw(*window);
```

means:

Obtain the actual `sf::RenderWindow` owned by the smart pointer and pass that window object by reference to `Player::draw()`.

The relationship can be summarized as:

```

window
|
+-- std::unique_ptr<sf::RenderWindow>
|
+-- smart pointer itself

*window
|
+-- actual sf::RenderWindow object

window->clear()
|
+-- call clear() on the owned object

player.draw(*window)
|
+-- pass the owned RenderWindow object by reference

```

B.16 Why Not Simply Store the Window Directly?

One could ask why the environment does not simply contain:

```
1 sf::RenderWindow window;
```

This can be perfectly valid in a program where the window should always exist for the full lifetime of the environment.

The choice depends on the desired architecture.

A direct member:

```
1 sf::RenderWindow window;
```

conceptually means:

The window is directly part of the environment object.

A smart pointer:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

conceptually means:

The environment may exclusively own a dynamically created window, or it may own no window at all.

For the reinforcement-learning architecture, the desired behavior is:

Training mode:

No graphical window required.

Visualization mode:

Create a graphical window.

Return to headless mode:

Destroy the window if desired.

A `unique_ptr` naturally represents these states.

B.17 Why Not Use a Raw Pointer for the Optional Window?

Technically, the following could be written:

```
1 sf::RenderWindow* window = nullptr;
```

and later:

```
1 window = new sf::RenderWindow(...);
```

However, the programmer would then be responsible for eventually writing:

```
1 delete window;
```

With:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

cleanup happens automatically.

For example:

```
1 {
2     GameEnvironment env(800, 600);
3
4     env.createWindow();
5
6 } // env is destroyed here
```

When `env` is destroyed:

1. The `GameEnvironment` object begins destruction.
2. Its member variables are destroyed.
3. The window unique pointer is destroyed.
4. The `sf::RenderWindow` owned by it is automatically destroyed.

No explicit:

```
1 delete window;
```

is required.

B.18 RAII: Resource Acquisition Is Initialization

The automatic cleanup behavior of `unique_ptr` follows a fundamental modern C++ principle called:

RAII: Resource Acquisition Is Initialization

The central idea is that a resource's lifetime should be tied to the lifetime of an object.

For a `unique_ptr`:

```

unique_ptr constructed
    |
    v
resource is owned
    |
    v
unique_ptr remains alive
    |
    v
resource remains owned
    |
    v
unique_ptr destroyed
    |
    v
owned resource automatically destroyed

```

This makes cleanup deterministic and significantly reduces the risk of memory leaks.

RAII is not limited to memory. The same principle is used throughout C++ for resources such as:

- dynamically allocated memory,
- files,
- mutex locks,
- network sockets,
- graphics resources,
- operating-system handles.

B.19 Direct Object, Raw Pointer, and `unique_ptr` Compared

The three approaches can be compared as follows:

Declaration	Meaning	Lifetime management
<code>sf::RenderWindow window;</code>	The window object is stored directly.	Destroyed automatically when its containing scope or object ends.
<code>sf::RenderWindow* window;</code>	A raw pointer stores an address.	Depends on ownership. If created using <code>new</code> , manual cleanup may be required.
<code>std::unique_ptr<sf::RenderWindow> window;</code>	A raw pointer may exclusively own one window or own nothing.	Owned object is destroyed automatically.

B.20 Complete Lifecycle of the Window in the RL Environment

The complete lifecycle can be visualized as follows:

1. Construct environment

```
GameEnvironment env(800, 600);
```

|
v

```
window == nullptr  
No graphical window exists.
```

2. Train agent headlessly

```
env.reset();  
env.step(action);  
env.getObservation();
```

|
v

```
Game logic runs.  
No rendering required.
```

3. Request visualization

```
env.createWindow();
```

|
v

```
std::make_unique<sf::RenderWindow>(...)  
creates a window.
```

|
v

```
window exclusively owns the RenderWindow.
```

4. Render the game

```
window->clear();  
player.draw(*window);  
coin.draw(*window);  
window->display();
```

5. Destroy the window manually if desired

```
window.reset();
```

```
    |
    v
```

RenderWindow destroyed.

window becomes nullptr.

6. Or simply destroy the environment

GameEnvironment destroyed

```
    |
    v
```

unique_ptr member destroyed

```
    |
    v
```

owned RenderWindow automatically destroyed

B.21 Important Clarification: Is unique_ptr Mandatory?

No.

Using a `unique_ptr` is not mandatory merely because the object is an SFML window.

The correct choice depends on the ownership and lifetime requirements.

If the window should always exist:

```
1 sf::RenderWindow window;
```

may be simpler.

If the environment should optionally own a window:

```
1 std::unique_ptr<sf::RenderWindow> window;
```

is a reasonable design.

Another possible modern C++ approach, depending on the type and architecture, is:

```
1 std::optional<sf::RenderWindow> window;
```

The main architectural requirement in this reinforcement-learning project is:

The logical environment must be capable of running without a graphical window during training, while still allowing a window to be created when visualization is required.

The `unique_ptr` solution satisfies this requirement by representing optional, exclusive ownership.

B.22 Key Mental Model

The most important ideas can be summarized as follows:

```
std::unique_ptr<Type> pointer;
    |
```

v

A smart pointer that can:

- own exactly one Type object, or
- own nothing.

|

v

Construct a Type object dynamically
and return exclusive ownership.

pointer->method()

|

v

Call a method on the owned object.

*pointer

|

v

Access the actual owned object.

pointer.reset()

|

v

Destroy the owned object
and make the pointer empty.

unique_ptr destroyed

|

v

Owned object automatically destroyed.

For the specific game environment:

RL TRAINING

|

v

window == nullptr

|

v

Game logic runs headlessly
without graphical rendering.

WATCHING THE AGENT

|

v

```
createWindow()  
  |  
  v  
window owns an sf::RenderWindow  
  |  
  v  
Frames are rendered visually.
```

The essential conclusion is:

`std::unique_ptr<sf::RenderWindow> window;` does not itself create an SFML window. It creates a smart pointer that can either exclusively own one `sf::RenderWindow` or own nothing. In the reinforcement-learning game environment, this allows the same environment to run headlessly during training and create a graphical window only when visualization is actually required.